ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP (CO4347)

Ngành: KỸ THUẬT MÁY TÍNH

ỨNG DỤNG TRÍ TUỆ NHÂN TẠO ĐỂ TỰ ĐỘNG NHẬN DẠNG MỤC TIÊU TRONG ẢNH ỐNG NHÒM

GIẢNG VIÊN HƯỚNG DẪN: TS. VÕ TUẤN BÌNH

SINH VIÊN THỰC HIỆN:

PHẠM HÀ BẢO LONG	2211893
NGUYỄN PHƯƠNG NGUYÊN	2212311
LÊ HẢI NGUYÊN	2212294

Thành phố Hồ Chí Minh, Tháng 5 Năm 2026

Lời cam đoan

Chúng em xin cam đoan toàn bộ nội dung và kết quả của đề tài **"Ứng dụng trí tuệ nhân tạo để tự động nhận dạng mục tiêu trong ảnh ống nhòm"** là sản phẩm nghiên cứu, tìm hiểu và phát triển của riêng nhóm chúng tôi dưới sự hướng dẫn khoa học của **TS. Võ Tuấn Bình**.

Các kết quả, số liệu được trình bày trong báo cáo là hoàn toàn trung thực, phản ánh đúng quá trình nghiên cứu và thực nghiệm của nhóm.

Trong quá trình thực hiện đề tài, nhóm có tham khảo và kế thừa các tài liệu, sách báo và các nguồn tài liệu chuyên ngành liên quan đến xử lý ảnh, học sâu, YOLO và hệ thống AI trên thiết bị biên. Tất cả các tài liệu tham khảo đều được trích dẫn đầy đủ và rõ ràng theo đúng quy định học thuật.

Nếu phát hiện có bất kỳ sự gian lận, sao chép không hợp lệ hay vi phạm các nguyên tắc về liêm chính học thuật, chúng em xin hoàn toàn chịu trách nhiệm và chấp nhận mọi hình thức xử lý theo quy định của nhà trường.

TP. Hồ Chí Minh, tháng 05 năm 2026

Nhóm sinh viên thực hiện

Nguyễn Phương Nguyên
Phạm Hà Bảo Long
Lê Hải Nguyên

Lời cảm ơn

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến Trường Đại học Bách khoa – ĐHQG TP.HCM cùng quý thầy cô Khoa Khoa học và Kỹ thuật Máy tính đã tận tình giảng dạy, truyền đạt cho chúng em những kiến thức quý báu trong suốt quá trình học tập và rèn luyện tại trường. Những kiến thức nền tảng cũng như kinh nghiệm thực tiễn mà quý thầy cô mang lại là hành trang quan trọng giúp chúng em có thể thực hiện và hoàn thành đề tài này.

Đặc biệt, nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn **TS. Võ Tuấn Bình** đã tận tình hướng dẫn, hỗ trợ và tạo điều kiện thuận lợi cho nhóm trong suốt quá trình thực hiện đồ án. Thầy đã định hướng chuyên môn, đóng góp nhiều ý kiến, giúp nhóm từng bước hoàn thiện nội dung nghiên cứu cũng như kỹ năng triển khai thực tế. Sự tận tâm và những lời góp ý của thầy là nguồn động lực lớn giúp nhóm vượt qua những khó khăn trong quá trình thực hiện đề tài.

Mặc dù nhóm đã cố gắng hoàn thiện đề tài trong khả năng tốt nhất, tuy nhiên do hạn chế về thời gian, kinh nghiệm và kiến thức thực tiễn nên báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự thông cảm và những ý kiến đóng góp quý báu từ quý thầy cô để có thể tiếp tục hoàn thiện hơn trong tương lai.

Nhóm chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 05 năm 2026

Nhóm sinh viên thực hiện

Nguyễn Phương Nguyên
Phạm Hà Bảo Long
Lê Hải Nguyên

Mục lục

Lời cam đoan **i**

Lời cảm ơn **ii**

Danh sách hình vẽ **vii**

Danh sách bảng **x**

Danh mục từ viết tắt **xi**

1 Giới thiệu **1**

1.1 Yêu cầu, thách thức và mục đích nghiên cứu 2

1.1.1 Yêu cầu 2

1.1.2 Thách thức 2

1.1.3 Mục đích nghiên cứu 2

1.2 Định hướng nghiên cứu 3

1.2.1 Bối cảnh ứng dụng 3

1.2.2 Phạm vi – giả định – tuân thủ 3

1.2.3 Cấu trúc báo cáo 3

2 Cơ sở lí thuyết **5**

2.1 Cơ sở lý thuyết về Ống nhòm 5

2.1.1 Định nghĩa và lịch sử phát triển cơ bản 6

2.1.2 Chi tiết cấu tạo các bộ phận chính của ống nhòm 7

2.1.3 Nguyên lý Hoạt động Quang học 8

2.2 Ảnh số và quá trình số hoá 9

2.2.1 Cấu trúc cơ bản của ảnh số 10

2.2.2 Phân loại và không gian màu 10

2.3 Cơ sở lý thuyết về ảnh RGB và ảnh IR 10

2.3.1 Khái niệm cơ bản 10

2.3.2 Đặc trưng tín hiệu 11

2.3.3 So sánh RGB và IR 11

2.3.4 Ý nghĩa đối với bài toán nhận dạng 11

2.4 Cơ sở lý thuyết về xử lý ảnh số 12

2.4.1 Khái niệm chung 12

2.4.2 Các bước cơ bản trong xử lý ảnh 12

2.5 Thuật toán YOLO trong nhận dạng đối tượng 13
2.5.1 Giới thiệu 13
2.5.2 Nguyên lý hoạt động của YOLO 13
2.6 Thư viện PyTorch 14
2.6.1 Các đặc điểm nổi bật của PyTorch 14
2.6.2 Ưu điểm và Nhược điểm của PyTorch 15
2.6.3 Ứng dụng thực tiễn của PyTorch trong thị giác máy tính (Computer Vision) 16
2.7 Tổng quan về Deep Learning 16
2.7.1 Nguyên lý hoạt động của Deep Learning 17
2.7.2 Ưu điểm và nhược điểm của Deep Learning 18
2.7.3 Ứng dụng thực tiễn của Deep Learning trong thị giác máy tính (Computer Vision) 20
2.8 Shortcut learning 21
2.8.1 Hiện tượng quan sát được 21
2.8.2 Bản chất của Shortcut Learning 22
2.8.3 Phân tích rủi ro Shortcut Learning và Chiến lược Data Augmentation 23
2.9 Các chỉ số đánh giá mô hình AI 23
2.9.1 Ý nghĩa của TP, TN, FP, FN 23
2.9.2 Các chỉ số Precision, Recall và mAP 24

3 Phân tích và Thiết kế Kiến trúc Hệ thống 26
3.1 Kiến trúc phần cứng 26
3.1.1 Giải pháp 1 — CPU thuần (Raspberry Pi) 26
3.1.2 Giải pháp 2 — Tăng tốc Edge TPU (Raspberry Pi + Google Coral) 27
3.1.3 So sánh hai giải pháp phần cứng 28
3.2 Kiến trúc Dashboard giám sát từ xa 28
3.2.1 Tổng quan kiến trúc bốn vùng 29
3.2.2 Luồng dữ liệu Dashboard 30
3.2.3 Truy cập từ xa qua Internet 30
3.3 Kiến trúc xử lý AI 30
3.3.1 EdgeVisionRT — Pipeline suy luận C++/NCNN 30
3.3.2 CoralVisionRT — Pipeline suy luận Python/Edge TPU 32
3.3.3 So sánh hai nhánh xử lý AI 33

4 Dataset và phương pháp đánh giá 34
4.1 Tổng quan về Bộ dữ liệu C2A (Combination to Application) 34
4.1.1 Cấu trúc và Thành phần dữ liệu 34
4.1.2 Thống kê và Đặc điểm kỹ thuật 34
4.1.3 Hệ thống nhãn (Annotation) 35
4.1.4 Đánh giá dataset 36
4.2 Data augmentation 37
4.2.1 Cơ sở lựa chọn phương pháp tăng cường dữ liệu 37
4.2.2 Tổng quan nghiên cứu 37

4.2.3 Tăng cường dữ liệu dựa trên mô hình vật lý (Analytical Data Augmentation)‘ ... 37
4.2.4 Quy trình tính toán và Chuyển đổi không gian ... 38
4.2.5 Kết quả thực nghiệm ... 39
4.2.6 Mô phỏng hiệu ứng Sương mù (Fog Simulation) ... 41
4.2.7 Mô phỏng hiệu ứng Mưa (Rain Simulation) ... 43
4.3 So sánh hiệu quả sử dụng Depth trong Data Augmentation ... 45
4.3.1 Mục tiêu ... 45
4.3.2 Thiết lập thực nghiệm ... 45
4.3.3 Kết quả thực nghiệm ... 45
4.3.4 Phân tích ... 46
4.3.5 Kết luận ... 47
4.4 Chiến lược Phân tầng Dữ liệu (Stratified Sampling) ... 47
4.5 Nghiên cứu khả năng tổng quát hóa của mô hình trong bài toán SAR ... 49
4.5.1 Những hạn chế của C2A ... 49
4.5.2 Đề xuất giải pháp ... 50
4.6 Phương Pháp Tiếp Cận và Thiết Lập Đánh Giá ... 51
4.6.1 Trộn Dữ Liệu (Mixed-Dataset Strategy) ... 51
4.6.2 Đặc Tả Ma Trận Mô Hình (Model Matrix Specification) ... 51
4.6.3 Đặc tả các Tập Dữ Liệu Sử Dụng để train (Dataset Profiles) ... 52
4.7 Đánh giá khả năng tổng quát hóa và tăng cường thời tiết ... 57
4.7.1 Mối liên hệ giữa các Model và các Tập Dataset ... 57
4.7.2 Đánh giá trên miền dữ liệu gốc ... 57
4.7.3 Đánh giá chuyển giao sang dữ liệu thực (Seen Domain & Unseen Domain) ... 68
4.7.4 Đánh giá chung ... 83

5 Hiện thực và đánh giá hệ thống 85
5.1 Đánh giá hiệu năng thực nghiệm trên Raspberry Pi 5 ... 85
5.1.1 Mục tiêu ... 85
5.1.2 Dữ liệu và thiết lập thử nghiệm ... 85
5.1.3 Chỉ số đánh giá ... 85
5.1.4 Kết quả benchmark chế độ Video ... 86
5.1.5 Kết quả benchmark chế độ Ảnh ... 86
5.1.6 So sánh Video vs Ảnh ... 86
5.1.7 Kết luận ... 87
5.2 Đánh giá hiệu năng thực nghiệm và cải thiện hiệu suất khi chạy thuần CPU trên Raspberry Pi-4B ... 87
5.2.1 Mục tiêu ... 87
5.2.2 Kết quả ... 88
5.2.3 Phân tích Ý nghĩa của logic tăng tốc CPU tối ưu tầng thấp (Kernel & SIMD) ... 88
5.2.4 Kết luận ... 89

5.3 Phân tích lựa chọn kiến trúc và độ phân giải cho mô hình nhận dạng trên Edge TPU 89
5.3.1 Mục tiêu 89
5.3.2 Google Coral Edge TPU và Định Mức Phần Cứng (Hardware Constraints) 90
5.3.3 Ảnh Hưởng Của Chiều Phân Giải Đầu Vào (Input Resolution: 320, 416, 512, 640) 91
5.3.4 Tổng kết cơ sở so sánh cho quá trình thực nghiệm 92
5.4 Kiểm định thông số các mô hình trên EDGE TPU 92
5.4.1 Bảng tổng hợp kiểm định thông số các mô hình trên Edge TPU ... 92
5.4.2 Kết luận 93
5.5 Hệ thống Dashboard và giám sát realtime 94
5.5.1 Kiến trúc hệ thống dashboard 94
5.5.2 Backend 94
5.5.3 Cơ sở dữ liệu 95
5.5.4 Frontend dashboard 95
5.5.5 Luồng dữ liệu realtime 95
5.5.6 Kết quả hiển thị 95

6 Tổng kết và hướng phát triển **97**
6.1 Tổng kết 97
6.2 Hạn chế của đề tài 100
6.3 Hướng phát triển 101
6.3.1 Mở rộng và chuẩn hóa dữ liệu thực tế 101
6.3.2 Cải tiến mô hình phát hiện đối tượng 101
6.3.3 Tích hợp đa cảm biến và dữ liệu hồng ngoại 102
6.3.4 Tối ưu triển khai trên thiết bị biên 102
6.3.5 Hoàn thiện dashboard và khả năng vận hành hệ thống 103
6.3.6 Thử nghiệm thực địa và đánh giá độ tin cậy 103
TÀI LIỆU THAM KHẢO 104

Danh sách hình vẽ

2.1 Ống nhòm loại Galilean thời Thế chiến thứ nhất 6
2.2 Ống nhòm Galilean. 6
2.3 Cấu tạo tổng thể với chi tiết các bộ phận 7
2.4 Đường đi ánh sáng qua lăng kính Porro và Roof. 9
2.5 Đặc tính ảnh số 10
2.6 Lưu đồ mô tả quá trình xử lý ảnh 12
2.7 Sơ đồ luồng hoạt động của thuật toán YOLO 14
2.8 Kiến trúc mạng nơ-ron 18
2.9 Hình ảnh hiện tượng Shortcut Learning: Thiên kiến vùng tối (Vignette Bias) 22
2.10 Hiệu ứng shortcuts 22

3.1 Sơ đồ phần cứng của giải pháp CPU thuần 26
3.2 Sơ đồ phần cứng của giải pháp Edge TPU 27
3.3 Kiến trúc bốn vùng của hệ thống Dashboard giám sát 29
3.4 Kiến trúc pipeline xử lý AI của EdgeVisionRT (CPU) 31

4.1 Hình ảnh các thảm họa về hỏa hoạn, ngập lụt, công trình, giao thông 35
4.2 Hình ảnh các thảm họa về hỏa hoạn, ngập lụt, công trình, giao thông (có label) 36
4.3 Thống kê số lượng ảnh theo tập 36
4.4 Thống kê phân bố ảnh theo điều kiện trong dataset 36
4.5 Hình ảnh các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông 39
4.6 Hình ảnh Depth Map các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông 39
4.7 ảnh gốc và ảnh sau khi áp dụng hiệu ứng sương mù (chưa áp dụng Depth map) 40
4.8 Ảnh sau khi áp dụng Depth map và hiệu ứng sương mù 40
4.9 Hình ảnh các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông 42
4.10 Hình ảnh Depth Map các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông 42
4.11 Hình ảnh sau khi áp dụng Depth map và hiệu ứng sương mù các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông 42
4.12 Ảnh sau khi áp dụng Depth map và hiệu ứng mưa 43

4.13 Hình ảnh các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông 44
4.14 Hình ảnh Depth Map các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông 44
4.15 Hình ảnh sau khi áp dụng Depth map và hiệu ứngmưa. Các vệt mưa được tạo ra với độ nghiêng ngẫu nhiên để mô phỏng tác động của gió trong các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông 44
4.17 Hình minh họa 49
4.18 Hình ảnh bộ dữ liệu AIDER và bộ dữ liệu LSP/MPII-MPHB 49
4.19 Hình ảnh các thảm họa về hỏa hoạn, ngập lụt, giao thông, công trình 49
4.20 So sánh khả năng tổng quát hóa của hai mô hình. 50
4.21 Hình ảnh Dataset VisDrone chưa qua augment 53
4.22 Hình ảnh Dataset VisDrone đã qua augment 53
4.23 Hình ảnh Dataset TinyPerson chưa qua augment 53
4.24 Hình ảnh Dataset TinyPerson đã qua augment 53
4.25 Hình ảnh Dataset CrowdHuman chưa qua augment 54
4.26 Hình ảnh Dataset CrowdHuman đã qua augment 54
4.27 Hình ảnh Dataset SARD 55
4.28 Hình ảnh Dataset Heridal 56
4.29 Hiệu suất các mô hình trên tập dữ liệu T1_C2A_Origina 58
4.30 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 59
4.31 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 60
4.32 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 61
4.33 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 62
4.34 Hiệu suất các mô hình trên tập dữ liệu T1_C2A_Weather 63
4.35 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 64
4.36 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 65
4.37 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 66
4.38 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 67
4.39 Hiệu suất các mô hình trên tập dữ liệu T2_VisDrone 68
4.40 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 69
4.41 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 70
4.42 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 71
4.43 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 72
4.44 Hiệu suất các mô hình trên tập dữ liệu T3_Heridal 73
4.45 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 74
4.46 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 75
4.47 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 76
4.48 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 77
4.49 Hiệu suất các mô hình trên tập dữ liệu T3_SARD 78
4.50 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 79
4.51 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 80
4.52 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 81
4.53 Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất 82

4.54 Biểu đồ Radar so sánh trung bình các chỉ số đánh giá toàn diện (mAP@50, mAP@50-95, AP75, Precision, Recall, F1-Score) của 4 mô hình trên toàn bộ các tập dữ liệu 83
4.55 Biểu đồ Heatmap thể hiện sự biến thiên của chỉ số mAP@50 qua các tập dữ liệu. 84

5.1 Hình ảnh các thảm họa về ngập lụt - 1 87
5.2 Hình ảnh các thảm họa về ngập lụt - 2 87
5.3 So sánh trực quan kết quả phát hiện đối tượng giữa hai cấu hình: CPU đơn luồng (baseline) và CPU đa luồng tối ưu trên Raspberry Pi 4B 88
5.4 Giao diện dashboard giám sát realtime của hệ thống 96

Danh sách bảng

2.1 Cấu tạo và chức năng các bộ phận chính của ống nhòm 8
2.2 So sánh RGB và ảnh hồng ngoại 11

3.1 So sánh hai giải pháp phần cứng 28
3.2 So sánh kiến trúc EdgeVisionRT và CoralVisionRT 33

4.1 So sánh hiệu năng giữa mô hình có và không sử dụng depth trong data augmentation 45
4.2 So sánh hiệu quả sử dụng depth 47
4.3 Kết quả đánh giá chi tiết các mô hình trên các tập dữ liệu 57

5.1 So sánh hiệu năng giữa hai chế độ chạy. 88
5.2 Hiệu năng các mô hình tại độ phân giải 320x320 92
5.3 Hiệu năng các mô hình tại độ phân giải 416x416 93
5.4 Hiệu năng các mô hình tại độ phân giải 512x512 93
5.5 Hiệu năng các mô hình tại độ phân giải 640x640 93

Danh mục từ viết tắt

Từ viết tắt	Thuật ngữ tiếng Anh	Giải nghĩa tiếng Việt
AE	Auto Exposure	Tự động phơi sáng
AGC	Automatic Gain Control	Tự động điều chỉnh độ lợi
AI	Artificial Intelligence	Trí tuệ nhân tạo
AIDER	Aerial Image Dataset for Emergency Response Applications	Bộ dữ liệu ảnh trên không ứng phó khẩn cấp
AOT	Ahead-Of-Time	Biên dịch trước khi thực thi
AP	Average Precision	Độ chính xác trung bình
API	Application Programming Interface	Giao diện lập trình ứng dụng
AR	Augmented Reality	Thực tế tăng cường
ARM	Advanced RISC Machine	Kiến trúc vi xử lý ARM
ASIC	Application-Specific Integrated Circuit	Vi mạch tích hợp chuyên dụng
AWB	Auto White Balance	Tự động cân bằng trắng
C2A	Combination to Application	Tên bộ dữ liệu thảm họa
CHW	Channel – Height – Width	Định dạng tensor (Kênh, Cao, Rộng)
CMYK	Cyan – Magenta – Yellow – Key (Black)	Không gian màu in ấn
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
COCO	Common Objects in Context	Bộ dữ liệu chuẩn đánh giá
CPU	Central Processing Unit	Bộ xử lý trung tâm
CSI	Camera Serial Interface	Giao tiếp nối tiếp cho camera
DIP	Digital Image Processing	Xử lý ảnh số
DMA	Direct Memory Access	Truy cập bộ nhớ trực tiếp

Từ viết tắt	Thuật ngữ tiếng Anh	Giải nghĩa tiếng Việt
DPT	Dense Prediction Transformer	Kiến trúc Transformer dự đoán dày đặc
DSI	Display Serial Interface	Giao tiếp nối tiếp hiển thị
EDID	Extended Display Identification Data	Dữ liệu nhận dạng màn hình mở rộng
EMI	Electromagnetic Interference	Nhiễu điện từ
FAIR	Facebook AI Research	Bộ phận nghiên cứu AI của Facebook
FFC	Flexible Flat Cable	Cáp dẹt linh hoạt
FLOPs	Floating Point Operations Per Second	Số phép toán dấu phẩy động mỗi giây
FN	False Negative	Âm tính giả
FP	False Positive	Dương tính giả
FPGA	Field-Programmable Gate Array	Mảng cổng lập trình được
FPS	Frames Per Second	Số khung hình trên mỗi giây
GAN	Generative Adversarial Network	Mạng đối nghịch sinh
GIL	Global Interpreter Lock	Khóa thông dịch toàn cục (Python)
GPIO	General-Purpose Input/Output	Cổng vào/ra đa dụng
GPS	Global Positioning System	Hệ thống định vị toàn cầu
HDMI	High-Definition Multimedia Interface	Giao tiếp đa phương tiện độ nét cao
HERIDAL	Human Rescue Imaging Database on Land	Cơ sở dữ liệu ảnh cứu hộ trên đất liền
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HUD	Head-Up Display	Hiển thị thông tin trên màn hình
IDE	Integrated Development Environment	Môi trường phát triển tích hợp
INT8	8-bit Integer Quantization	Lượng tử hóa số nguyên 8-bit

Từ viết tắt	Thuật ngữ tiếng Anh	Giải nghĩa tiếng Việt
IoU	Intersection over Union	Chỉ số đánh giá độ chồng lấp
IR	Infrared	Hồng ngoại
ISO	International Organization for Standardization	Tổ chức Tiêu chuẩn hóa Quốc tế
JIT	Just-In-Time	Biên dịch tức thời
LCD	Liquid Crystal Display	Màn hình tinh thể lỏng
LSTM	Long Short-Term Memory	Bộ nhớ dài–ngắn hạn
LWIR	Long-Wave Infrared	Hồng ngoại sóng dài
mAP	mean Average Precision	Trung bình độ chính xác toàn phần
MIPI	Mobile Industry Processor Interface	Giao diện bộ xử lý công nghiệp di động
MJPEG	Motion JPEG	Chuẩn nén video JPEG liên tục
MMAP	Memory-Mapped I/O	Ánh xạ bộ nhớ vào/ra
MPHB	Multi-Person Human Body	Bộ dữ liệu tư thế cơ thể người
MRI	Magnetic Resonance Imaging	Chụp cộng hưởng từ
MWIR	Mid-Wave Infrared	Hồng ngoại sóng trung
NCNN	Tencent NCNN Framework	Framework suy luận mạng nơ-ron
NEON	ARM Advanced SIMD Extension	Tập lệnh SIMD mở rộng của ARM
NIR	Near-Infrared	Hồng ngoại gần
NMS	Non-Maximum Suppression	Triệt tiêu không cực đại
NPU	Neural Processing Unit	Bộ xử lý mạng nơ-ron
ONNX	Open Neural Network Exchange	Định dạng trao đổi mạng nơ-ron mở
ORM	Object-Relational Mapping	Ánh xạ đối tượng – quan hệ

Từ viết tắt	Thuật ngữ tiếng Anh	Giải nghĩa tiếng Việt
PCIe	Peripheral Component Interconnect Express	Chuẩn kết nối linh kiện ngoại vi
PWM	Pulse Width Modulation	Điều chế độ rộng xung
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
R-CNN	Region-based Convolutional Neural Network	Mạng nơ-ron tích chập theo vùng
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
RTSP	Real Time Streaming Protocol	Giao thức truyền phát thời gian thực
SAR	Search and Rescue	Tìm kiếm và cứu nạn
SARD	Search And Rescue Dataset	Bộ dữ liệu tìm kiếm cứu nạn
SDL	Simple DirectMedia Layer	Thư viện đa phương tiện
SIMD	Single Instruction, Multiple Data	Một lệnh, nhiều dữ liệu
SRAM	Static Random Access Memory	Bộ nhớ truy cập ngẫu nhiên tĩnh
TFLite	TensorFlow Lite	Phiên bản nhẹ của TensorFlow
TFT	Thin Film Transistor	Bóng bán dẫn màng mỏng
TN	True Negative	Âm tính thật
TOPS	Tera Operations Per Second	Nghìn tỷ phép tính mỗi giây
TP	True Positive	Dương tính thật
TPU	Tensor Processing Unit	Bộ xử lý tensor
UAV	Unmanned Aerial Vehicle	Máy bay không người lái
URL	Uniform Resource Locator	Định vị tài nguyên thống nhất
V4L2	Video4Linux 2	API video cho hệ điều hành Linux
YOLO	You Only Look Once	Thuật toán nhận dạng đối tượng
YUYV	Y-U-Y-V (4:2:2 Packed Format)	Định dạng mã hóa video

Chương 1

Giới thiệu

Trong các tình huống tìm kiếm – cứu nạn (Search & Rescue – SAR) như lũ quét, sạt lở đất, cháy rừng, tai nạn trên biển hoặc người lạc trong rừng núi, việc nhanh chóng phát hiện và định vị nạn nhân là yếu tố sống còn. Lực lượng cứu hộ hiện nay chủ yếu dựa vào quan sát trực tiếp, tuần tra thủ công, kết hợp với các thiết bị hỗ trợ như ống nhòm quang học hoặc camera giám sát. Tuy nhiên, khi phải quan sát liên tục trong thời gian dài và trong các điều kiện môi trường bất lợi (sương mù, khói bụi, mưa nhẹ...), con người dễ rơi vào trạng thái mệt mỏi, suy giảm khả năng tập trung, dẫn đến nguy cơ bỏ sót nạn nhân hoặc phản ứng chậm trễ.

Sự phát triển của thị giác máy tính và các mô hình học sâu (Deep Learning) cho phép tự động hóa quá trình phát hiện đối tượng trong ảnh và video. Đặc biệt, các mô hình nhận dạng thời gian thực như YOLO có khả năng phát hiện con người với tốc độ cao, phù hợp triển khai trên các nền tảng tính toán biên (edge computing) như Raspberry Pi. Tuy nhiên, trong bối cảnh SAR, dữ liệu quan sát thường bị ảnh hưởng bởi nhiều yếu tố bất lợi như điều kiện thời tiết (sương mù, mưa), ánh sáng yếu, hoặc góc nhìn phức tạp, gây khó khăn cho mô hình nhận dạng.

Đề tài này tập trung xây dựng một hệ thống hỗ trợ tìm kiếm – cứu nạn dựa trên camera quang học (RGB) kết hợp với trí tuệ nhân tạo, có khả năng tự động phát hiện con người trong thời gian thực. Hệ thống được triển khai trên nền tảng edge (Raspberry Pi), tích hợp các kỹ thuật tăng cường dữ liệu (data augmentation) nhằm mô phỏng điều kiện môi trường thực tế, đồng thời hỗ trợ giám sát từ xa thông qua dashboard. Giải pháp hướng tới vai trò như một "trợ lý quan sát thông minh", giúp nâng cao hiệu quả và độ tin cậy trong quá trình tìm kiếm – cứu nạn.

Tổng quát, dự án nghiên cứu gồm các bước chính:

1. Xây dựng và gán nhãn dữ liệu chuyên biệt.
2. Thiết kế–huấn luyện–hiệu chỉnh mô hình.
3. Tối ưu suy luận thời gian thực trên thiết bị nhúng/tính toán biên (edge) có tài nguyên hạn chế.
4. Tích hợp giao diện hiển thị thân thiện cho người dùng.

1.1 Yêu cầu, thách thức và mục đích nghiên cứu

1.1.1 Yêu cầu

- **Chỉ số đánh giá:** Đạt các chỉ số mAP (Mean Average Precision) và F1-score theo bộ dữ liệu/kịch bản; ưu tiên độ nhạy với mục tiêu nhỏ/xa.
- **Thời gian thực:** Duy trì tốc độ $\geq$ 15 FPS với độ trễ thấp, ổn định trong điều kiện nhiễu và biến thiên ánh sáng.
- **Hiển thị kết quả trực tiếp:** Vẽ bounding box, nhãn lớp, độ tin cậy ngay trên khung hình/clip từ ống nhòm; hỗ trợ ghi log/ảnh phục vụ đánh giá.
- **Triển khai thực tế:** Mô hình tối ưu kích thước và thông lượng để chạy trên thiết bị nhúng (ví dụ: SoC có GPU/NPU) lắp cùng ống nhòm hoặc máy tính đi kèm.

1.1.2 Thách thức

- **Môi trường khắc nghiệt:** Điều kiện quan sát thực tế như sương mù, mưa, ánh sáng yếu hoặc khói bụi làm suy giảm chất lượng ảnh (giảm độ tương phản, tăng nhiễu), gây khó khăn cho quá trình nhận dạng và làm giảm độ chính xác của mô hình.
- **Dịch miền dữ liệu (Domain Shift):** Sự khác biệt giữa dữ liệu huấn luyện (synthetic/augmented) và dữ liệu thực tế (drone, SAR) khiến mô hình khó tổng quát hóa, đặc biệt trong các kịch bản chưa từng xuất hiện (unseen domain).
- **Tài nguyên tính toán hạn chế trên edge:** Hệ thống triển khai trên Raspberry Pi yêu cầu mô hình phải nhẹ, tốc độ suy luận nhanh, tiêu thụ ít tài nguyên (CPU, RAM, điện năng), do đó cần tối ưu mô hình và pipeline xử lý.
- **Đặc thù đối tượng nhỏ và phức tạp:** Trong ảnh trên không hoặc khoảng cách xa, con người thường xuất hiện với kích thước nhỏ, dễ bị che khuất hoặc nhầm lẫn với nền, làm tăng độ khó của bài toán phát hiện.
- **Thu thập và gán nhãn dữ liệu:** Việc xây dựng dataset chất lượng cao cho bài toán SAR đòi hỏi đa dạng kịch bản, điều kiện môi trường và góc nhìn, đồng thời cần quá trình gán nhãn chính xác và nhất quán.

1.1.3 Mục đích nghiên cứu

- Xây dựng hệ thống nhận dạng mục tiêu tự động hỗ trợ quan sát, giám sát và nâng cao nhận thức tình huống trong các kịch bản có điều kiện nhìn hạn chế.
- Đóng góp vào việc ứng dụng AI trong lĩnh vực quốc phòng và an ninh.
- Chứng minh tính khả thi triển khai trên prototype nhúng với tốc độ thời gian thực và độ chính xác đủ dùng.

1.2 Định hướng nghiên cứu

1.2.1 Bối cảnh ứng dụng

Sản phẩm hướng tới phục vụ lực lượng tìm kiếm – cứu nạn (Search & Rescue – SAR) trong các hoạt động hỗ trợ phát hiện và định vị nạn nhân tại những môi trường khó quan sát như rừng núi, khu vực sạt lở, vùng lũ lụt hoặc trong điều kiện thời tiết bất lợi (sương mù, mưa, ánh sáng yếu).

Hệ thống được thiết kế như một công cụ hỗ trợ quan sát thông minh, sử dụng camera quang học (RGB) kết hợp với mô hình trí tuệ nhân tạo để tự động phát hiện con người trong thời gian thực. Thay vì phụ thuộc hoàn toàn vào quan sát thủ công, hệ thống giúp giảm tải cho người vận hành, tăng khả năng phát hiện mục tiêu trong các tình huống phức tạp, đặc biệt khi đối tượng ở xa, kích thước nhỏ hoặc bị che khuất một phần.

Ngoài ra, hệ thống có thể triển khai trên các nền tảng tính toán biên (edge devices) như Raspberry Pi, cho phép hoạt động độc lập tại hiện trường mà không phụ thuộc vào kết nối mạng liên tục, đồng thời có khả năng truyền dữ liệu về dashboard để giám sát từ xa khi cần thiết.

1.2.2 Phạm vi – giả định – tuân thủ

Phạm vi: Hệ thống tập trung vào bài toán phát hiện và nhận dạng con người trong môi trường phức tạp nhằm hỗ trợ quan sát và ra quyết định cho lực lượng tìm kiếm – cứu nạn. Hệ thống đóng vai trò công cụ hỗ trợ, không thay thế hoàn toàn phán đoán của con người và không tham gia vào các chức năng điều khiển tự động hay tác chiến.

Giả định: Dữ liệu đầu vào từ camera quang học (RGB) có chất lượng ổn định (tốc độ khung hình và độ phân giải xác định trước). Hệ thống hoạt động trong các điều kiện môi trường đa dạng nhưng vẫn đảm bảo khả năng thu nhận hình ảnh cơ bản phục vụ cho mô hình AI.

Tuân thủ & đạo đức: Hệ thống tuân thủ các quy định pháp lý liên quan đến an toàn thông tin và bảo vệ dữ liệu. Việc thu thập và xử lý dữ liệu đảm bảo quyền riêng tư của cá nhân, đồng thời tuân theo các nguyên tắc đạo đức trong ứng dụng trí tuệ nhân tạo.

1.2.3 Cấu trúc báo cáo

Báo cáo được tổ chức như sau:

- **Chương 1 – Giới thiệu đề tài:** Trình bày tổng quan bài toán, yêu cầu, thách thức, mục tiêu và định hướng nghiên cứu.
- **Chương 2 – Cơ sở lý thuyết:** Trình bày các nền tảng lý thuyết liên quan, bao gồm xử lý ảnh, học sâu, mô hình YOLO và các chỉ số đánh giá.
- **Chương 3 – Phân tích và thiết kế hệ thống:** Đề xuất các kiến trúc hệ thống, mô tả chi tiết các thành phần và phân tích trade-off giữa các giải pháp.

- **Chương 4 – Dataset và phương pháp đánh giá:** Mô tả bộ dữ liệu sử dụng, các kỹ thuật tăng cường dữ liệu (data augmentation), và thiết lập phương pháp đánh giá khả năng tổng quát hóa của mô hình.
- **Chương 5 – Hiện thực và đánh giá hệ thống:** Trình bày kết quả thực nghiệm trên các nền tảng phần cứng (Raspberry Pi, Edge TPU), phân tích hiệu năng (FPS, độ chính xác) và mô tả hệ thống dashboard giám sát.
- **Chương 6 – Tổng kết và hướng phát triển:** Tổng kết các kết quả đạt được, nêu ra hạn chế của hệ thống và đề xuất các hướng phát triển trong tương lai.

Chương 2

Cơ sở lí thuyết

Trong các hoạt động tác chiến và giám sát quân sự hiện đại, việc nâng cao năng lực nhận thức tình huống là một yêu cầu quan trọng. Ống nhòm là một công cụ quan sát cơ bản, tuy nhiên, việc quan sát thủ công trong thời gian dài bộc lộ nhiều hạn chế như: sự mệt mỏi của người lính làm giảm hiệu quả, khả năng bỏ sót mục tiêu do lỗi của con người, và đặc biệt là hiệu suất quan sát suy giảm nghiêm trọng trong các điều kiện tầm nhìn kém như mưa bão, sương mù, hoặc khói bụi. Những yếu tố này có thể dẫn đến việc chậm trễ trong việc phát hiện mối đe dọa, ảnh hưởng trực tiếp đến an toàn và kết quả của nhiệm vụ.

Đồ án này đặt ra mục tiêu nghiên cứu và phát triển một hệ thống thông minh, tích hợp trí tuệ nhân tạo (AI) vào ống nhòm quân sự để tự động hóa quy trình nhận dạng mục tiêu. Cốt lõi là xây dựng một công cụ hỗ trợ hiệu quả, giúp giảm tải cho người quan sát, tăng cường độ chính xác và rút ngắn thời gian phản ứng trong mọi điều kiện môi trường.

Dự án sẽ được thực hiện dưới dạng một hệ thống được triển khai trên một thiết bị nhúng nhỏ gọn (Raspberry Pi), mô phỏng việc tích hợp vào một hệ thống ống nhòm di động. Các đóng góp chính của đồ án bao gồm: Xây dựng một bộ dữ liệu ảnh chuyên biệt, được gán nhãn cho bài toán nhận dạng mục tiêu qua ống nhòm; tối ưu hóa một kiến trúc phát hiện đối tượng tiên tiến để đạt được hiệu suất thời gian thực (tối thiểu 15 FPS) trên phần cứng có tài nguyên hạn chế và thực hiện một quy trình đánh giá hiệu năng hệ thống một cách khoa học và toàn diện trong các kịch bản tác chiến mô phỏng.

2.1 Cơ sở lý thuyết về Ống nhòm

Trong bối cảnh đề tài nghiên cứu ứng dụng trí tuệ nhân tạo (AI) để tự động nhận dạng mục tiêu trong ảnh ống nhòm, phần cơ sở lý thuyết về ống nhòm đóng vai trò quan trọng để hiểu rõ nguồn gốc dữ liệu đầu vào cho các mô hình học sâu như CNN, YOLO hay Faster R-CNN. Phần này sẽ trình bày chi tiết về định nghĩa, cấu tạo, nguyên lý hoạt động, các loại ống nhòm và ứng dụng liên quan, dựa trên các nguyên tắc quang học cơ bản.

2.1.1 Định nghĩa và lịch sử phát triển cơ bản

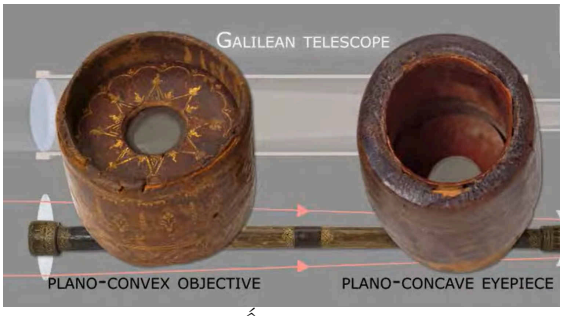
Ống nhòm, hay còn gọi là ống ngắm hay con mắt xa hoặc ống dòm, là một hệ hai kính viễn vọng quang học được gắn cạnh nhau và cùng hướng, cho phép người quan sát đặt cả hai mắt để quan sát các vật ở xa để có tầm nhìn rộng hơn, tạo hình ảnh ba chiều (có chiều sâu). Khác với kính thiên văn đơn, ống nhòm tận dụng thị giác hai mắt để tăng độ chính xác quan sát.



Hình 2.1: *Ống nhòm loại Galilean thời Thế chiến thứ nhất*

Cấu tạo của ống nhòm gồm vật kính, thị kính và thường có thêm hệ thống lăng kính đảo ảnh . Vật kính bao gồm một thấu kính hội tụ, thị kính là thấu kính hội tụ có tiêu cự nhỏ nhất nhưng sẽ làm hình ảnh bị ngược nên cần thêm lăng kính đảo ảnh, còn đơn giản hơn là thị kính làm từ thấu kính phân kì có kích thước nhỏ nhất để làm thị kính mà không bị đảo ảnh với chi phí thấp (tuy nhiên lại ít được sử dụng vì khó căn chỉnh và độ phóng đại thường thấp). Có nhiều loại ống nhòm được dùng phổ biến trong thiên văn, trong săn bắn, quân sự và đời sống.

Trong lịch sử, ống nhòm bắt đầu từ thế kỷ 17, sau khi kính thiên văn được phát minh. Ban đầu, chúng sử dụng quang học Galilean với thấu kính lồi và lõm để tạo hình ảnh thẳng đứng nhưng trường nhìn hẹp.



Hình 2.2: *Ống nhòm Galilean.*

Đến cuối thế kỷ 19, Ignazio Porro phát minh lăng kính Porro, giúp ống nhòm hiện đại hơn, và Carl Zeiss sản xuất hàng loạt vào năm 1894.
Trong cuộc kháng chiến chống Mỹ cứu nước, ống nhòm là một công cụ trinh sát vô cùng quan trọng, đặc biệt hữu ích trong điều kiện rừng rậm và tầm nhìn hạn chế. Về phía quân đội Mỹ, họ sử dụng các loại ống nhòm hiện đại như M18, một mẫu được phát triển trong chiến tranh, tích hợp tia hồng ngoại để phát hiện thân nhiệt, và M19 (loại 7x50), vốn phổ biến nhờ độ bền cao với lớp vỏ vinyl chắc chắn. Trong khi đó, Quân đội Nhân dân Việt Nam và Quân Giải phóng miền Nam Việt Nam đã nhận được sự viện trợ quý báu từ các nước xã hội chủ nghĩa. Các loại khí tài quang học phổ biến bao gồm ống nhòm Carl Zeiss Jena (loại 8x30) của Đông Đức, thường có lưới ngắm để hỗ trợ hiệu chỉnh cho pháo binh, và ống nhòm Type 62 do Trung Quốc sản xuất, dựa trên mẫu B8x30 của Liên Xô.

2.1.2 Chi tiết cấu tạo các bộ phận chính của ống nhòm

Về mặt quang học, cấu tạo của ống nhòm bao gồm ba cụm linh kiện chính: **vật kính (objective lens assembly)**, **hệ thống lăng kính đảo ảnh (image erection system)**, và **thị kính (eyepiece lens assembly)**. Mỗi bộ phận đảm nhiệm một chức năng chuyên biệt để tạo ra hình ảnh phóng đại, thuận chiều cho người quan sát.



Hình 2.3: *Cấu tạo tổng thể với chi tiết các bộ phận*

Bảng dưới đây trình bày chi tiết về cấu trúc và chức năng của các bộ phận này.

Bảng 2.1: *Cấu tạo và chức năng các bộ phận chính của ống nhòm*

Bộ phận	Mô tả và Chức năng
Vật kính (Objective lens)	Hệ thấu kính hội tụ, thường gồm 2-3 thấu kính ghép (hệ tiêu sắc hoặc phức tiêu sắc) để khử quang sai.
Hệ lăng kính đảo ảnh (Prism)	Gồm hai loại chính: lăng kính Porro (cấu trúc hình chữ Z, 4 lần phản xạ) và lăng kính Roof (cấu trúc thẳng hàng, 6 lần phản xạ).
Thị kính (Eyepiece lens)	Hệ thấu kính phức hợp (thiết kế Kellner, Erfle, Plössl) với 3-6 lớp kính, có vai trò như một kính lúp để phóng đại ảnh trung gian.
Cơ cấu điều chỉnh tiêu cự (Focus adjustment)	Thường là một bánh xe trung tâm hoặc vòng xoay độc lập, tác động dịch chuyển tương đối giữa thị kính và vật kính để làm rõ nét hình ảnh.
Thân vỏ và các thành phần cơ khí	Chế tạo từ hợp kim nhôm, magie hoặc vật liệu composite. Có chức năng bảo vệ các linh kiện quang học khỏi va đập, hơi ẩm và bụi bẩn.

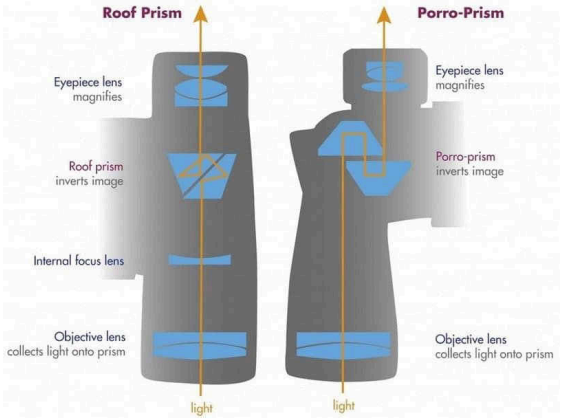
Trong khuôn khổ của đề tài, cấu trúc quang học này đóng vai trò then chốt. Việc thu thập ánh sáng hiệu quả của vật kính và khả năng giảm thiểu quang sai đảm bảo hình ảnh đầu vào (dạng phổ kiến (RGB)) có độ phân giải và độ sắc nét cao. Đây là yếu tố tiên quyết để xây dựng bộ dữ liệu chất lượng, phục vụ cho quá trình huấn luyện và kiểm thử mô hình trí tuệ nhân tạo (AI) nhận dạng mục tiêu.

2.1.3 Nguyên lý Hoạt động Quang học

Nguyên lý hoạt động của ống nhòm dựa trên các định luật cơ bản về **khúc xạ và phản xạ toàn phần của ánh sáng**. Quá trình tạo ảnh diễn ra như sau:

1. Ánh sáng từ mục tiêu đi vào **vật kính**. Hệ thấu kính này hoạt động như một thấu kính hội tụ, tạo ra một ảnh thật, nhỏ hơn và ngược chiều tại tiêu diện.

2. Chùm sáng mang ảnh này tiếp tục đi qua **hệ thống lăng kính**. Tại đây, nhờ hiện tượng phản xạ toàn phần, ảnh được lật lại để trở thành ảnh thuận chiều.
3. Cuối cùng, **thị kính** đóng vai trò như một kính lúp, phóng đại ảnh thuận chiều này lên, tạo ra một ảnh ảo ở vô cực, cho phép mắt người quan sát thoải mái mà không bị mỏi.



Hình 2.4: *Đường đi ánh sáng qua lăng kính Porro và Roof*

Độ phóng đại góc (G) của ống nhòm được xác định bởi tỉ số giữa tiêu cự của vật kính (f_{vk}) và tiêu cự của thị kính (f_{tk}), theo công thức:

$$G = \frac{f_{vk}}{f_{tk}} \tag{2.1}$$

Ví dụ, một ống nhòm có thông số 7x50 có độ phóng đại là 7 lần và đường kính vật kính là 50 mm.

Bên cạnh đó, chất lượng hình ảnh phụ thuộc vào khả năng **khử quang sai** (sắc sai và cầu sai) thông qua việc sử dụng các thấu kính chất lượng cao. Đường kính vật kính còn quyết định **độ sáng** của ảnh và đường kính của **đồng tử thoát** (exit pupil), được tính bằng cách lấy đường kính vật kính chia cho độ phóng đại. Đồng tử thoát tối ưu nên nằm trong khoảng 5-7 mm để phù hợp với khả năng giãn nở của đồng tử mắt người trong điều kiện ánh sáng yếu.

2.2 Ảnh số và quá trình số hoá

Ảnh số là hình ảnh được mã hoá bằng các giá trị số để lưu trữ và xử lý trên máy tính. Ảnh từ thế giới thực được chuyển từ dạng analog thành dạng số qua quá trình số hóa (digitization) gồm hai bước:

- **Lấy mẫu (sampling):** tạo lưới điểm ảnh (pixel).
- **Lượng tử hoá (quantization):** gán giá trị số cho mỗi điểm ảnh.

Ảnh số khác ảnh tương tự ở chỗ có thể thao tác tính toán trực tiếp trên dữ liệu mã hoá.

2.2.1 Cấu trúc cơ bản của ảnh số

Cấu trúc của ảnh số bao gồm các thành phần sau: điểm ảnh (pixel), độ phân giải và kênh màu.

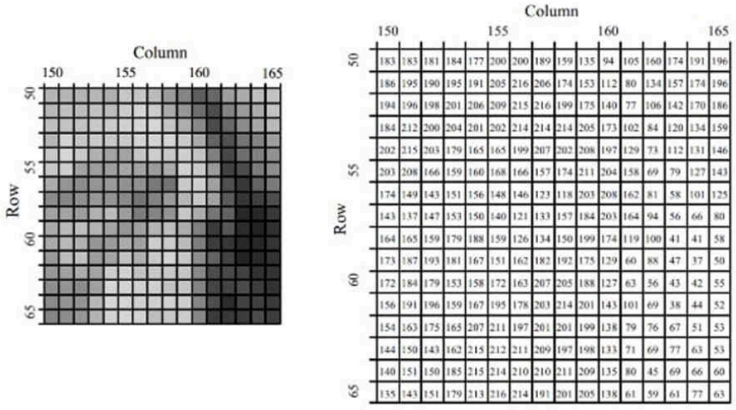
a) **Điểm ảnh (pixel):** gắn với vị trí (x, y) và giá trị độ sáng/màu. Ảnh xám thường 0–255, ảnh màu lưu ba kênh R, G, B.

b) **Độ phân giải:** kích thước $H \times W$ (ví dụ 1920×1080) hoặc dpi. Độ phân giải cao → chi tiết hơn nhưng tốn bộ nhớ.

c) **Kênh màu và độ sâu màu (bit depth):** 1 kênh (xám) hoặc nhiều kênh (RGB,...). Bit depth quyết định số mức biểu diễn: 8-bit → 256 mức; 24-bit (RGB) → ≈16.7 triệu màu.

2.2.2 Phân loại và không gian màu

- Ảnh đen trắng: *nhị phân* (1 bit/pixel), *đa cấp xám* (8-bit → 256 mức).
- Ảnh màu: mô tả trong các không gian RGB (phổ biến), CMYK (in ấn).



Hình 2.5: *Đặc tính ảnh số*

2.3 Cơ sở lý thuyết về ảnh RGB và ảnh IR

2.3.1 Khái niệm cơ bản

Ảnh RGB: (Red–Green–Blue) biểu diễn cảnh bằng ba kênh cường độ ánh sáng đỏ, lục và lam theo mô hình cộng (additive). Mỗi điểm ảnh $\mathbf{p}$ có vector màu $\mathbf{I}(\mathbf{p}) = [R(\mathbf{p}), G(\mathbf{p}), B(\mathbf{p})]^\top$.

Dải phổ của RGB nằm trong vùng ánh sáng nhìn thấy (khoảng 400–700 nm), phù hợp với cảm nhận của mắt người. Ảnh RGB chứa nhiều thông tin về màu sắc và kết

cấu, rất phù hợp cho các bài toán nhận dạng đối tượng trong điều kiện ánh sáng đầy đủ.

Ảnh IR (Infrared): thu nhận bức xạ ngoài vùng ánh sáng khả kiến, thường được sử dụng trong các hệ thống quan sát ban đêm hoặc điều kiện môi trường khắc nghiệt. Tùy theo dải phổ, IR có thể bao gồm NIR (near-infrared) và thermal (MWIR/LWIR).

Tuy nhiên, trong phạm vi đề tài này, hệ thống chỉ sử dụng ảnh RGB do hạn chế về phần cứng và chi phí, cũng như phù hợp với mục tiêu triển khai trên nền tảng tính toán biên.

2.3.2 Đặc trưng tín hiệu

- **RGB:** Giàu thông tin màu sắc và chi tiết kết cấu, nhưng phụ thuộc mạnh vào điều kiện ánh sáng và dễ suy giảm chất lượng trong môi trường sương mù, mưa hoặc ánh sáng yếu.
- **IR:** Ít phụ thuộc vào ánh sáng môi trường, có thể hoạt động tốt trong điều kiện tối hoặc nhiều vật cản, nhưng thường thiếu thông tin màu và có độ phân giải thấp hơn.

2.3.3 So sánh RGB và IR

Bảng 2.2: *So sánh RGB và ảnh hồng ngoại*

Khía cạnh	**RGB**	**IR**
Dải phổ	400–700 nm	>700 nm
Thông tin màu	Có	Không hoặc giả màu
Điều kiện ánh sáng	Cần ánh sáng	Hoạt động tốt trong tối
Độ phân giải	Cao	Thường thấp hơn
Ứng dụng	Nhận dạng chi tiết, màu sắc	Quan sát ban đêm, phát hiện nhiệt

2.3.4 Ý nghĩa đối với bài toán nhận dạng

Trong bài toán phát hiện con người sử dụng ảnh RGB, mô hình học sâu có thể tận dụng thông tin về màu sắc và kết cấu để nhận dạng đối tượng. Tuy nhiên, hiệu năng có thể suy giảm trong các điều kiện môi trường bất lợi như sương mù, mưa hoặc ánh sáng yếu.

Do đó, đề tài tập trung cải thiện khả năng tổng quát hóa của mô hình thông qua các kỹ thuật tăng cường dữ liệu (data augmentation), mô phỏng các điều kiện môi trường thực tế thay vì sử dụng các cảm biến đa phổ như IR.

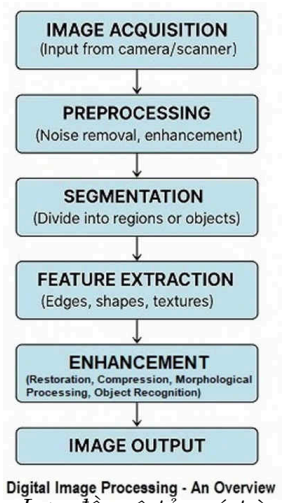
2.4 Cơ sở lý thuyết về xử lý ảnh số

2.4.1 Khái niệm chung

Xử lý ảnh số (Digital Image Processing – DIP) là nhánh của lý thuyết tín hiệu, tập trung vào việc dùng máy tính và thuật toán để biến đổi, cải thiện, hoặc phân tích ảnh số. Ảnh đầu vào là ma trận pixel, qua thuật toán tạo ra ảnh đầu ra rõ nét, giàu thông tin hơn hoặc phù hợp cho phân tích.

2.4.2 Các bước cơ bản trong xử lý ảnh

1. **Thu nhận ảnh (Image Acquisition):** camera, máy quét, cảm biến.
2. **Tiền xử lý (Preprocessing):** khử nhiễu, cân bằng sáng, tăng tương phản.
3. **Phân đoạn (Segmentation):** chia ảnh thành vùng hoặc đối tượng có ý nghĩa.
4. **Trích chọn đặc trưng (Feature Extraction):** phát hiện cạnh, hình dạng, kết cấu.
5. **Cải thiện & khôi phục ảnh (Enhancement & Restoration):** làm sắc nét, loại bỏ mờ nhòe.
6. **Nén ảnh (Compression):** giảm dung lượng lưu trữ và truyền tải.
7. **Xử lý hình thái học (Morphological Processing):** tinh chỉnh hình dạng, biến dạng đối tượng.
8. **Nhận dạng đối tượng (Object Recognition):** gán nhãn hoặc phân loại đối tượng.



Hình 2.6: *Lưu đồ mô tả quá trình xử lý ảnh*

2.5 Thuật toán YOLO trong nhận dạng đối tượng

2.5.1 Giới thiệu

Phát hiện đối tượng (Object Detection) là một trong những bài toán quan trọng nhất của thị giác máy tính (Computer Vision), với ứng dụng rộng rãi trong nhiều lĩnh vực như: xe tự hành, giám sát an ninh, y tế, nông nghiệp, và quân sự. Mục tiêu của bài toán là xác định "có gì trong ảnh" (phân loại) và "nó ở đâu" (xác định vị trí thông qua bounding box).

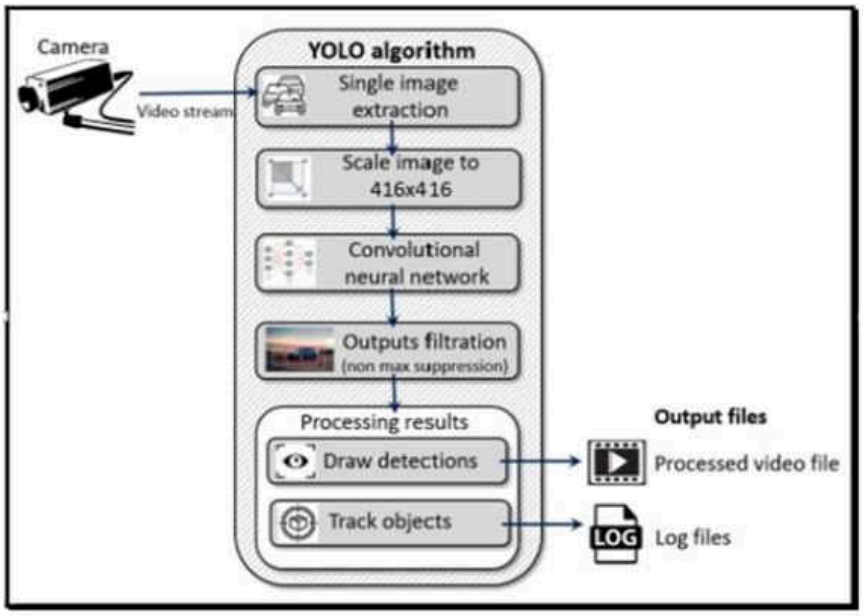
Trong số các phương pháp hiện đại, **YOLO (You Only Look Once)** nổi bật nhờ khả năng cân bằng tốt giữa tốc độ và độ chính xác, đồng thời triển khai thuận lợi trên nhiều nền tảng khác nhau (GPU, CPU, FPGA, thiết bị nhúng).

2.5.2 Nguyên lý hoạt động của YOLO

Quy trình hoạt động cơ bản:

1. **Chia ảnh thành lưới (grid):** ảnh đầu vào được chia thành $S \times S$ ô.
2. **Dự đoán tại mỗi ô:** mỗi ô dự đoán một số bounding box, confidence score, và xác suất lớp.
3. **Kết hợp thông tin:** xác suất lớp $\times$ confidence score $\rightarrow$ độ tin cậy cuối cùng.
4. **Non-Maximum Suppression (NMS):** loại bỏ bounding box trùng lặp, giữ lại dự đoán tốt nhất.
5. **Kết quả:** tập hợp các đối tượng với vị trí và nhãn trong ảnh.

Đặc trưng quan trọng của YOLO là chỉ cần một lượt truyền mạng (single forward pass), nên tốc độ rất nhanh, phù hợp ứng dụng *real-time*.



Hình 2.7: *Sơ đồ luồng hoạt động của thuật toán YOLO*

2.6 Thư viện PyTorch

PyTorch là một **thư viện học máy (Machine Learning) mã nguồn mở** được phát triển bởi Facebook AI Research (FAIR), kế thừa và mở rộng từ thư viện Torch. PyTorch cung cấp một nền tảng mạnh mẽ và linh hoạt cho việc xây dựng, huấn luyện và triển khai các mô hình học sâu (Deep Learning). Với thiết kế trực quan, dễ sử dụng cùng khả năng tính toán mạnh mẽ trên GPU, PyTorch đã nhanh chóng trở thành một trong những công cụ phổ biến nhất trong cộng đồng nghiên cứu và phát triển trí tuệ nhân tạo.

2.6.1 Các đặc điểm nổi bật của PyTorch

- **Đồ thị tính toán động (Dynamic Computation Graph):** Khác với nhiều framework sử dụng đồ thị tĩnh (static graph), PyTorch xây dựng đồ thị tính toán ngay khi thực thi (cơ chế **define-by-run**). Cách tiếp cận này cho phép quan sát và điều chỉnh cấu trúc mạng nơ-ron một cách linh hoạt trong vòng lặp huấn luyện, hỗ trợ tốt cho việc thử nghiệm và tối ưu hoá mô hình.

- **Tensor:** Tensor trong PyTorch là cấu trúc dữ liệu trung tâm, tương tự mảng (array) của NumPy, nhưng được mở rộng với khả năng tính toán trên GPU. Điều này giúp tăng tốc đáng kể các phép toán số học và xử lý dữ liệu song song. PyTorch đồng thời cung cấp bộ hàm và API đa dạng, hỗ trợ thao tác với Tensor một cách hiệu quả và trực quan.

- **Autograd (Tự động tính gradient):** Tính năng *autograd* cho phép PyTorch tự động tính toán gradient của các phép toán trên Tensor. Nhờ vậy, việc xây dựng và huấn luyện các mô hình học sâu trở nên đơn giản hơn, loại bỏ nhu cầu viết thủ công quy tắc chuỗi (chain rule), tiết kiệm thời gian và giảm thiểu sai sót cho nhà phát triển.
- **Modules:** Lớp `torch.nn.Module` được xem như những khối xây dựng cơ bản trong việc thiết kế mô hình học sâu. PyTorch đi kèm với một thư viện phong phú các module dựng sẵn, bao gồm lớp mạng nơ-ron (CNN, RNN, LSTM), hàm mất mát (loss function), và thuật toán tối ưu (optimizer). Người dùng có thể dễ dàng kết hợp chúng để tạo và huấn luyện mô hình.
- **Hỗ trợ GPU:** PyTorch tận dụng hiệu quả sức mạnh xử lý song song của GPU, giúp rút ngắn đáng kể thời gian tính toán và huấn luyện mô hình phức tạp. Việc chuyển đổi giữa CPU và GPU cũng rất linh hoạt, thường chỉ cần một vài dòng lệnh.

2.6.2 Ưu điểm và Nhược điểm của PyTorch

Ưu điểm

- **Tính linh hoạt vượt trội với đồ thị động:** Điểm mạnh cốt lõi của PyTorch chính là *dynamic computation graph*. Cơ chế này cho phép định nghĩa và thay đổi cấu trúc tính toán của mô hình ngay trong quá trình chạy (*Define-by-Run*). Sự linh hoạt này đặc biệt hữu ích trong nghiên cứu, nơi cần thử nghiệm các kiến trúc mạng mới hoặc mô hình có cấu trúc thay đổi (như xử lý ngôn ngữ tự nhiên với độ dài câu khác nhau).
- **Tích hợp Python tự nhiên (Pythonic):** PyTorch được thiết kế theo phong cách "Pythonic", với cú pháp quen thuộc và dễ đọc, tương tự NumPy. Điều này giúp người đã quen với Python dễ dàng tiếp cận, tận dụng kỹ năng lập trình sẵn có và kết hợp PyTorch với các thư viện dữ liệu phổ biến (NumPy, Pandas, Scikit-learn, Matplotlib). Việc chuyển đổi giữa `Tensor` và `ndarray` cũng rất thuận tiện thông qua `.numpy()` và `torch.from_numpy()`.
- **Dễ dàng gỡ lỗi (Debugging):** Nhờ cơ chế đồ thị động, khi lỗi xảy ra (ví dụ: kích thước Tensor không khớp), PyTorch ném ra *exception* Python tại chính dòng code lỗi. Người dùng có thể sử dụng trực tiếp các công cụ gỡ lỗi Python thông thường như `pdb` hoặc IDE. Điều này khác biệt so với framework dùng đồ thị tĩnh, vốn thường chỉ báo lỗi ở giai đoạn chạy mô hình, gây khó khăn trong truy vết. Khả năng gỡ lỗi trực quan này giúp tiết kiệm nhiều thời gian cho nhà phát triển, đặc biệt trong các mô hình phức tạp.

Nhược điểm

- **Triển khai sản phẩm (Deployment) từng là thách thức:** Trong quá khứ, việc triển khai PyTorch vào môi trường production kém thuận tiện hơn TensorFlow. Do tính chất "Pythonic" và đồ thị động, việc đóng gói mô hình để chạy độc lập ngoài môi trường Python gặp nhiều khó khăn. Gần đây, PyTorch đã cải thiện đáng kể với *TorchScript*, *TorchServe* và khả năng xuất mô hình sang định dạng ONNX, tuy nhiên vẫn yêu cầu nhà phát triển học thêm công cụ và quy trình bổ sung.
- **Tối ưu hóa hiệu suất tĩnh phức tạp hơn:** Framework đồ thị tĩnh có lợi thế trong việc tối ưu toàn cục trước khi thực thi (ví dụ: hợp nhất phép toán, loại bỏ nhánh thừa), giúp giảm độ trễ và tăng thông lượng. Trong PyTorch, để đạt hiệu quả tương tự cần dùng tới TorchScript (JIT – Just-In-Time) hoặc các công cụ AOT(Ahead-Of-Time) như TorchDynamo. Điều này làm tăng độ phức tạp khi tối ưu hóa mô hình cho môi trường hạn chế (mobile, edge device).
- **Đánh đổi giữa linh hoạt và hiệu năng:** PyTorch ưu tiên tính linh hoạt trong nghiên cứu và phát triển, nhưng để đạt hiệu suất suy luận tối đa trên production thì đòi hỏi nhiều bước tối ưu hơn. Đây là sự đánh đổi tự nhiên mà người dùng cần cân nhắc tùy theo mục tiêu sử dụng.

2.6.3 Ứng dụng thực tiễn của PyTorch trong thị giác máy tính (Computer Vision)

- **Nhận dạng hình ảnh (Image Classification & Object Detection):** PyTorch được dùng để xây dựng các hệ thống nhận dạng hình ảnh hiệu quả cao, có khả năng phân loại hình ảnh, phát hiện và xác định vị trí của các đối tượng trong ảnh. Ứng dụng tiêu biểu gồm nhận diện khuôn mặt, phân loại sản phẩm trong công nghiệp, phát hiện lỗi trên dây chuyền sản xuất, và hỗ trợ chẩn đoán y khoa qua hình ảnh.
- **Phân đoạn ảnh (Image Segmentation):** PyTorch là công cụ đắc lực cho các bài toán phân đoạn ảnh, chia hình ảnh thành nhiều vùng mang ý nghĩa ngữ nghĩa riêng. Ứng dụng trong công nghệ xe tự hành (xác định làn đường, phương tiện, người đi bộ), trong y tế (phân tích cấu trúc trong ảnh chụp X-quang, MRI), và trong nông nghiệp (đánh giá tình trạng cây trồng qua hình ảnh).
- **Tạo sinh ảnh (Image Generation):** Các mô hình sinh ảnh dựa trên PyTorch (GANs, Diffusion Models) có thể tạo ra những hình ảnh mới chân thực hoặc sáng tạo. Ứng dụng trong nghệ thuật số, thiết kế thời trang, trò chơi điện tử, và tạo khuôn mặt ảo phục vụ nhiều mục đích khác nhau.

2.7 Tổng quan về Deep Learning

Deep Learning (học sâu) là một lĩnh vực con của **Machine Learning** (học máy), trong đó máy tính có khả năng tự học và cải thiện hiệu suất thông qua việc sử dụng

các thuật toán phức tạp, được lấy cảm hứng từ cơ chế hoạt động của não người. Từ "deep" (sâu) thể hiện việc mô hình sử dụng nhiều lớp mạng nơ-ron nhân tạo để phân tích, xử lý và hiểu dữ liệu.

2.7.1 Nguyên lý hoạt động của Deep Learning

Deep Learning hoạt động dựa trên việc sử dụng mạng nơ-ron nhân tạo (artificial neural networks) để mô phỏng cách tư duy của bộ não con người. Mạng nơ-ron trong học sâu được tổ chức thành nhiều lớp (layers) liên kết với nhau, mỗi lớp đảm nhận một vai trò riêng như phát hiện đặc trưng, nhận diện mẫu hay đưa ra dự đoán. Khi số lượng lớp tăng lên, mô hình có thể học được các đặc trưng phức tạp hơn và cho ra kết quả dự đoán chính xác hơn.

Các thành phần chính trong mạng nơ-ron nhân tạo

Mạng nơ-ron nhân tạo bao gồm một số thành phần cơ bản như sau:

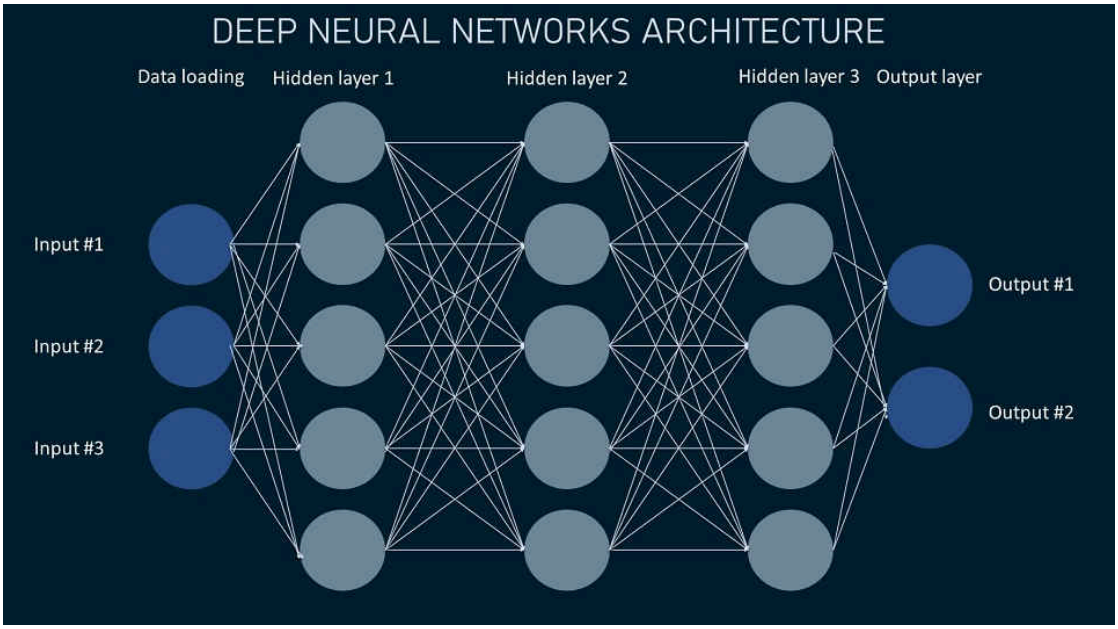
- **Lớp đầu vào (Input Layer):** Là nơi tiếp nhận dữ liệu thô từ môi trường bên ngoài. Mỗi nơ-ron trong lớp này tương ứng với một đặc trưng của dữ liệu, chẳng hạn như một pixel trong hình ảnh hoặc một từ trong câu văn.
- **Lớp ẩn (Hidden Layers):** Nằm giữa lớp đầu vào và lớp đầu ra, các lớp này thực hiện các phép tính phức tạp để trích xuất thông tin và đặc trưng từ dữ liệu. Số lượng và cấu trúc của các lớp ẩn ảnh hưởng trực tiếp đến khả năng học và độ chính xác của mô hình.
- **Nơ-ron (Neurons):** Là các đơn vị tính toán cơ bản của mạng. Mỗi nơ-ron nhận tín hiệu từ các nơ-ron ở lớp trước, xử lý thông tin qua một hàm kích hoạt, sau đó truyền kết quả đến các nơ-ron ở lớp tiếp theo. Cấu trúc này mô phỏng cơ chế hoạt động của các tế bào thần kinh trong não người.
- **Trọng số và độ chệch (Weights và Biases):** Mỗi kết nối giữa hai nơ-ron được gán một trọng số, thể hiện mức độ ảnh hưởng của tín hiệu đầu vào đến đầu ra. Độ chệch (*bias*) giúp điều chỉnh đầu ra của nơ-ron, tăng tính linh hoạt và khả năng học của mô hình.

Quy trình xử lý trong mạng nơ-ron

Quy trình xử lý dữ liệu trong mạng nơ-ron diễn ra theo các bước cơ bản như sau:

- **Nhận dữ liệu đầu vào:** Dữ liệu thô được đưa vào thông qua lớp đầu vào (*Input Layer*). Mỗi nơ-ron trong lớp này đại diện cho một đặc trưng của dữ liệu, ví dụ như một điểm ảnh trong hình ảnh hoặc một giá trị cảm biến.

- **Tính toán trong các lớp ẩn:** Ở các lớp ẩn (*Hidden Layers*), mỗi nơ-ron thực hiện các phép toán bao gồm nhân giá trị đầu vào với trọng số, cộng với độ chệch, sau đó áp dụng hàm kích hoạt để tạo ra đầu ra phi tuyến tính. Kết quả này được truyền tiếp qua các lớp kế tiếp để trích xuất đặc trưng sâu hơn của dữ liệu.
- **Trả kết quả tại lớp đầu ra:** Sau khi dữ liệu đi qua toàn bộ các lớp ẩn, kết quả cuối cùng được tạo ra tại lớp đầu ra (*Output Layer*). Tùy vào bài toán, đầu ra có thể là một giá trị dự đoán, một xác suất, hoặc một nhãn phân loại.



Kiến trúc mạng nơ-ron

2.7.2 Ưu điểm và nhược điểm của Deep Learning

Ưu điểm

- **Tự động hóa trích xuất đặc trưng:** Các hệ thống Deep Learning có khả năng tự động học và trích xuất các đặc trưng từ dữ liệu, giúp giảm sự can thiệp thủ công trong quá trình xử lý. Điều này tiết kiệm thời gian và tăng hiệu quả huấn luyện mô hình.
- **Khả năng phát hiện mẫu và mối quan hệ phức tạp:** Deep Learning có thể phân tích lượng lớn dữ liệu để nhận diện các mẫu phức tạp trong hình ảnh, âm thanh hoặc văn bản. Nhờ vậy, công nghệ này được ứng dụng rộng rãi trong các hệ thống nhận diện khuôn mặt, dịch tự động và gợi ý thông minh.
- **Xử lý dữ liệu đa dạng:** Khác với các phương pháp truyền thống, Deep Learning có thể xử lý cả dữ liệu có cấu trúc và dữ liệu không có cấu trúc như hình ảnh,

video hay âm thanh.

- **Độ chính xác cao:** Với nhiều lớp nơ-ron, các mô hình học sâu đạt độ chính xác vượt trội khi được huấn luyện đúng cách, đặc biệt trong các lĩnh vực như nhận dạng giọng nói và phân loại hình ảnh.
- **Khả năng mở rộng với dữ liệu lớn:** Hiệu suất của Deep Learning thường tăng khi lượng dữ liệu huấn luyện tăng, rất phù hợp với các ứng dụng khai thác Big Data.
- **Tiết kiệm chi phí dài hạn:** Dù chi phí ban đầu cho việc huấn luyện mô hình có thể cao, nhưng về lâu dài, các doanh nghiệp có thể tiết kiệm chi phí nhờ giảm sai sót và tự động hóa quy trình.
- **Giảm sự can thiệp của con người:** Deep Learning giúp giảm bớt sự phụ thuộc vào con người trong việc phân tích và xử lý dữ liệu, từ đó nâng cao hiệu quả và độ chính xác trong các hệ thống tự động.

Nhược điểm

- **Yêu cầu tài nguyên tính toán cao:** Deep Learning cần phần cứng mạnh mẽ như GPU hoặc TPU để xử lý khối lượng phép toán lớn. Những thiết bị này đắt đỏ và tiêu thụ nhiều năng lượng, làm tăng chi phí triển khai.
- **Phụ thuộc vào lượng lớn dữ liệu huấn luyện:** Các mô hình học sâu chỉ đạt hiệu quả tối ưu khi có đủ dữ liệu đa dạng và chất lượng cao. Việc thu thập, xử lý và gắn nhãn dữ liệu thường tốn kém thời gian và nguồn lực.
- **Thời gian đào tạo kéo dài:** Việc huấn luyện mô hình Deep Learning có thể kéo dài từ nhiều ngày đến vài tuần, đặc biệt khi mô hình phức tạp hoặc phần cứng hạn chế, gây khó khăn cho các ứng dụng cần phản hồi nhanh.
- **Khả năng giải thích hạn chế:** Deep Learning thường được ví như một "hộp đen" vì khó giải thích cách mô hình đưa ra quyết định hoặc dự đoán. Đây là vấn đề lớn trong các lĩnh vực yêu cầu tính minh bạch như y tế, tài chính và pháp luật.
- **Dễ mắc sai lệch (bias):** Nếu dữ liệu huấn luyện có thành kiến, mô hình có thể tái tạo hoặc khuếch đại sai lệch này, dẫn đến kết quả không công bằng, đặc biệt trong các hệ thống như nhận diện khuôn mặt hoặc ra quyết định tự động.
- **Khả năng đa nhiệm hạn chế:** Các mô hình Deep Learning thường chỉ được huấn luyện cho một nhiệm vụ cụ thể. Khi áp dụng cho bài toán khác, cần huấn luyện lại toàn bộ hệ thống, gây tốn thời gian và tài nguyên.
- **Thiếu khả năng suy luận:** Deep Learning chưa có khả năng thực hiện lập luận logic hoặc lên kế hoạch dài hạn. Mô hình chỉ xử lý dữ liệu dựa trên các mẫu đã học, thay vì áp dụng tư duy hoặc suy luận như con người.

2.7.3 Ứng dụng thực tiễn của Deep Learning trong thị giác máy tính (Computer Vision)

Nhiều ứng dụng AI được ra mắt trên thị trường nhờ vào những bước tiến mạnh mẽ của lĩnh vực thị giác máy tính, với deep learning đóng một vai trò then chốt trong quá trình phát triển này. Đáng chú ý, nó đã thúc đẩy sự phát triển khả năng phát hiện đối tượng, phân loại hình ảnh và phân vùng ảnh trong thị giác máy tính.

1. Phân loại hình ảnh (Image Classification)

Phân loại hình ảnh là một trong những nhiệm vụ cơ bản nhất của thị giác máy tính, nhằm gán một nhãn (label) cho hình ảnh từ một tập danh mục xác định trước. Nhờ sự phát triển của mạng nơ-ron tích chập (CNN – Convolutional Neural Network), độ chính xác và hiệu quả của các mô hình phân loại hình ảnh đã được cải thiện đáng kể.

- **Ứng dụng:**
  - **Chẩn đoán y khoa:** CNN được sử dụng để phân loại các ảnh y tế như X-quang, MRI nhằm phát hiện bệnh lý như viêm phổi, khối u và nhiều vấn đề khác.
  - **Xe tự hành:** Trong xe tự lái, mô hình phân loại hình ảnh giúp nhận dạng biển báo, người đi bộ và phương tiện khác.
  - **Bán lẻ:** Doanh nghiệp sử dụng phân loại hình ảnh để sắp xếp và phân loại sản phẩm, giúp cải thiện tìm kiếm và trải nghiệm khách hàng.

2. Phát hiện đối (Object Detection)

Khác với phân loại hình ảnh, phát hiện vật thể không chỉ nhận dạng đối tượng trong ảnh mà còn xác định vị trí của chúng thông qua các hộp bao (bounding boxes). Các mô hình Deep Learning nổi bật trong lĩnh vực này bao gồm Faster R-CNN, và YOLO (*You Only Look Once*).

- **Ứng dụng:**
  - **Giám sát an ninh:** Phát hiện và theo dõi người, phương tiện hoặc các hành vi đáng ngờ trong thời gian thực.
  - **Y tế:** Hỗ trợ phát hiện và khoanh vùng các bất thường (ví dụ: khối u) trên hình ảnh chụp X-quang hoặc MRI.
  - **Sản xuất:** Trong dây chuyền kiểm tra tự động, phát hiện vật thể giúp nhận diện lỗi sản phẩm để đảm bảo chất lượng.

3. Phân đoạn hình ảnh (Image Segmentation)

Phân đoạn hình ảnh là quá trình chia ảnh thành nhiều vùng (segment) hoặc đối tượng khác nhau nhằm xác định chính xác ranh giới và vị trí của chúng. Có hai dạng chính: *s*emantic segmentation – gán nhãn cho từng điểm ảnh, và *i*nstance segmentation – phân biệt giữa các đối tượng riêng biệt thuộc cùng một lớp.

- **Ứng dụng:**
  - **Y học:** Giúp xác định rõ ranh giới các cấu trúc giải phẫu và vùng bất thường trong ảnh y học, hỗ trợ chẩn đoán và lập kế hoạch điều trị.
  - **Xe tự hành:** Hỗ trợ nhận diện làn đường, biển báo, vật cản và môi trường xung quanh xe.
  - **Thực tế tăng cường (AR):** Giúp gắn kết chính xác các đối tượng ảo vào cảnh thật, nâng cao trải nghiệm AR.

4. Nhận diện khuôn mặt (Facial Recognition)

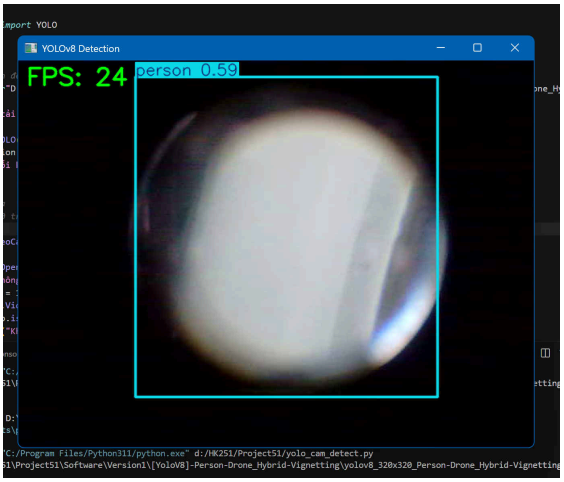
Nhận diện khuôn mặt là kỹ thuật xác định hoặc xác minh danh tính cá nhân dựa trên đặc trưng khuôn mặt. Các mô hình Deep Learning, đặc biệt là CNN, đã giúp cải thiện đáng kể độ chính xác và khả năng nhận diện trong nhiều điều kiện khác nhau.

- **Ứng dụng:**
  - **An ninh và giám sát:** Sử dụng để xác định danh tính, kiểm soát ra vào, hoặc phát hiện người trong khu vực công cộng.
  - **Thiết bị di động:** Nhiều smartphone hiện nay tích hợp công nghệ nhận diện khuôn mặt để mở khóa và xác thực người dùng.
  - **Mạng xã hội:** Các nền tảng như Facebook sử dụng nhận diện khuôn mặt để tự động gợi ý gắn thẻ (tag) người dùng trong ảnh, giúp tăng tương tác và trải nghiệm.

2.8 Shortcut learning

2.8.1 Hiện tượng quan sát được

Mô hình có xu hướng nhận diện khung viền đen (vignette/lỗ tròn) của ống nhòm là đặc trưng chính để phân loại, thay vì học các đặc điểm hình thái của đối tượng bên trong. Theo Geirhos và cộng sự (2020), trong bài báo "Shortcut Learning in Deep Neural Networks", hiện tượng này được gọi là Shortcut Learning (Học tắt). Đây là các quy tắc quyết định hoạt động tốt trên tập dữ liệu chuẩn nhưng thất bại khi chuyển sang các điều kiện kiểm thử thách thức hơn hoặc dữ liệu thực tế.

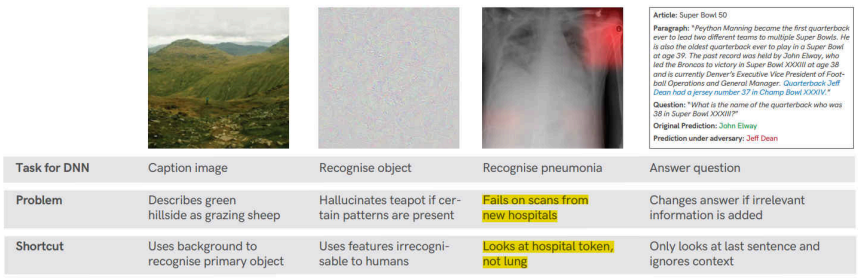


Hình 2.9: *Hình ảnh hiện tượng Shortcut Learning: Thiên kiến vùng tối (Vignette Bias)*

2.8.2 Bản chất của Shortcut Learning

Shortcut learning là hiện tượng mà mô hình học sâu sử dụng các chiến lược quyết định đơn giản, hiệu quả trên benchmark tiêu chuẩn nhưng thất bại khi chuyển sang điều kiện thử nghiệm thách thức hơn.

Ví dụ điển hình từ bài báo là trong chẩn đoán viêm phổi (pneumonia) từ ảnh X-quang, mô hình đạt độ chính xác cao nhưng không nhìn vào phổi mà nhìn vào "đồng xu kim loại"(metal token) đặc trưng của bệnh viện. Thay vì học "dấu hiệu bệnh lý trong phổi → viêm phổi", mô hình học "dấu hiệu bệnh viện → xác suất viêm phổi cao"(vì bệnh viện đó thường tiếp nhận ca nặng).



Hình 2.10: *Hiệu ứng shortcuts*

Tương tự, trong nhận diện bò, mô hình sử dụng nền cỏ thay vì hình dạng bò. Liên hệ với thí nghiệm: Vùng tối vignette tương tự như metal token hoặc nền cỏ – một shortcut đơn giản theo Principle of Least Effort.

2.8.3 Phân tích rủi ro Shortcut Learning và Chiến lược Data Augmentation

1. Vấn đề Shortcut Learning từ Hiệu ứng Vignette (Lỗ đen): Thực nghiệm cho thấy hiệu ứng Vignette đóng vai trò là một "Shortcut Feature"mạnh. Do tính chất hình học cố định và độ tương phản cao, mô hình có xu hướng học mối tương quan giả: nhận diện sự hiện diện của viền đen thay vì đối tượng mục tiêu. Do đó, chiến lược của nhóm là loại bỏ hoặc giảm thiểu sự phụ thuộc vào Vignette , ưu tiên trước hết là kỹ thuật **Center Cropping** (crop frame trước khi đưa ảnh vào mô hình) hoặc bổ sung **Negative Samples** (ảnh nền có Vignette nhưng không có người).

2. Vai trò Tích cực của Data Augmentation Ngược lại, các yếu tố như sương mù, nhiễu hạt, hay giả lập màu khuếch đại ánh sáng ban đêm (Image Intensifier) không tạo ra các shortcut hình học. Việc áp dụng các hiệu ứng này buộc mô hình không thể dựa vào các đặc trưng bề mặt (như màu sắc cụ thể hay kết cấu rõ nét) mà phải học các đặc trưng hình dáng (Shape Bias) bền vững hơn.

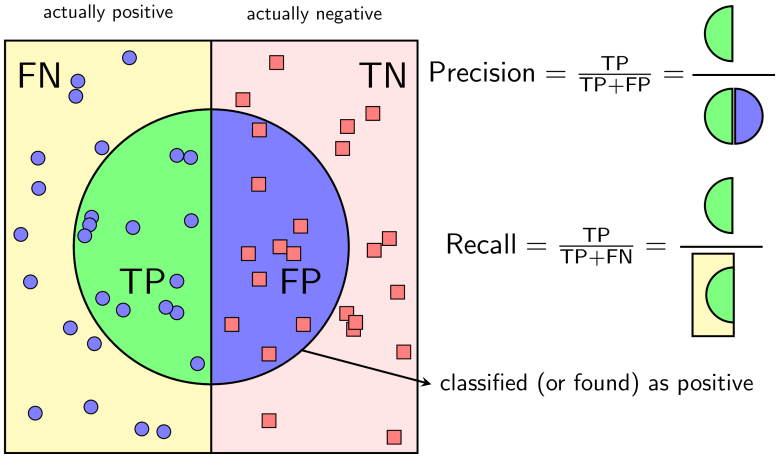
Kết luận: Trong khi Vignette cung cấp một "đường tắt"(shortcut) để mô hình gian lận điểm số, thì các hiệu ứng môi trường (sương, màu) tạo ra "chướng ngại vật"để mô hình học sâu và hiệu quả hơn.

2.9 Các chỉ số đánh giá mô hình AI

2.9.1 Ý nghĩa của TP, TN, FP, FN

Trong bài toán phân lớp nhị phân, ta thường quy ước một lớp là **Positive (dương tính)** và lớp còn lại là **Negative (âm tính)**. Khi đó, dựa trên ma trận nhầm lẫn (*confusion matrix*), kết quả phân loại của mô hình có thể được chia thành bốn trường hợp cơ bản như sau:

- **True Positive (TP):** số lượng mẫu thực sự thuộc lớp *Positive* và được mô hình dự đoán đúng là *Positive*. Đây là các trường hợp mô hình phát hiện đúng đối tượng cần quan tâm.
- **True Negative (TN):** số lượng mẫu thực sự thuộc lớp *Negative* và được mô hình dự đoán đúng là *Negative*. Điều này thể hiện khả năng mô hình loại bỏ đúng các mẫu không thuộc lớp mục tiêu.
- **False Positive (FP):** số lượng mẫu thực sự thuộc lớp *Negative* nhưng bị mô hình dự đoán nhầm thành *Positive*. Trường hợp này còn được xem là một dạng *báo động nhầm*, vì hệ thống cho rằng có đối tượng cần phát hiện trong khi thực tế không có.
- **False Negative (FN):** số lượng mẫu thực sự thuộc lớp *Positive* nhưng bị mô hình dự đoán nhầm thành *Negative*. Đây là trường hợp *bỏ sót*, thường nghiêm trọng trong các bài toán như chẩn đoán bệnh hoặc phát hiện vật thể nguy hiểm. :



Bốn đại lượng trên thường được trình bày trong ma trận nhầm lẫn cho bài toán phân lớp nhị phân như sau:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

2.9.2 Các chỉ số Precision, Recall và mAP

Để đánh giá chất lượng của mô hình phân loại hoặc phát hiện đối tượng, ngoài các đại lượng cơ bản như TP, FP, TN, FN, người ta thường sử dụng các chỉ số *Precision*, *Recall* và *mAP*. Đây là những thước đo quan trọng giúp phản ánh khả năng dự đoán đúng của mô hình trên từng khía cạnh khác nhau.

Precision

Precision là tỉ lệ số mẫu được dự đoán là dương tính và thực sự đúng trên tổng số mẫu mà mô hình dự đoán là dương tính. Chỉ số này cho biết trong tất cả các kết quả mà mô hình nhận diện là đúng đối tượng, có bao nhiêu kết quả là chính xác.

$$Precision = \frac{TP}{TP + FP}$$

Giá trị Precision càng cao thì số lượng dự đoán nhầm dương tính giả (*False Positive*) càng thấp. Vì vậy, Precision đặc biệt quan trọng trong các bài toán mà cảnh báo nhầm có thể gây ảnh hưởng lớn, chẳng hạn như nhận diện lỗi sản phẩm hoặc phát hiện xâm nhập.

Recall

Recall là tỉ lệ số mẫu dương tính thực sự được mô hình nhận diện đúng trên tổng số mẫu dương tính có trong thực tế. Chỉ số này phản ánh khả năng phát hiện đầy đủ

các đối tượng cần quan tâm của mô hình.

$$Recall = \frac{TP}{TP + FN}$$

Recall càng cao thì số lượng bỏ sót (*False Negative*) càng thấp. Chỉ số này đặc biệt quan trọng trong các bài toán mà việc bỏ sót đối tượng là không mong muốn, ví dụ như chẩn đoán bệnh, phát hiện cháy hoặc phát hiện người trong hệ thống giám sát an toàn.

Mối quan hệ giữa Precision và Recall

Precision và Recall thường có mối quan hệ đánh đổi lẫn nhau. Khi mô hình được điều chỉnh để tăng khả năng phát hiện đối tượng, Recall có thể tăng nhưng đồng thời số lượng dự đoán nhầm cũng tăng, làm giảm Precision. Ngược lại, nếu mô hình chỉ đưa ra các dự đoán thật chắc chắn, Precision có thể tăng nhưng Recall lại giảm do bỏ sót nhiều đối tượng. Vì vậy, trong thực tế cần lựa chọn ngưỡng dự đoán phù hợp tùy theo yêu cầu của bài toán.

mAP

mAP là viết tắt của *mean Average Precision*, là một chỉ số được sử dụng rất phổ biến trong các bài toán phát hiện đối tượng (*object detection*). Chỉ số này phản ánh chất lượng tổng thể của mô hình thông qua việc kết hợp khả năng phát hiện đúng đối tượng và khả năng định vị chính xác vị trí của đối tượng.

Trước hết, đối với từng lớp đối tượng, người ta xây dựng đường cong *Precision-Recall*. Từ đường cong này, giá trị *Average Precision* (AP) được tính như diện tích dưới đường cong Precision-Recall. Sau đó, giá trị mAP được xác định bằng trung bình cộng của AP trên tất cả các lớp:

$$mAP = \frac{1}{N} \sum_{i=1}^{N} AP_i$$

Trong đó:

- N là số lớp đối tượng,
- AP_i là giá trị Average Precision của lớp thứ i.

Trong các bài toán phát hiện đối tượng, mAP thường được tính tại một ngưỡng IoU nhất định, ví dụ **mAP@0.5**. Điều này có nghĩa là một đối tượng được xem là phát hiện đúng nếu vùng dự đoán và vùng thực tế có chỉ số IoU lớn hơn hoặc bằng 0.5. Ngoài ra, một số bộ dữ liệu còn sử dụng chỉ số **mAP@0.5:0.95**, tức là lấy trung bình AP trên nhiều mức IoU khác nhau, từ 0.5 đến 0.95, nhằm đánh giá mô hình một cách chặt chẽ hơn.

Chương 3

Phân tích và Thiết kế Kiến trúc Hệ thống

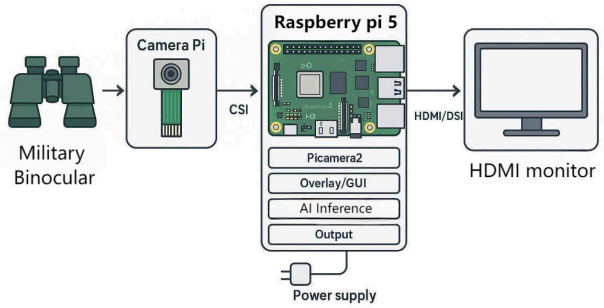
Chương này trình bày toàn bộ kiến trúc hệ thống của đồ án. Hệ thống được thiết kế theo mô hình phân tầng (layered architecture) với ba lớp chính: (1) **Kiến trúc phần cứng** — lựa chọn nền tảng tính toán và các thành phần ngoại vi; (2) **Kiến trúc Dashboard giám sát** — hệ thống web cho phép theo dõi và điều khiển từ xa; và (3) **Kiến trúc xử lý AI** — pipeline suy luận thời gian thực được triển khai dưới hai nhánh: *EdgeVisionRT* (C++/NCNN, tối ưu CPU) và *CoralVisionRT* (Python/TFLite, tăng tốc Edge TPU).

Mỗi giải pháp đại diện cho một điểm cân bằng khác nhau giữa **hiệu năng**, **chi phí** và **độ phức tạp triển khai**. Việc phân tích đồng thời hai kiến trúc cho phép nhóm nghiên cứu đánh giá khách quan các đánh đổi (trade-offs) và lựa chọn phương án phù hợp nhất cho từng kịch bản sử dụng thực tế.

3.1 Kiến trúc phần cứng

3.1.1 Giải pháp 1 — CPU thuần (Raspberry Pi)

Giải pháp này tuân theo triết lý *"Software-First"* — ưu tiên tối ưu hóa phần mềm trên phần cứng phổ thông, chi phí thấp. Trọng tâm là tận dụng tối đa khả năng tính toán của CPU ARM trên Raspberry Pi mà không cần bất kỳ bộ tăng tốc AI chuyên dụng nào.



Hình 3.1: *Sơ đồ phần cứng của giải pháp CPU thuần*

Thành phần chính:

- **Vi xử lý trung tâm:** Raspberry Pi 4 Model B (Cortex-A72) hoặc Raspberry Pi 5 (Cortex-A76).
- **Thu nhận hình ảnh:** USB Webcam (V4L2, định dạng YUYV 640×480) hoặc Camera CSI.
- **Hiển thị:** Màn hình TFT LCD kết nối qua DSI/GPIO hoặc HDMI Monitor.
- **Nguồn cung cấp:** Pin sạc dự phòng USB 5V/≥3A.

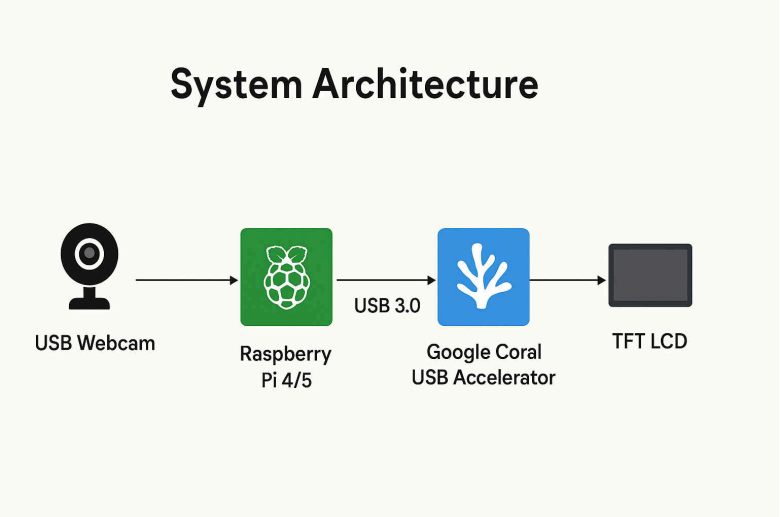
Toàn bộ pipeline — từ thu nhận khung hình, tiền xử lý, suy luận AI, hậu xử lý đến hiển thị — đều chạy trên CPU ARM. Đây là kiến trúc đơn giản nhất, chi phí thấp nhất, nhưng CPU trở thành nút thắt cổ chai khi phải xử lý đồng thời nhiều tác vụ nặng.

Ưu điểm: Chi phí đầu tư thấp, dễ triển khai, cộng đồng hỗ trợ lớn.

Nhược điểm: Hiệu năng hạn chế (CPU chiếm 80–90% cho suy luận), khó đạt real-time mượt mà, tiêu thụ năng lượng không hiệu quả khi hoạt động dưới tải nặng.

3.1.2 Giải pháp 2 — Tăng tốc Edge TPU (Raspberry Pi + Google Coral)

Giải pháp này áp dụng triết lý *"Offload and Accelerate"* — dỡ tải tác vụ suy luận AI sang bộ đồng xử lý chuyên dụng Google Coral USB Accelerator. CPU được giải phóng để đảm nhận các công việc quản lý hệ thống, tiền/hậu xử lý và giao diện người dùng.



Hình 3.2: *Sơ đồ phần cứng của giải pháp Edge TPU*

Thành phần chính:

- **Vi xử lý trung tâm:** Raspberry Pi 4B (yêu cầu cổng USB 3.0).

- **Bộ tăng tốc AI:** Google Coral USB Accelerator (4 TOPS, 2 TOPS/W).
- **Thu nhận hình ảnh:** USB Webcam hoặc Camera CSI.
- **Hiển thị:** Màn hình TFT LCD qua framebuffer `/dev/fb0`.

Luồng dữ liệu offload:

1. CPU thu nhận khung hình và thực hiện tiền xử lý (resize, letterbox, normalize).
2. Dữ liệu đã tiền xử lý được gửi qua USB 3.0 đến Edge TPU.
3. Edge TPU thực hiện toàn bộ suy luận mô hình AI (INT8 quantized).
4. Kết quả (bounding box, confidence) được trả về CPU.
5. CPU hậu xử lý (NMS, vẽ bbox) và xuất ra framebuffer/màn hình.

Ưu điểm: Tăng tốc suy luận hàng chục lần, giải phóng CPU, hiệu quả năng lượng cao, cải thiện UX tổng thể.
Nhược điểm: Tăng chi phí, phụ thuộc hệ sinh thái TensorFlow Lite, giới hạn loại mô hình (phải biên dịch cho Edge TPU).

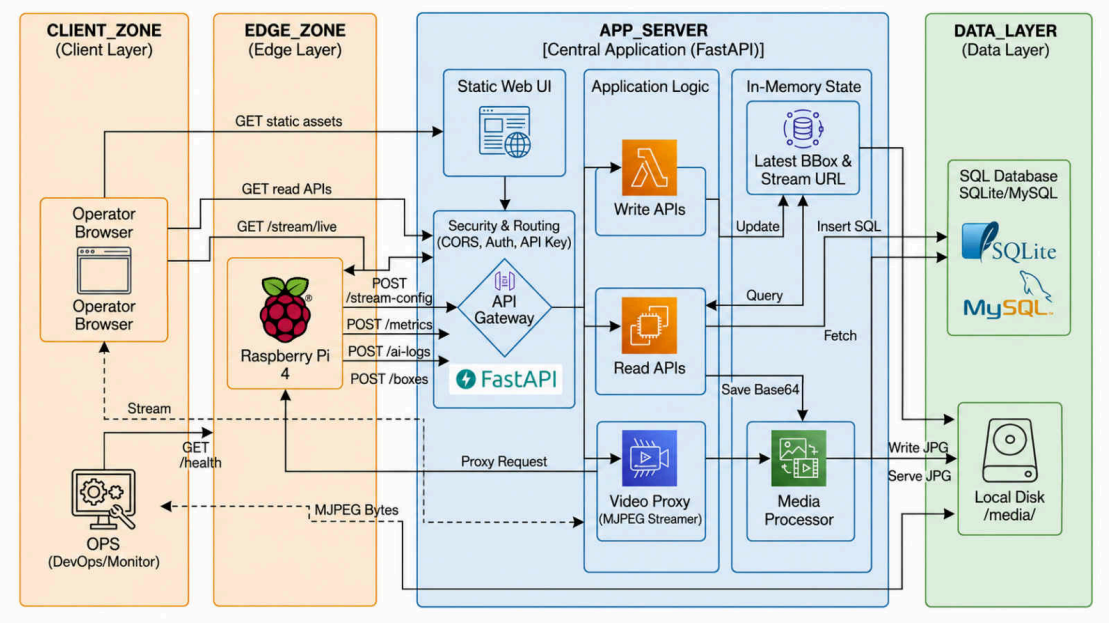
3.1.3 So sánh hai giải pháp phần cứng

Bảng 3.1: *So sánh hai giải pháp phần cứng*

Tiêu chí	CPU thuần (GP1)	Edge TPU (GP2)
Runtime suy luận	NCNN (C++)	TFLite + Edge TPU
Hiệu năng suy luận	5–15 FPS	20–40+ FPS
Tải CPU khi suy luận	80–100%	20–40%
Chi phí phần cứng	Thấp	Trung bình
Tiêu thụ năng lượng	Cao (CPU full load)	Thấp hơn tổng thể
Linh hoạt mô hình	Cao (FP32/FP16/INT8)	Hạn chế (INT8 Edge TPU)

3.2 Kiến trúc Dashboard giám sát từ xa

Hệ thống Dashboard cho phép người vận hành giám sát hoạt động của thiết bị ống nhòm AI từ xa thông qua trình duyệt web. Kiến trúc được thiết kế theo mô hình **bốn vùng** (four-zone architecture) nhằm tách biệt rõ ràng các tầng chức năng.



Hình 3.3: *Kiến trúc bốn vùng của hệ thống Dashboard giám sát*

3.2.1 Tổng quan kiến trúc bốn vùng

- **CLIENT_ZONE (Client Layer):** Trình duyệt web của người vận hành (Operator Browser) và hệ thống giám sát vận hành (OPS/DevOps Monitor). Truy cập thông qua HTTPS hoặc kết nối trực tiếp MJPEG.
- **EDGE_ZONE (Edge Layer):** Raspberry Pi chạy ứng dụng suy luận AI (EdgeVisionRT hoặc CoralVisionRT). Đẩy dữ liệu telemetry lên Dashboard qua các API:
  - `POST /metrics` — gửi số liệu FPS, latency, nhiệt độ CPU, RAM.
  - `POST /stream-config` — đăng ký URL MJPEG stream.
  - `POST /boxes` — gửi bounding box dạng YOLO TXT.
  - `POST /ai-log` — gửi nhật ký suy luận AI.
- **APP_SERVER (Central Application):** Ứng dụng FastAPI chạy trên cổng 8000, bao gồm:
  - *API Gateway* với CORS, xác thực API Key, định tuyến request.
  - *Application Logic* xử lý nghiệp vụ, quản lý trạng thái in-memory (latest BBox, Stream URL).
  - *Video Proxy* (MJPEG Streamer) — proxy luồng video từ Pi đến Client.
  - *Media Processor* — xử lý snapshot JPEG, lưu trữ Base64.

  - *Static Web UI* — giao diện HTML/JS/Chart.js phục vụ biểu đồ thời gian thực.

- **DATA_LAYER (Data Layer):** SQLite/MySQL lưu trữ chuỗi thời gian metrics; thư mục `/media/` lưu ảnh snapshot phục vụ truy xuất lịch sử.

3.2.2 Luồng dữ liệu Dashboard

Hệ thống Dashboard hoạt động theo hai luồng chính:

Luồng ghi (Write APIs): Ứng dụng suy luận AI trên Pi định kỳ gửi metrics, bounding box và snapshot lên FastAPI qua HTTP POST. Dữ liệu metrics được lưu vào SQLite; bounding box và stream URL được giữ trong bộ nhớ RAM (in-memory state) để phục vụ truy vấn nhanh.

Luồng đọc (Read APIs): Trình duyệt web gọi các endpoint `GET /latest`, `GET /timeseries`, `GET /boxes` để lấy dữ liệu mới nhất và vẽ biểu đồ. Video MJPEG được proxy qua endpoint `GET /stream` hoặc truy cập trực tiếp qua Tailscale VPN (`http://100.x.x.x:8080/stream`).

3.2.3 Truy cập từ xa qua Internet

Hệ thống sử dụng kết hợp hai kênh truy cập:

- **Cloudflare Tunnel:** Tạo kết nối HTTPS an toàn từ Internet đến FastAPI (`127.0.0.1:8000`) mà không cần mở port hay IP tĩnh. Phục vụ Dashboard web, metrics và snapshot cho người dùng công khai.

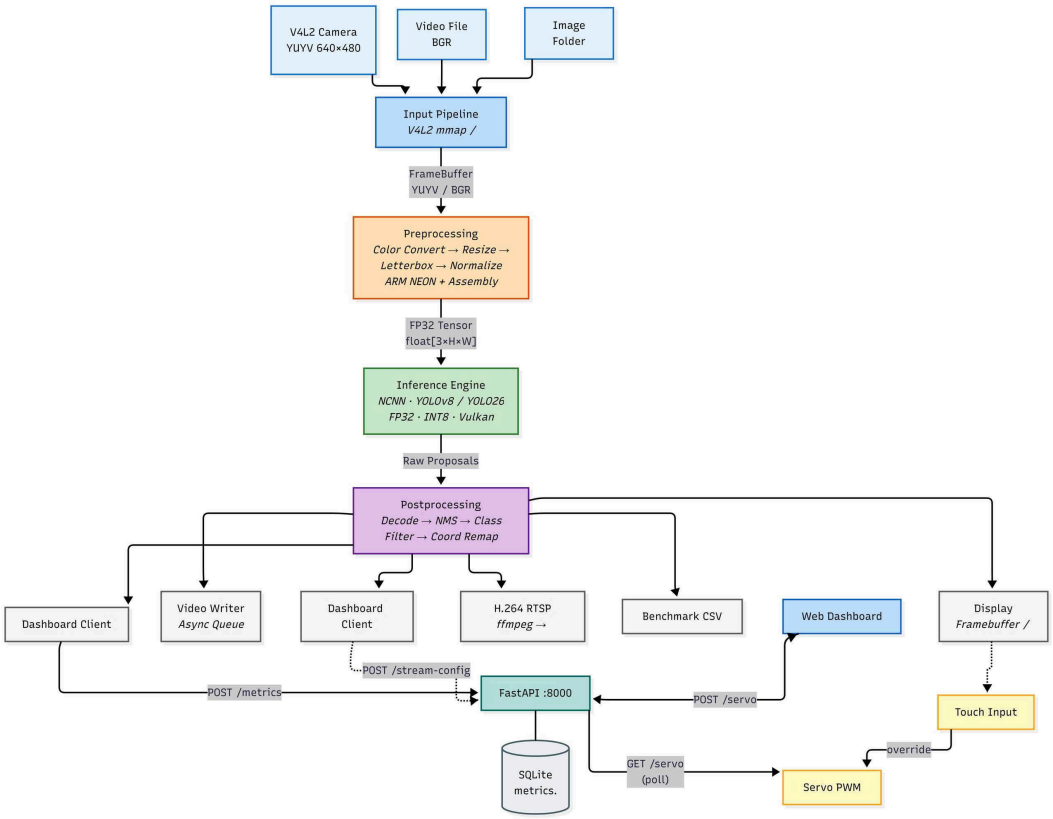
- **Tailscale VPN:** Mạng mesh WireGuard cấp IP `100.x.x.x`. Phục vụ luồng video MJPEG chất lượng cao với độ trễ thấp cho người dùng được ủy quyền.

3.3 Kiến trúc xử lý AI

Phần này trình bày chi tiết pipeline xử lý AI của hai nhánh triển khai: **EdgeVisionRT** (giải pháp CPU thuần, viết bằng C++) và **CoralVisionRT** (giải pháp Edge TPU, viết bằng Python). Cả hai nhánh đều chia sẻ cùng kiến trúc pipeline bốn giai đoạn: *Input Pipeline* → *Preprocessing* → *Inference Engine* → *Postprocessing*, nhưng khác nhau về cách triển khai và tối ưu hóa.

3.3.1 EdgeVisionRT — Pipeline suy luận C++/NCNN

EdgeVisionRT là hệ thống suy luận AI hiệu năng cao được viết hoàn toàn bằng C++, tối ưu hóa sâu cho kiến trúc ARM64 trên Raspberry Pi. Hệ thống sử dụng framework NCNN (Tencent) làm runtime suy luận và khai thác triệt để tập lệnh ARM NEON cùng kernel assembly tùy chỉnh.



Hình 3.4: *Kiến trúc pipeline xử lý AI của EdgeVisionRT (CPU)*

Các mô-đun chính:

1. **Input Pipeline**: Thu nhận khung hình từ ba nguồn: camera V4L2 (định dạng YUYV, sử dụng `mmap` zero-copy), tệp video (OpenCV `VideoCapture` trả BGR), hoặc thư mục ảnh. Module cung cấp interface thống nhất `FrameBuffer` cho các giai đoạn sau.

2. **Preprocessing**: Đây là tầng được tối ưu hóa sâu nhất với các kỹ thuật:

   - *Chuyển đổi không gian màu*: YUYV→RGB sử dụng NEON SIMD (xử lý 16 pixel/lần).
   - *Letterbox resize*: Co giãn bilinear giữ tỷ lệ, đệm xám (giá trị 114).
   - *Chuẩn hóa + chuyển layout*: RGB HWC → CHW FP32 (÷255) bằng kernel assembly ARM64.
   - *Buffer tĩnh*: Cấp phát bộ nhớ aligned 64B một lần duy nhất, tránh `malloc` trong vòng lặp chính.
   - *Đường tắt BGR*: Với video/ảnh BGR, bỏ qua hoàn toàn bước chuyển đổi YUYV, thực hiện resize + BGR→RGB + normalize trong một lượt duy nhất.

3. **Inference Engine**: Sử dụng NCNN với các tối ưu hóa:
   - Zero-copy input: tạo `ncnn::Mat` trực tiếp từ con trỏ FP32 mà không sao chép dữ liệu.
   - Hỗ trợ đa chế độ: FP32, FP16 packed/storage, INT8 quantized, Vulkan GPU (VideoCore VII trên Pi 5).
   - Tự động phát hiện kích thước đầu vào/ra của mô hình từ metadata.
   - Pool allocator và workspace allocator được cấu hình exact-match để giảm phân mảnh bộ nhớ.
4. **Postprocessing**:
   - Transpose output tensor bằng kernel assembly tùy chỉnh ($[C, N] \rightarrow [N, C]$).
   - Giải mã đầu ra YOLOv8: tách tọa độ bbox (cx, cy, w, h) và class scores.
   - Non-Maximum Suppression (NMS) trên danh sách đã sắp xếp theo confidence.
   - Ánh xạ tọa độ ngược về không gian ảnh gốc (undo letterbox padding & scale).
5. **Output & Tích hợp:** Kết quả sau postprocessing được phân phối song song đến nhiều đầu ra:
   - *Framebuffer Display*: Vẽ bbox/nhãn lên `/dev/fb0` qua DRM/KMS (RGB565/BGRA), không cần X11.
   - *Video Writer*: Ghi video có bbox ra tệp (async queue).
   - *MJPEG Stream Server*: Phục vụ luồng video JPEG qua HTTP (cổng 8080).
   - *H.264 RTSP*: Pipe BGR qua `ffmpeg` với codec `h264_v4l2m2m` đến MediaMTX.
   - *Dashboard Client*: POST metrics/bbox/snapshot lên FastAPI.
   - *Benchmark CSV*: Xuất số liệu latency, FPS, nhiệt độ theo cửa sổ thời gian.
   - *Servo Control*: Điều khiển motor servo qua PWM GPIO (tùy chọn).
   - *Touch Input*: Nhận sự kiện cảm ứng từ màn hình LCD.

3.3.2 CoralVisionRT — Pipeline suy luận Python/Edge TPU

CoralVisionRT là nhánh triển khai sử dụng Google Coral Edge TPU để dỡ tải hoàn toàn tác vụ suy luận AI khỏi CPU. Hệ thống được viết bằng Python với kiến trúc đa luồng (multi-threaded), tách biệt các worker: camera, inference, display, servo và touch.

Các mô-đun chính:

1. **Camera Worker**: Sử dụng OpenCV `VideoCapture` thu khung hình BGR (640×480, 30 FPS). Chạy trên luồng riêng, cung cấp phương thức `get_latest_frame()` cho inference worker.

2. **Infer Worker**:
   - Tải mô hình `.tflite` đã biên dịch cho Edge TPU thông qua `libedgetpu.so.1` delegate.
   - Tự động phát hiện layout đầu vào (NHWC/NCHW) và kiểu dữ liệu (INT8/UINT8/FLOAT32).
   - Tiền xử lý: letterbox resize giữ tỷ lệ, chuyển BGR→RGB, chuẩn hóa theo yêu cầu mô hình.
   - Với mô hình INT8: dịch pixel từ không gian UINT8 (0–255) sang INT8 (−128 đến 127).
   - Hậu xử lý: dequantize output, tách bbox/scores, ánh xạ tọa độ ngược, NMS bằng `cv2.dnn.NMSBoxes`.
   - Fallback tự động về CPU nếu không tìm thấy phần cứng Edge TPU.
3. **Display Worker**: Ghi trực tiếp lên framebuffer `/dev/fb0` (RGB565), vẽ bbox, nhãn, FPS, thời gian suy luận, thanh servo. Chạy trên luồng riêng để tránh block pipeline chính.
4. **Dashboard Client**: Luồng nền gửi metrics (FPS, latency, nhiệt độ, CPU%, RAM%), bbox TXT và snapshot JPEG lên FastAPI. Hỗ trợ đăng ký MJPEG stream URL (ưu tiên Tailscale IP).
5. **Servo & Touch Workers:** điều khiển motor servo qua PWM GPIO; nhận sự kiện cảm ứng. Dashboard hỗ trợ điều khiển servo từ xa qua endpoint `POST /servo`.

3.3.3 So sánh hai nhánh xử lý AI

Bảng 3.2: *So sánh kiến trúc EdgeVisionRT và CoralVisionRT*

Tiêu chí	**EdgeVisionRT**	**CoralVisionRT**
Ngôn ngữ	C++ (biên dịch AOT)	Python (đa luồng)
Runtime	NCNN	TFLite + Edge TPU delegate
Camera	V4L2 mmap (zero-copy YUYV)	OpenCV VideoCapture (BGR)
Tiền xử lý	ARM NEON + Assembly tùy chỉnh	NumPy + OpenCV
Lượng tử hóa	FP32, FP16	INT8 (bắt buộc cho Edge TPU)
Hiển thị	DRM/KMS framebuffer (C++)	Framebuffer RGB565 (Python)
MJPEG	Server C++ tích hợp	`ThreadingHTTPServer` stdlib
Dashboard	`httplib` (C++)	`requests` (Python)

Chương 4

Dataset và phương pháp đánh giá

4.1 Tổng quan về Bộ dữ liệu C2A (Combination to Application)

Bộ dữ liệu C2A được giới thiệu bởi Nihal và cộng sự (2024) [3], được thiết kế chuyên biệt để giải quyết thách thức thiếu hụt dữ liệu huấn luyện cho các mô hình học máy trong các kịch bản tìm kiếm và cứu nạn (SAR) sử dụng UAV. Đây là bộ dữ liệu tổng hợp (synthesized dataset) được tạo ra bằng quy trình phủ (overlay) hình ảnh con người lên các bối cảnh thảm họa thực tế.

4.1.1 Cấu trúc và Thành phần dữ liệu

Bộ dữ liệu được xây dựng dựa trên sự kết hợp của hai nguồn dữ liệu chính:

- **Nền cảnh thảm họa (Background):** Sử dụng bộ dữ liệu AIDER (Aerial Image Dataset for Emergency Response Applications), bao gồm các hình ảnh thực tế chụp từ trên cao về 4 loại thảm họa chính: Hỏa hoạn/Khói (Fire/Smoke), Lũ lụt (Flood), Tòa nhà sập/Đống đổ nát (Collapsed Building/Rubble), và Tai nạn giao thông (Traffic Accidents).
- **Đối tượng con người (Foreground):** Sử dụng bộ dữ liệu LSP/MPII-MPHB chứa 26,675 hình ảnh với 29,732 trường hợp cơ thể người. Các đối tượng này được tách nền và ghép vào ảnh thảm họa với các kỹ thuật xử lý để mô phỏng sự che khuất (occlusion) thường thấy trong thực tế.

4.1.2 Thống kê và Đặc điểm kỹ thuật

- **Quy mô dữ liệu:** Bộ dữ liệu bao gồm tổng cộng 10,215 hình ảnh với hơn 360,000 đối tượng con người được gán nhãn (annotated instances).
- **Độ phân giải:** Kích thước hình ảnh rất đa dạng, trải rộng từ 123×152 pixel đến các ảnh độ phân giải cao 5184×3456 pixel. Phổ biến nhất là các ảnh có chiều rộng từ 322 đến 600 pixel (chiếm hơn 50%).
- **Kích thước đối tượng (Object Size):** Do đặc thù góc nhìn từ UAV ở độ cao lớn, kích thước đối tượng người trong dataset rất nhỏ. Cụ thể, 47% đối tượng có kích thước dưới 10 pixel và 52% nằm trong khoảng 10-50 pixel. Không có đối tượng

nào lớn hơn 300 pixel. Đây là một thách thức lớn đối với các mô hình phát hiện vật thể thông thường.

- **Mật độ đối tượng:** Khác với các bộ dữ liệu thông thường như MS COCO (trung bình 7.7 đối tượng/ảnh), C2A có mật độ đối tượng rất cao, tập trung nhiều nhất ở khoảng 20-40 đối tượng/ảnh, mô phỏng sự hỗn loạn thực tế tại hiện trường thảm họa.

4.1.3 Hệ thống nhãn (Annotation)

Điểm đặc biệt của C2A là việc tích hợp thông tin ngữ cảnh chi tiết. Dữ liệu không chỉ bao gồm bounding box mà còn phân loại 5 tư thế con người khác nhau, giúp đánh giá mức độ nghiêm trọng của nạn nhân:

1. Cong người (Bent)
2. Quỳ (Kneeling)
3. Nằm (Lying) - thường chỉ dấu hiệu chấn thương cần cấp cứu.
4. Ngồi (Sitting)
5. Đứng thẳng (Upright)

Việc sử dụng bộ dữ liệu C2A giúp cải thiện đáng kể khả năng tổng quát hóa của mô hình trong các điều kiện thảm họa phức tạp so với việc chỉ sử dụng các bộ dữ liệu người thông thường.

- C2A Dataset: Human Detection in Disaster Scenarios từ Kaggle "Reference Dataset" , làm dataset chính. Lý do: Dataset này có 10.215 hình ảnh có độ phân giải cao, hơn 360.000 annotated human instances, với 5 loại tư thế con người (Bent, Kneeling, Lying, Sitting, Upright) và 4 loại kịch bản thiên tai (Fire/Smoke, Flood, Collapsed Building/Rubble, Traffic Accidents).



Hình 4.1: *Hình ảnh các thảm họa về hỏa hoạn, ngập lụt, công trình, giao thông*



Hình 4.2: *Hình ảnh các thảm họa về hỏa hoạn, ngập lụt, công trình, giao thông (có label)*

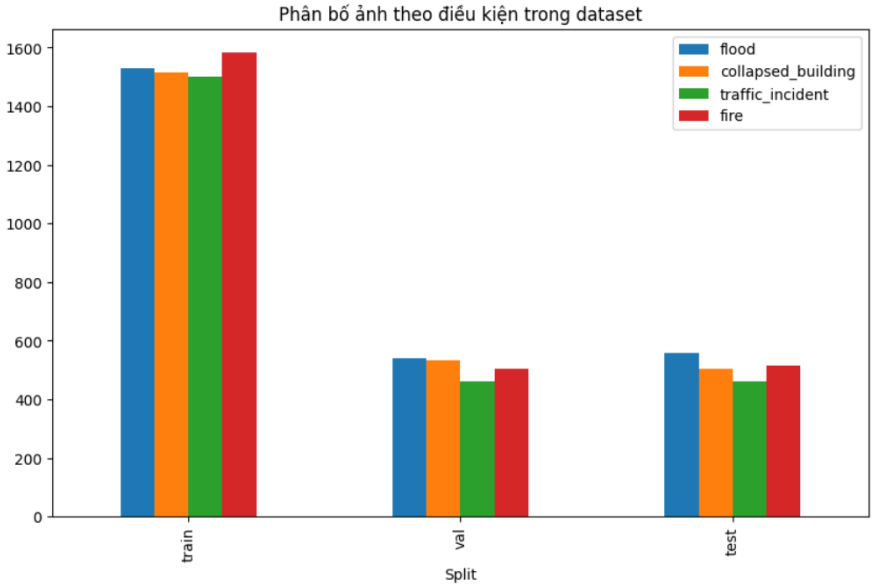
4.1.4 Đánh giá dataset

- Kiểm tra phân bố ảnh theo tỉ lệ train, valid, test

	Split	Số ảnh
0	train	6129
1	val	2043
2	test	2043

Hình 4.3: *Thống kê số lượng ảnh theo tập*

- Kiểm tra phân bố ảnh theo kịch bản thảm họa



Hình 4.4: *Thống kê phân bố ảnh theo điều kiện trong dataset*

Kết quả phân tích cho thấy dữ liệu được phân bố tương đối đồng đều giữa các bối cảnh trong từng tập, không xuất hiện hiện tượng mất cân bằng nghiêm trọng. Điều này cho

thấy dataset có chất lượng tốt và phù hợp để huấn luyện cũng như đánh giá mô hình YOLO trong bài toán phát hiện sự cố và thiên tai.

4.2 Data augmentation

4.2.1 Cơ sở lựa chọn phương pháp tăng cường dữ liệu

Trong ứng dụng tìm kiếm cứu nạn (SAR) thông qua ảnh ống nhòm từ UAV, tính mạng nạn nhân phụ thuộc vào khả năng phát hiện mục tiêu của hệ thống AI trong các điều kiện môi trường khắc nghiệt. Thực tế cho thấy, các thuật toán nhận diện như YOLO thường bị suy giảm độ chính xác nghiêm trọng khi gặp sương mù dày đặc hoặc mưa lớn do các đặc trưng hình thái học bị che khuất [23].

Mục tiêu của công đoạn này là:

- Xây dựng một tập dữ liệu tổng hợp (synthetic dataset - tập dữ liệu nhân tạo do máy tính sinh ra) mô phỏng các điều kiện thời tiết xấu dựa trên các quy luật vật lý, thay vì chỉ phủ nhiễu ngẫu nhiên.
- Đảm bảo mô hình không bị "quên" cách nhận diện trong điều kiện lý tưởng. Tránh hiện tượng **Catastrophic Forgetting** (Quên thảm khốc - tình trạng AI học các đặc trưng nhiễu mới nhưng lại quên mất cách nhận diện vật thể ở điều kiện chuẩn ban đầu) bằng cách duy trì tỷ lệ 1:1 giữa dữ liệu gốc và dữ liệu tăng cường [23].
- Tạo ra mức độ nhiễu (intensity) đa dạng với các ngưỡng an toàn để mạng nơ-ron hình thành **Shape Bias** (Thiên kiến hình dáng - ép mô hình học cấu trúc hình học của vật thể thay vì học vẹt các đốm nhiễu), qua đó tránh hiện tượng **Hallucination** (Ảo giác - AI tự tưởng tượng ra vật thể không hề tồn tại do bị nhiễu đánh lừa) [24].

4.2.2 Tổng quan nghiên cứu

Các nghiên cứu gần đây chứng minh rằng phương pháp **Analytical Data Augmentation** (Tăng cường dữ liệu phân tích - sử dụng công thức toán lý rõ ràng để sinh dữ liệu) kết hợp **Depth Map** (Bản đồ chiều sâu - ảnh thể hiện khoảng cách xa gần) vượt trội hơn các mạng sinh ảnh (GAN) vì giữ nguyên được cấu trúc vật lý và khung bao vị trí mục tiêu (bounding box) [23]. Bộ tiêu chuẩn đánh giá KITTI-C cũng khuyến nghị sử dụng **multi-severity** (nhiễu đa cấp độ từ nhẹ đến nặng) kết hợp với ngưỡng chặn (cap) để mô hình có khả năng nhận diện hình dáng cục bộ (partial shape bias) ngay cả khi bị che khuất [7].

4.2.3 Tăng cường dữ liệu dựa trên mô hình vật lý (Analytical Data Augmentation)'

Việc thu thập dữ liệu thực tế (Real-world Data) có gán nhãn chính xác trong điều kiện bão lũ nguy hiểm là cực kỳ khó khăn và không an toàn. Do đó, phương pháp ***Tăng***

cường dữ liệu dựa trên mô hình vật lý (Analytical Data Augmentation) là lời giải tối ưu cho bài toán này.

Khác với các phương pháp Generative (như GAN) có thể làm thay đổi cấu trúc hình ảnh hoặc tạo ra các chi tiết giả (hallucinations) – điều tối kỵ trong cứu hộ vì có thể dẫn đến xác định sai vị trí nạn nhân – phương pháp Analytical tuân theo các quy luật vật lý (như định luật ***Beer-Lambert*** cho sương mù hay mô phỏng hạt mưa dựa trên độ sâu).

Độ sâu trong mô phỏng thời tiết: Trong các phương pháp tăng cường dữ liệu truyền thống, nhiễu (noise) thường được phủ lên toàn bộ ảnh một cách đồng nhất. Tuy nhiên, trong điều kiện thực tế, các hiện tượng thời tiết như sương mù và mưa có tính chất vật lý phụ thuộc vào khoảng cách (distance-dependent). Theo nghiên cứu của Gupta và cộng sự [23], để tạo ra dữ liệu tổng hợp (synthetic data) có tính chân thực cao phục vụ cho bài toán phát hiện vật thể (Object Detection), việc tích hợp thông tin chiều sâu là bắt buộc.

Cụ thể, các hiệu ứng quang học tuân theo định luật Beer-Lambert về sự suy giảm ánh sáng trong môi trường, được mô tả như sau:

$$I_{fog}(x) = I(x)e^{-\beta d(x)} + I_{al}(1 - e^{-\beta d(x)}) \tag{4.1}$$

Do đó, mục tiêu là ước lượng bản đồ độ sâu D (chứa các giá trị $d(x)$) từ một ảnh đầu vào đơn lẻ (Single-image Depth Estimation) để làm tham số đầu vào cho phương trình (4.1) và các thuật toán tạo mưa/tuyết.

- **Mô hình MiDaS và Kiến trúc DPT** Để giải quyết bài toán ước lượng độ sâu từ một ảnh đơn, đề tài sử dụng mô hình MiDaS v3.0 với kiến trúc nền tảng (backbone) là Dense Prediction Transformer (DPT-Large) [10].

Lý do lựa chọn MiDaS:

1. **Khả năng tổng quát hóa cao (Zero-shot Cross-dataset Transfer):** MiDaS được huấn luyện trên tập dữ liệu khổng lồ đa nguồn (trộn lẫn giữa dữ liệu phim 3D, ảnh stereo, và ảnh laser). Điều này giúp mô hình hoạt động tốt trên các dữ liệu chưa từng thấy (unseen data) như ảnh thảm họa từ trên cao (top-down view) trong đề tài này.

2. **Kiến trúc Transformer:** DPT sử dụng cơ chế attention thay vì mạng tích chập truyền thống (CNN), giúp nắm bắt ngữ cảnh toàn cục (global context) tốt hơn, từ đó tách biệt chính xác hơn các vật thể (tòa nhà, xe cộ) ra khỏi nền đất.

4.2.4 Quy trình tính toán và Chuyển đổi không gian

Mô hình MiDaS không trực tiếp xuất ra khoảng cách mét (Metric Depth) mà xuất ra nghịch đảo độ sâu tương đối (Relative Inverse Depth hay Disparity), ký hiệu là δ. Để phù hợp với các công thức vật lý [23], , thực hiện bước chuyển đổi không gian giá trị như sau:

Dự đoán bản đồ Disparity Với ảnh đầu vào $I \in \mathbb{R}^{H \times W \times 3}$, qua mạng DPT thu được bản đồ dự đoán δ_{pred}:

$$\delta_{pred} = \mathcal{F}_{MiDaS}(I) \tag{4.2}$$

Trong đó, giá trị δ_{pred} càng lớn tương ứng với vật thể càng gần, và δ_{pred} càng nhỏ tương ứng với vật thể càng xa.

Chuyển đổi sang Metric Depth (Khoảng cách thực) Để thu được giá trị $d(x)$ dùng cho công thức (4.1), ta cần nghịch đảo giá trị dự đoán. Áp dụng phép biến đổi dựa trên mô hình camera lỗ kim (pinhole camera model):

$$d(x) = \frac{f \cdot b}{\delta_{pred}(x) \cdot s + \epsilon} \tag{4.3}$$

Trong đó:

- f: Tiêu cự giả định của camera (focal length).
- b: Độ dài cơ sở giả định (baseline).
- s: Hệ số tỉ lệ ảnh (image scale).
- ϵ: Hằng số nhỏ ($1e^{-3}$) để tránh lỗi chia cho 0.

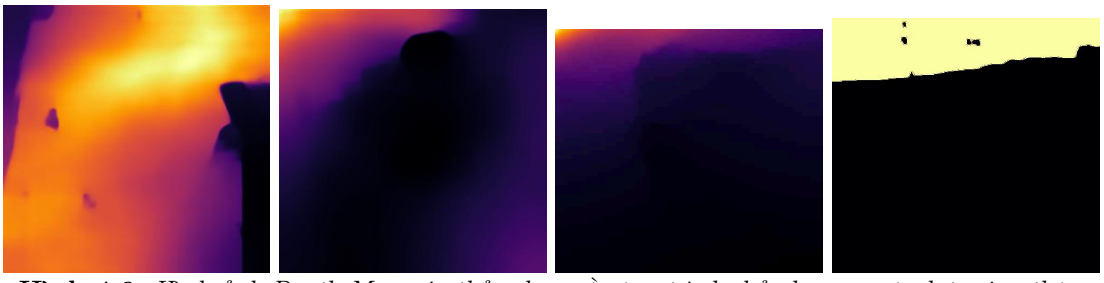
Trong thực nghiệm, bản đồ độ sâu $d(x)$ sau khi tính toán được chuẩn hóa về đoạn $[0, 1]$ hoặc được giữ nguyên giá trị tương đối để phục vụ tính toán ngưỡng che khuất (occlusion threshold) cho các lớp mưa và tuyết.

4.2.5 Kết quả thực nghiệm

Quá trình xử lý được thực hiện trên các tập dữ liệu lũ lụt, hỏa hoạn và giao thông. Kết quả cho thấy mô hình MiDaS phân tách hiệu quả các cấu trúc hình học nổi bật (nhà cao tầng, phương tiện) so với mặt đường và nước, mặc dù góc chụp từ trên cao là một thách thức lớn.



Hình 4.5: *Hình ảnh các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông*



Hình 4.6: *Hình ảnh Depth Map các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông*

Kết quả ước lượng độ sâu. Hàng trên: Ảnh gốc đầu vào (Input). Hàng dưới: Bản đồ độ sâu (Depth Map) tương ứng. (Lưu ý: Trong bản đồ nhiệt, màu tối thể hiện vật thể ở gần, màu sáng thể hiện vật thể ở xa). Thuật toán trích xuất độ sâu được tóm tắt trong Mã giả 1.

Input: Ảnh đầu vào I_{RGB}
Output: Bản đồ độ sâu D_{metric}
$I_{norm} \leftarrow Normalize(Resize(I_{RGB}, (384, 384)))$
$\delta_{disp} \leftarrow MiDaS_Model(I_{norm})$ `// Dự đoán nghịch đảo độ sâu`
$\delta_{resized} \leftarrow Interpolate(\delta_{disp}, size = OriginalSize)$
Xử lý ngoại lệ: $\delta_{resized}[\delta_{resized} < 0] \leftarrow 0$
Chuyển đổi: $D_{metric} \leftarrow \frac{baseline \times focal}{\delta_{resized} \times scale}$ `// Theo công thức 4.3`
return D_{metric}

Algorithm 1: Quy trình tạo bản đồ độ sâu cho mô phỏng thời tiết



Hình 4.7: *ảnh gốc và ảnh sau khi áp dụng hiệu ứng sương mù (chưa áp dụng Depth map)*



Hình 4.8: *Ảnh sau khi áp dụng Depth map và hiệu ứng sương mù*

Mã nguồn tạo hiệu ứng sương mù (chưa áp dụng Depth map) Đoạn mã minh họa quá trình tạo hiệu ứng sương mù nhân tạo cho ảnh đầu vào. Thuật toán sử dụng một lớp sương mù có cường độ xác định và trộn với ảnh gốc để tạo ảnh mới.

```python
import cv2
import numpy as np

def add_fog(image, fog_intensity=0.5):

    h, w, _ = image.shape
    fog_layer = np.ones((h, w, 3), dtype=np.uint8) * 255
    fog_image = cv2.addWeighted(
        image, 1 - fog_intensity,
        fog_layer, fog_intensity,
        0
    )
    return fog_image


img = cv2.imread("input.jpg")
fog_img = add_fog(img, fog_intensity=0.6)
cv2.imwrite("fog_output.jpg", fog_img)
```

4.2.6 Mô phỏng hiệu ứng Sương mù (Fog Simulation)

Sau khi thu được bản đồ độ sâu $d(x)$ từ mô hình MiDaS, nhóm thực hiện mô phỏng hiện tượng sương mù dựa trên Mô hình Tán xạ Khí quyển (Atmospheric Scattering Model)[12]. Đây là mô hình vật lý tiêu chuẩn trong thị giác máy tính để mô tả sự suy giảm tầm nhìn do các hạt lơ lửng trong không khí.

Cơ sở toán học Mô hình tạo sương mù được định nghĩa bởi phương trình:

$$I_{fog}(x) = J(x)t(x) + A(1 - t(x)) \tag{4.4}$$

Trong đó:

- $I_{fog}(x)$: Cường độ điểm ảnh quan sát được (ảnh sau khi thêm sương).
- $J(x)$: Cường độ điểm ảnh gốc (Scene Radiance).
- A: Ánh sáng môi trường toàn cục (Global Atmospheric Light), thường được chọn là giá trị cường độ sáng cao nhất trong ảnh hoặc một màu xám/trắng đặc trưng ($A \approx [0.8, 0.9]$).
- $t(x)$: Bản đồ truyền dẫn (Transmission Map), thể hiện lượng ánh sáng từ vật thể đi tới camera mà không bị tán xạ.

Theo định luật Beer-Lambert, hệ số truyền dẫn $t(x)$ phụ thuộc trực tiếp vào khoảng cách:

$$t(x) = e^{-\beta d(x)} \tag{4.5}$$

Với β là hệ số suy giảm (attenuation coefficient), đại diện cho mật độ sương mù.

- Nếu β lớn: Sương mù dày đặc, tầm nhìn giảm nhanh theo độ sâu.

- Nếu β nhỏ: Sương mù mỏng, tầm nhìn xa hơn.

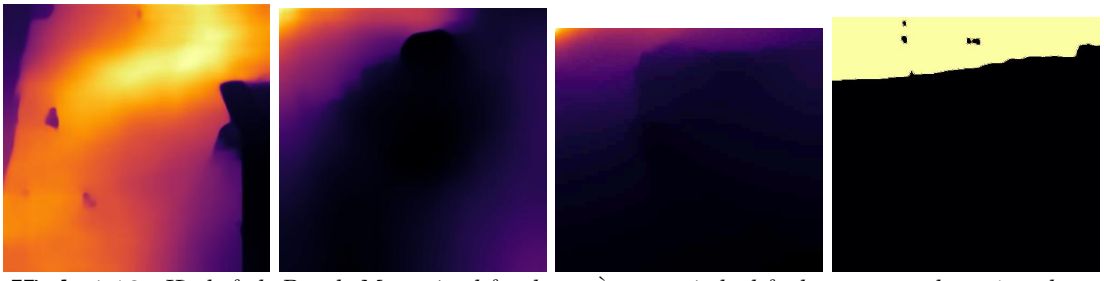
Quy trình thực hiện Khác với các phương pháp phủ lớp mờ (overlay) thông thường làm giảm độ tương phản toàn bộ ảnh, phương pháp này sử dụng thông tin độ sâu $d(x)$ để tạo ra hiệu ứng sương mù thể tích (volumetric fog):

1. Các vật thể ở gần (giá trị $d(x)$ nhỏ) sẽ có $t(x) \approx 1$, do đó giữ nguyên màu sắc gốc.
2. Các vật thể ở xa (giá trị $d(x)$ lớn) sẽ có $t(x) \to 0$, do đó màu sắc bị hòa trộn hoàn toàn với ánh sáng môi trường A (bị "bạc"màu).

Điều này giúp dữ liệu huấn luyện giữ được các đặc trưng quan trọng của đối tượng ở gần (nơi UAV có khả năng phát hiện tốt nhất) trong khi vẫn mô phỏng chính xác sự mất mát thông tin ở xa.



Hình 4.9: *Hình ảnh các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông*



Hình 4.10: *Hình ảnh Depth Map các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông*



Hình 4.11: *Hình ảnh sau khi áp dụng Depth map và hiệu ứng sương mù các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông*

4.2.7 Mô phỏng hiệu ứng Mưa (Rain Simulation)

Mô phỏng mưa trong ảnh tĩnh là một thách thức lớn vì mưa bao gồm hai thành phần vật lý: (1) Các vệt mưa (Rain streaks) do chuyển động nhanh của hạt nước và (2) Hiệu ứng che khuất/mờ do màn mưa (Rain veil).

Kỹ thuật tạo vệt mưa (Rain Streaks Synthesis) Để tạo ra các vệt mưa chân thực, nhóm sử dụng phương pháp tạo nhiễu ngẫu nhiên kết hợp với làm mờ chuyển động (Motion Blur), được tham số hóa như sau:

Bước 1: Tạo nhiễu hạt (Salt-and-Pepper Noise).
Tạo một lớp mặt nạ nhiễu N có cùng kích thước với ảnh, trong đó các điểm ảnh trắng (giá trị 1) được rải ngẫu nhiên với mật độ ρ thấp để đại diện cho các hạt mưa.

Bước 2: Mô phỏng chuyển động (Motion Blur).
Áp dụng bộ lọc làm mờ chuyển động tuyến tính lên lớp nhiễu N để tạo thành các vệt R. Bộ lọc này được xác định bởi hai tham số:

- Độ dài vệt mưa L: Mô phỏng tốc độ rơi (mưa càng to, tốc độ càng nhanh, vệt càng dài).
- Góc rơi θ: Mô phỏng hướng gió (thường dao động từ $-15°$ đến $15°$ so với phương thẳng đứng).

Bước 3: Tích hợp độ sâu (Depth-aware Rain).
Trong thực tế, hạt mưa ở gần camera sẽ to và dài hơn, trong khi hạt mưa ở xa sẽ nhỏ và dày đặc hơn. Nhóm sử dụng bản đồ độ sâu $d(x)$ để điều chỉnh cường độ hòa trộn của lớp mưa:

$$I_{rain}(x) = I_{fog}(x) + \alpha \cdot R(x) \tag{4.6}$$

Trong đó I_{fog} là ảnh nền (có thể đã có sương mù nhẹ do hơi nước), $R(x)$ là lớp vệt mưa, và α là hệ số hiển thị.

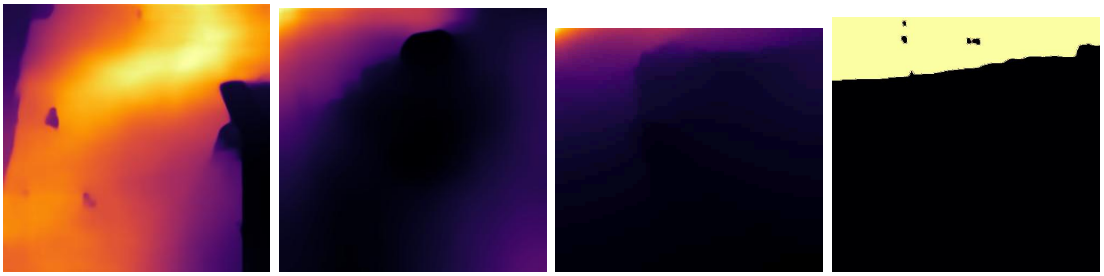
Bằng cách kết hợp cả sương mù và mưa, nhóm tạo ra được các mẫu dữ liệu "Heavy Rain"(Mưa lớn), nơi tầm nhìn bị hạn chế nghiêm trọng, ép buộc mô hình phải học các đặc trưng hình dáng (shape) thay vì dựa vào texture sắc nét.



Hình 4.12: *Ảnh sau khi áp dụng Depth map và hiệu ứng mưa*



Hình 4.13: *Hình ảnh các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông*



Hình 4.14: *Hình ảnh Depth Map các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông*



Hình 4.15: *Hình ảnh sau khi áp dụng Depth map và hiệu ứngmưa. Các vệt mưa được tạo ra với độ nghiêng ngẫu nhiên để mô phỏng tác động của gió trong các thảm họa về công trình, hỏa hoạn, ngập lụt, giao thông*

Kết quả đạt được

- Tập dữ liệu huấn luyện đã được mở rộng bằng hiệu ứng sương mù và mưa.
- Tăng độ đa dạng của dữ liệu huấn luyện, giúp mô hình thích nghi tốt hơn với các điều kiện môi trường khác nhau.
- Chuẩn bị đầy đủ dữ liệu cho giai đoạn huấn luyện và đánh giá mô hình trong các tuần tiếp theo.

4.3 So sánh hiệu quả sử dụng Depth trong Data Augmentation

4.3.1 Mục tiêu

Thí nghiệm được thiết lập nhằm phân tích định lượng tác động của thông tin độ sâu vật lý trong quy trình **Analytical Data Augmentation**. Mục tiêu cốt lõi là xác định liệu việc mô phỏng nhiễu thời tiết (sương mù, mưa) dựa trên quy luật truyền quang theo khoảng cách có giúp mô hình cải thiện tính bền vững (Robustness) và khả năng tổng quát hóa (Generalization) so với phương pháp tăng cường ngẫu nhiên truyền thống hay không.

Hai phương pháp được so sánh:

- **M4_with_depth**: Mô hình được huấn luyện bằng kỹ thuật Analytical Data Augmentation, sử dụng bản đồ độ sâu (Depth Map) để phân lớp nhiễu, đảm bảo các mục tiêu ở xa bị che khuất mạnh hơn các mục tiêu ở gần theo mô hình truyền quang thực tế.
- **M4_no_depth**: Mô hình được huấn luyện với các kỹ thuật tăng cường dữ liệu truyền thống (không tính đến chiều sâu vật lý khi thêm nhiễu thời tiết).

4.3.2 Thiết lập thực nghiệm

- **Dataset đánh giá**:
  - T1_C2A_Original (dữ liệu gốc)
  - T1_C2A_Weather (dữ liệu có nhiễu thời tiết)
  - T2_VisDrone (dữ liệu thực – seen domain)
  - T3_Heridal (unseen domain)
  - T3_SARD (unseen domain)
- **Chỉ số đánh giá**: mAP@50, mAP@50-95, AP75, Precision, Recall, F1-score

4.3.3 Kết quả thực nghiệm

Model	Dataset	mAP@50	mAP@50-95	AP75	Precision	Recall	F1-Score
M4_with_depth	T1_C2A_Original	0.4766	0.2602	0.2529	0.7877	0.5112	0.6200
M4_no_depth	T1_C2A_Original	0.4911	0.2555	0.2398	0.7265	0.5426	0.6212
M4_with_depth	T1_C2A_Weather	0.4768	0.2577	0.2508	0.7683	0.5152	0.6168
M4_no_depth	T1_C2A_Weather	0.3171	0.1466	0.1249	0.6440	0.3820	0.4795
M4_with_depth	T2_VisDrone	0.1254	0.0498	0.0290	0.6568	0.1498	0.2440
M4_no_depth	T2_VisDrone	0.0890	0.0345	0.0192	0.5143	0.1113	0.1830
M4_with_depth	T3_Heridal	0.1564	0.0615	0.0369	0.5554	0.2276	0.3229
M4_no_depth	T3_Heridal	0.1631	0.0591	0.0268	0.3179	0.2902	0.3034
M4_with_depth	T3_SARD	0.2687	0.1476	0.1472	0.6673	0.2790	0.3934
M4_no_depth	T3_SARD	0.2125	0.1026	0.0836	0.4882	0.2278	0.3106

Bảng 4.1: *So sánh hiệu năng giữa mô hình có và không sử dụng depth trong data augmentation*

Trên dữ liệu gốc (T1_C2A_Original)

- M4_no_depth đạt mAP@50 = 0.4911 (cao hơn)
- M4_with_depth đạt mAP@50 = 0.4766

Nhận xét: Không sử dụng depth cho kết quả tốt hơn nhẹ trên dữ liệu sạch do môi trường không có nhiễu vật lý phức tạp.

Trên dữ liệu có nhiễu thời tiết (T1_C2A_Weather)

- M4_with_depth: mAP@50 = 0.4768
- M4_no_depth: mAP@50 = 0.3171

Nhận xét: Sử dụng depth giúp cải thiện hiệu năng, do mô phỏng hiện tượng vật lý chính xác hơn.

Trên dữ liệu thực (T2_VisDrone)

- M4_with_depth: mAP@50 = 0.1254
- M4_no_depth: mAP@50 = 0.089

Nhận xét: Model sử dụng depth có khả năng tổng quát hóa tốt hơn trên dữ liệu thực.

Trên unseen domain (T3_Heridal)

- M4_with_depth: Recall = 0.2276
- M4_no_depth: Recall = 0.2902

Nhận xét: Kết quả không đồng nhất, cho thấy depth không phải lúc nào cũng cải thiện hiệu năng.

Trên unseen domain (T3_SARD)

- M4_with_depth: mAP@50 = 0.2687
- M4_no_depth: mAP@50 = 0.2125

Nhận xét: Depth tiếp tục cho kết quả tốt hơn trong môi trường thực tế.

4.3.4 Phân tích

Ưu điểm của phương pháp sử dụng depth

- Mô phỏng hiện tượng vật lý chính xác hơn (sương mù, mưa theo khoảng cách)
- Giúp mô hình học được cấu trúc không gian của cảnh

Nhược điểm của phương pháp không sử dụng depth

- Augmentation mang tính ngẫu nhiên
- Dễ dẫn đến hiện tượng **shortcut learning**

Xu hướng chung

Điều kiện	Depth tốt hơn
Dữ liệu sạch	Không rõ ràng
Dữ liệu nhiễu	Có
Dữ liệu thực	Có
Unseen domain	Không ổn định

Bảng 4.2: *So sánh hiệu quả sử dụng depth*

4.3.5 Kết luận

Việc sử dụng **depth trong data augmentation**:

- Không cần thiết đối với dữ liệu sạch
- Nhưng rất quan trọng trong môi trường thực tế có nhiễu (fog, rain)

Depth giúp:

- Tăng khả năng tổng quát hóa (generalization)
- Giảm hiện tượng shortcut learning
- Cải thiện hiệu năng trên dữ liệu thực và unseen domain

Do đó, phương pháp augmentation có sử dụng depth được lựa chọn cho hệ thống đề xuất.

4.4 Chiến lược Phân tầng Dữ liệu (Stratified Sampling)

Stratified Sampling (Lấy mẫu phân tầng) là phương pháp chia nhỏ dữ liệu theo từng nhóm đặc trưng để đảm bảo tính đại diện. Để duy trì chất lượng huấn luyện, thuật toán được thiết kế với cơ chế phân tầng 3 cấp độ chặt chẽ:

- **Tầng 1 - Bảo toàn cấu trúc (Split Preservation):** Dữ liệu sinh ra được tự động phân bổ chính xác vào các tập huấn luyện (`train`), kiểm định (`val`), và kiểm thử (`test`) tương ứng với ảnh gốc, đi kèm với việc đồng bộ chính xác file nhãn YOLO.
- **Tầng 2 - Cân bằng Môi trường (Environmental Stratification):** Các hiệu ứng thời tiết được phân bổ đều trên cả 4 kịch bản thảm họa của bộ dữ liệu C2A gốc bao gồm: hỏa hoạn (`fire`), lũ lụt (`flood`), sập công trình (`collapsed_building`), và tai nạn giao thông (`traffic_incident`) [25].
- **Tầng 3 - Tỷ lệ Hiệu ứng và Cường độ (Effect & Intensity Caps):**
  - Dữ liệu tăng cường được chia tỷ lệ đồng đều: **50% hiệu ứng Sương mù** và **50% hiệu ứng Mưa**.

– Mỗi hiệu ứng được phân tách thành 3 mức độ khó (severity) nhằm kiểm tra giới hạn hội tụ của AI:

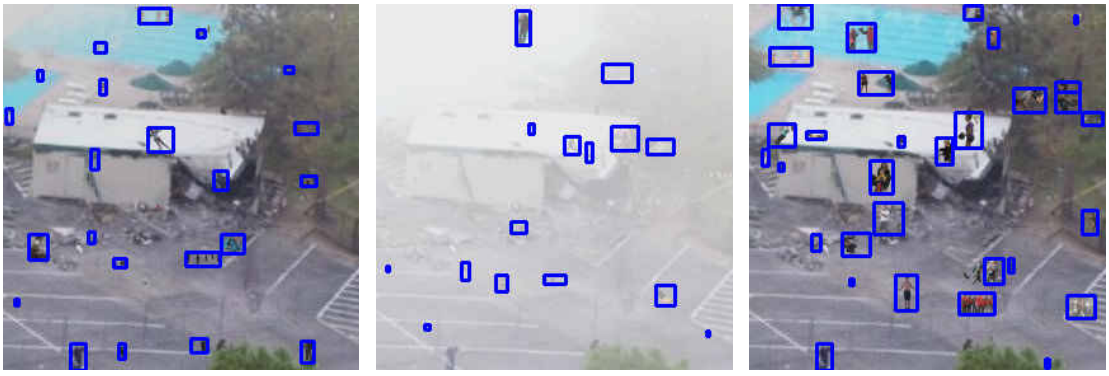
  * **Light (70%):** Mức độ nhẹ, tầm nhìn xa từ 1000m – 3000m.
  * **Medium (20%):** Mức độ vừa, tầm nhìn trung bình từ 400m – 1000m.
  * **Hard (10%):** Mức độ cực đoan, giới hạn tầm nhìn ở 150m – 400m. Đặc biệt, ngưỡng 150m được thiết lập làm giới hạn chặn (cap) nhằm đảm bảo mục tiêu không bị che khuất hoàn toàn. Điều này giúp mạng nơ-ron vẫn có khả năng trích xuất được tín hiệu hình dáng thay vì học nhầm nhiễu nền [23, 7]. Việc thiết lập ngưỡng chặn này là cực kỳ quan trọng để ngăn chặn hiện tượng **Diminishing Returns** (Hiệu suất giảm dần - ép mô hình học quá nhiều ảnh mù mịt sẽ làm độ chính xác đi lùi thay vì tốt lên).

Kết quả Đạt được

Sau khi chạy thuật toán, bộ dữ liệu đã được mở rộng thành công và phân tách rõ ràng để phục vụ quá trình huấn luyện:

- **Tập gốc (Clean Dataset):** 10.000 ảnh từ bộ C2A (dữ liệu trong trẻo, không nhiễu) [25].
- **Tập tăng cường (Augmented_Only_10k):** 10.000 ảnh được bổ sung các mô phỏng thời tiết ở đa dạng cường độ.

Việc sử dụng tập `Mixed_Dataset_20k` kết hợp với phương pháp mô phỏng vật lý đa luồng giúp AI học được các đặc trưng cốt lõi. Nhờ đó, mô hình duy trì độ chính xác cao khi quan sát thời tiết đẹp, đồng thời gia tăng mạnh mẽ tính **Robustness** (Độ bền vững - khả năng chống chịu và hoạt động ổn định) trong môi trường cứu hộ thực tế có tầm nhìn hạn chế [23].



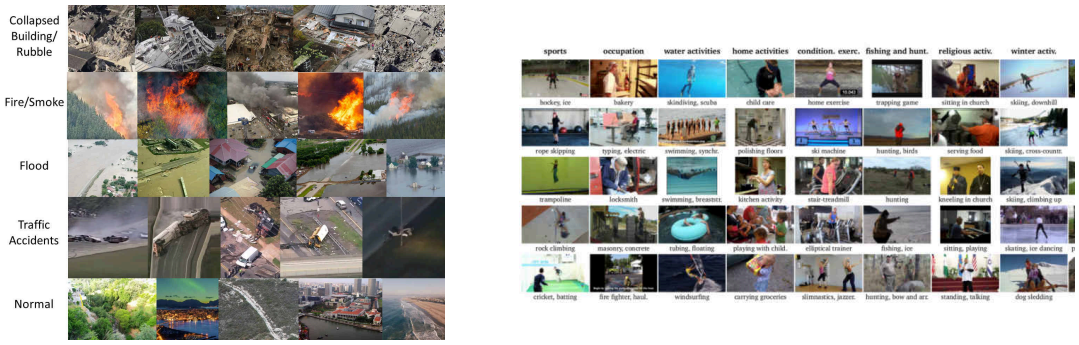


Hình 4.17: *Hình minh họa*

4.5 Nghiên cứu khả năng tổng quát hóa của mô hình trong bài toán SAR

4.5.1 Những hạn chế của C2A

Ban đầu, nhóm lựa chọn bộ dữ liệu C2A (Combination to Application)-một bộ dữ liệu được thiết kế chuyên biệt để mô phỏng hình ảnh cứu hộ bằng cách ghép nối (overlay) các tư thế đa dạng của con người từ bộ dữ liệu LSP/MPII-MPHB. trên các bối cảnh thảm họa thực tế (lũ lụt, hỏa hoạn, động đất) từ bộ dữ liệu AIDER



Hình 4.18: *Hình ảnh bộ dữ liệu AIDER và bộ dữ liệu LSP/MPII-MPHB*



Hình 4.19: *Hình ảnh các thảm họa về hỏa hoạn, ngập lụt, giao thông, công trình*

Hệ quả là mô hình chỉ detect tốt trên các bức ảnh có đối tượng được cắt ghép theo đúng phong cách của C2A. Khi đối mặt với người thực tế, nó dễ dàng bị đánh lừa bởi nhiễu nền, bỏ sót hoàn toàn người thật. Ngược lại, sau khi được tổng quát hóa bằng dữ liệu hỗn hợp, mô hình có phần khắc phục được các bias này, tập trung đúng vào đặc trưng của con người.



(a) *Baseline C2A: phát hiện tốt trên ảnh tổng hợp nhưng dự đoán sai hoặc bỏ sót trên người thực.*



(b) *C2A Generalized: nhận diện đúng người thật và bỏ qua nhiễu nền.*

Hình 4.20: *So sánh khả năng tổng quát hóa của hai mô hình.*

Vấn đề đặt ra là: Làm sao để mô hình học được đặc trưng "người thật"thay vì chỉ học các "vết nhiễu/vết cắt ghép"của ảnh tổng hợp?

4.5.2 Đề xuất giải pháp

Để giải quyết triệt để sự thiếu hụt trong việc học các đặc trưng thực tế, nhóm đặt ra các mục tiêu cụ thể như sau:

- **Đề xuất chiến lược dữ liệu hỗn hợp (*Mixed-Dataset*).** Xây dựng và huấn luyện mô hình trên một tập dữ liệu kết hợp có kiểm soát giữa C2A và các bộ dữ liệu chụp người thực tế và từ nhiều góc nhìn(như CrowdHuman, VisDrone, TinyPerson). Mục tiêu của chiến lược này là chuyển hướng mô hình từ việc học các đặc tính của dữ liệu cắt ghép sang việc làm giàu bối cảnh thực tế và học các đặc trưng trong môi trường tự nhiên.
- **Tối ưu hóa khả năng chống chịu môi trường.** Đánh giá tác động của phương pháp tăng cường dữ liệu thời tiết nhân tạo (*Weather Augmentation*). Qua đó, nghiên cứu tìm kiếm thiết lập tối ưu giúp mô hình vừa tăng độ bền vững (*robustness*) trước các nhiễu loạn hình ảnh như sương mù hoặc mưa, vừa không làm suy giảm các đặc trưng gốc của vật thể siêu nhỏ.

- **Xây dựng giao thức kiểm chứng thực (*Cross-dataset Evaluation Protocol*).** Thiết lập một quy trình đánh giá chéo trên các tập dữ liệu hoàn toàn chưa xuất hiện trong quá trình huấn luyện (*unseen datasets*) như SARD và Heridal. Mục tiêu là đánh giá chính xác khả năng tổng quát hóa (*generalization*) của mô hình trong các bối cảnh thực tế khác nhau.
- **Xác định mô hình khả thi cho ứng dụng SAR.** Nghiên cứu không phụ thuộc hoàn toàn vào độ đo AP50 truyền thống mà tập trung vào việc tìm ra mô hình duy trì được độ nhạy (*Recall*) cao nhất, đồng thời kiểm soát tỷ lệ báo động giả (*False Positive*) ở mức chấp nhận được trên các tập dữ liệu thực tế. Điều này đặc biệt quan trọng trong các kịch bản *Search and Rescue (SAR)*, nơi việc bỏ sót mục tiêu có thể ảnh hưởng trực tiếp đến an toàn sinh mạng khi triển khai thực tế.

4.6 Phương Pháp Tiếp Cận và Thiết Lập Đánh Giá

4.6.1 Trộn Dữ Liệu (Mixed-Dataset Strategy)

Để khắc phục sự chênh lệch về mặt đặc trưng (*domain gap*) giữa ảnh tổng hợp và ảnh thực tế, mô hình cần được huấn luyện trên một tập dữ liệu hỗn hợp (*mixed-dataset*) có sự kiểm soát chặt chẽ. Chiến lược mới chuyển hướng sang việc "làm giàu bối cảnh thật và đặc trưng con người thực tế".

Quá trình trộn dữ liệu được hiện thực hóa bằng cách kết hợp bộ dữ liệu gốc C2A với các bộ dữ liệu chụp người thực tế từ góc nhìn UAV (bao gồm *CrowdHuman*, *VisDrone*, và *TinyPerson*).

4.6.2 Đặc Tả Ma Trận Mô Hình (Model Matrix Specification)

Để đảm bảo tính công bằng trong quá trình so sánh, tất cả các mô hình trong thực nghiệm đều sử dụng chung một kiến trúc mạng YOLO, đồng thời được thiết lập cùng một cấu hình hậu xử lý (bao gồm *Confidence Threshold*, *NMS threshold*, và *Image Size*).

Dưới đây là đặc tả chi tiết bốn phiên bản mô hình được xây dựng nhằm kiểm chứng các luận điểm nghiên cứu:

- **Mô hình M1 (Baseline In-domain)**
  - *Dữ liệu huấn luyện:* C2A
  - *Phương pháp augmentation:* Chuẩn của YOLO
  - *Mục đích:* Đóng vai trò là mốc tham chiếu nội bộ, kiểm tra xem mô hình có học tốt bài toán trên dữ liệu tổng hợp ban đầu hay không.
- **Mô hình M2 (Robustness Baseline)**
  - *Dữ liệu huấn luyện:* C2A
  - *Phương pháp augmentation:* C2A + Weather Augmentation (mô phỏng nhiễu thời tiết nhân tạo với tỉ lệ **7** gốc : **2** vừa : **1** nặng)

  - *Mục đích:* Đánh giá tác động độc lập của việc chỉ thêm nhiễu thời tiết đối với khả năng chống chịu của mô hình.

- **Mô hình M3 (Mixed Dataset Base)**
  - *Dữ liệu huấn luyện:* C2A + CrowdHuman + TinyPerson + VisDrone
  - *Phương pháp augmentation:* Chuẩn của YOLO
  - *Mục đích:* Kiểm chứng khả năng giải quyết domain gap bằng cách bổ sung đặc trưng người từ dữ liệu thực tế.

- **Mô hình M4 (Mixed Dataset + Weather Augmentation)**
  - *Dữ liệu huấn luyện:* C2A + CrowdHuman + TinyPerson + VisDrone
  - *Phương pháp augmentation:* Chuẩn YOLO + Weather Augmentation (cũng được tinh chỉnh với tỷ lệ **7** gốc : **2** vừa : **1** nặng)
  - *Mục đích:* Tìm kiếm sự kết hợp tối ưu giữa việc học từ dữ liệu thực và nâng cao khả năng chống chịu trước điều kiện thời tiết xấu.

4.6.3 Đặc tả các Tập Dữ Liệu Sử Dụng để train (Dataset Profiles)

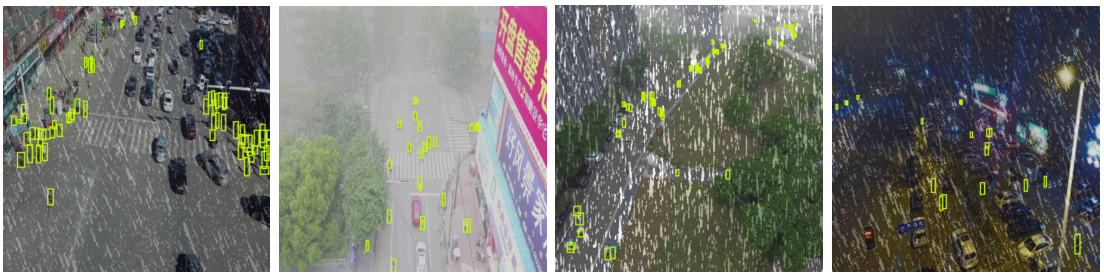
Tập Dữ Liệu Tổng Hợp (Synthetic Training Domain) Đây là bộ dữ liệu được thiết kế chuyên biệt để mô phỏng hình ảnh tìm kiếm cứu nạn trong thảm họa. Các bức ảnh trong C2A [25] được tạo ra bằng phương pháp tổng hợp: cắt nền các tư thế con người đa dạng từ tập LSP/MPII-MPHB, sau đó ghép nối (overlay) lên các bối cảnh thảm họa thực tế như lũ lụt, hỏa hoạn, động đất từ tập AIDER.

Tập Dữ Liệu Bổ Sung Đặc Trưng Thực Tế Để cải thiện khả năng *khái quát hóa* (generalization) của mô hình trên dữ liệu thực tế, bên cạnh dữ liệu tổng hợp C2A, nhóm bổ sung ba bộ dữ liệu thực tế đại diện cho các bối cảnh khác nhau gồm: **VisDrone**, **TinyPerson** và **CrowdHuman**. Ba bộ dữ liệu này lần lượt đại diện cho các tình huống: ảnh UAV từ trên cao, đối tượng người kích thước rất nhỏ ở khoảng cách xa, và cảnh đông người với mức độ che khuất (occlusion) cao.

- **VisDrone:** Bộ dữ liệu VisDrone-DET2019 [8] là một bộ chuẩn đánh giá lớn cho bài toán *object detection* từ UAV. Bộ dữ liệu này bao gồm **10.209 ảnh tĩnh**, được thu thập tại 14 thành phố ở Trung Quốc trong nhiều điều kiện khác nhau như khu vực đô thị hoặc nông thôn, độ cao bay đa dạng và thời tiết thay đổi. Tổng cộng, tập dữ liệu có hơn **2,6 triệu nhãn bounding box** cho **10 lớp đối tượng**, gồm: pedestrian, person, car, van, bus, truck, motor, bicycle, awning-tricycle và tricycle. Sau khi lọc riêng các lớp liên quan đến con người, nhóm thu được **7.478 ảnh** và tiếp tục tăng cường dữ liệu theo tỉ lệ **7:2:1**. Link dataset



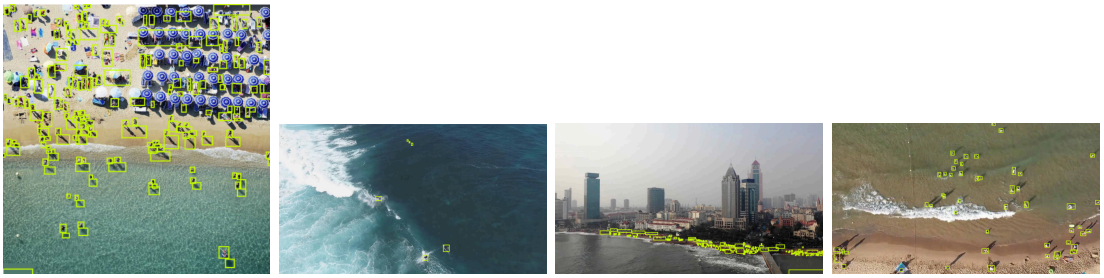
Hình 4.21: *Hình ảnh Dataset VisDrone chưa qua augment*



Hình 4.22: *Hình ảnh Dataset VisDrone đã qua augment*

- **TinyPerson:** TinyPerson [26] là bộ dữ liệu được thiết kế cho bài toán *tiny person detection*, tập trung vào việc phát hiện người có kích thước rất nhỏ trong ảnh. Dataset gồm **1.610 ảnh** với **72.651 annotations**, trong đó phần lớn các đối tượng người có kích thước **nhỏ hơn 20 pixel**.

  Các hình ảnh được thu thập chủ yếu từ video độ phân giải cao trên Internet với bối cảnh biển hoặc bờ biển. Dataset gồm hai lớp chính: *sea person* (người trên biển hoặc trên thuyền) và *earth person* (người trên đất liền). Nhóm đã gộp class lại chung thành 1 class người và augment thời tiết. Nhóm sử dụng phiên bản Roboflow đã được xử lý: Link dataset



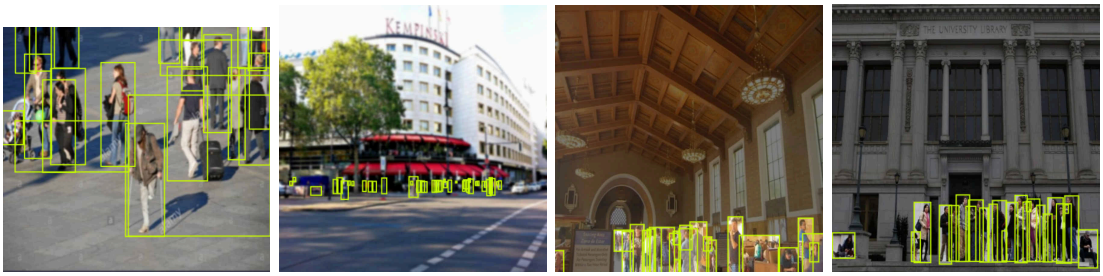
Hình 4.23: *Hình ảnh Dataset TinyPerson chưa qua augment*



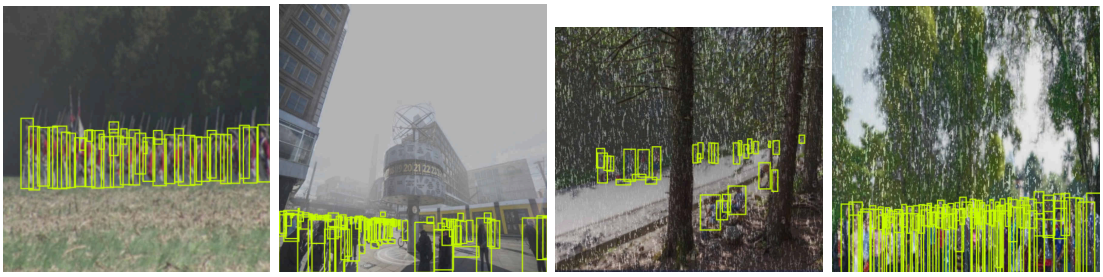
Hình 4.24: *Hình ảnh Dataset TinyPerson đã qua augment*

- **CrowdHuman**: Bộ dataset CrowdHuman [9] là một benchmark phổ biến cho bài toán *human detection trong môi trường đông đúc*. Dataset bao gồm **15.000 ảnh train**, **4.370 ảnh validation** và **5.000 ảnh test**, với khoảng **470.000 instance người** trong tập train và validation. Trung bình mỗi ảnh chứa khoảng **23 người**, với nhiều trường hợp *occlusion* nặng hoặc che khuất một phần.

  Nhóm đã bỏ bớt và chỉ sử dụng gần 5000 ảnh cho việc trộn dataset và cũng augment trên tập này với tỉ lệ (**7 : 2 : 1**). Link dataset



Hình 4.25: *Hình ảnh Dataset CrowdHuman chưa qua augment*



Hình 4.26: *Hình ảnh Dataset CrowdHuman đã qua augment*

Vậy việc bổ sung ba tập **VisDrone-Test**, **TinyPerson-Test** và **CrowdHuman-Test** cho phép đánh giá toàn diện khả năng *generalization* của mô hình trên nhiều bối cảnh thực tế khác nhau, đồng thời bổ trợ cho dữ liệu tổng hợp C2A trong các thí nghiệm so sánh hiệu suất.

Tập Dữ Liệu Kiểm Chứng Chéo Chưa Từng Thấy (Unseen SAR Domains)

Ưu tiên cao nhất trong quá trình đánh giá là đảm bảo không có bất kỳ hình ảnh nào từ các tập dữ liệu sau được sử dụng trong quá trình huấn luyện hoặc tinh chỉnh mô hình.

- **SARD:** Bộ dữ liệu SARD [20], là bộ dữ liệu **Search And Rescue Dataset** chuyên biệt cho nhiệm vụ tìm kiếm cứu nạn (SAR) bằng drone. Dataset bao gồm khoảng **5.600 ảnh RGB** với chỉ **1 lớp (person/human)**. Các hình ảnh được thu thập từ drone DJI Phantom 4 Advanced ở Croatia, tập trung vào các tư thế người trong tình huống thương vong/thương tích (nằm, ngồi gục, nằm sấp, nằm ngửa, đứng, đi bộ, chạy...). Đây là tập dữ liệu *unseen* hoàn toàn, giúp đánh giá khả năng tổng quát hóa của mô hình trong môi trường cứu hộ thực tế với nhiều

tư thế khó và điều kiện ngoài trời. Drone SAR RGB với các tư thế người tương đồng với tình huống thương vong, chẳng hạn như nằm hoặc ngồi gục. Nhóm sử dụng phiên bản Roboflow đã chuẩn bị sẵn cho YOLO: Link dataset



Hình 4.27: *Hình ảnh Dataset SARD*

- **Heridal:** Heridal [21] là bộ dữ liệu chuyên biệt được thiết kế cho nhiệm vụ tìm kiếm cứu nạn (Search and Rescue - SAR) bằng ảnh UAV trên địa hình tự nhiên. Tên đầy đủ là **Human Rescue Imaging Database on Land (HERIDAL)**. Dataset bao gồm khoảng **1.600 ảnh** với chỉ **1 lớp (person)** và hàng nghìn annotation người. Các hình ảnh được thu thập từ góc nhìn trên cao trong môi trường rừng núi, công viên và cảnh quan điển hình Trung Âu (cây cối, đá, bụi rậm, địa hình gồ ghề).

  Đặc điểm nổi bật của Heridal là **nhiễu nền cực mạnh** (cây cỏ, đá, bóng cây, địa hình phức tạp) — đây chính là thách thức lớn nhất gây ra False Positive khi phát hiện người. Bộ dữ liệu này chưa từng xuất hiện trong quá trình huấn luyện của bất kỳ mô hình nào, do đó rất phù hợp để kiểm tra khả năng *generalization* và độ tin cậy thực tế của mô hình trong môi trường cứu hộ rừng núi.



Hình 4.28: *Hình ảnh Dataset Heridal*

Các hình ảnh minh họa trong bộ dữ liệu Heridal cho thấy mức độ khó rất cao của bài toán phát hiện người trong môi trường tự nhiên. Đối tượng người thường xuất hiện với kích thước nhỏ, màu sắc gần với nền và bị che khuất một phần bởi cỏ, đá hoặc bóng cây. Bên cạnh đó, nền ảnh có nhiều chi tiết phức tạp như thảm thực vật, địa hình gồ ghề và các vùng tương phản không đồng đều, khiến mô hình dễ nhầm lẫn giữa người và các cấu trúc nền. Chính vì vậy, Heridal là tập dữ liệu phù hợp để kiểm tra khả năng khái quát hóa của mô hình trong các tình huống SAR thực tế, nơi mục tiêu không nổi bật rõ ràng mà thường hòa lẫn vào môi trường xung quanh.

Một điểm đáng chú ý khác là các ảnh Heridal thường được chụp ở độ cao tương đối lớn, khiến tỷ lệ giữa đối tượng người và toàn khung hình rất nhỏ. Điều này làm cho bài toán không chỉ dừng ở phát hiện người trong nền phức tạp mà còn là phát hiện *tiny object* trong bối cảnh ngoài trời. Vì vậy, kết quả trên Heridal phản ánh khá rõ năng lực của mô hình trong các tình huống thực tế, đặc biệt là khi người cần tìm kiếm chỉ xuất hiện như những vùng tín hiệu rất nhỏ giữa không gian rừng núi rộng lớn.

4.7 Đánh giá khả năng tổng quát hóa và tăng cường thời tiết

Bảng 4.3: *Kết quả đánh giá chi tiết các mô hình trên các tập dữ liệu*

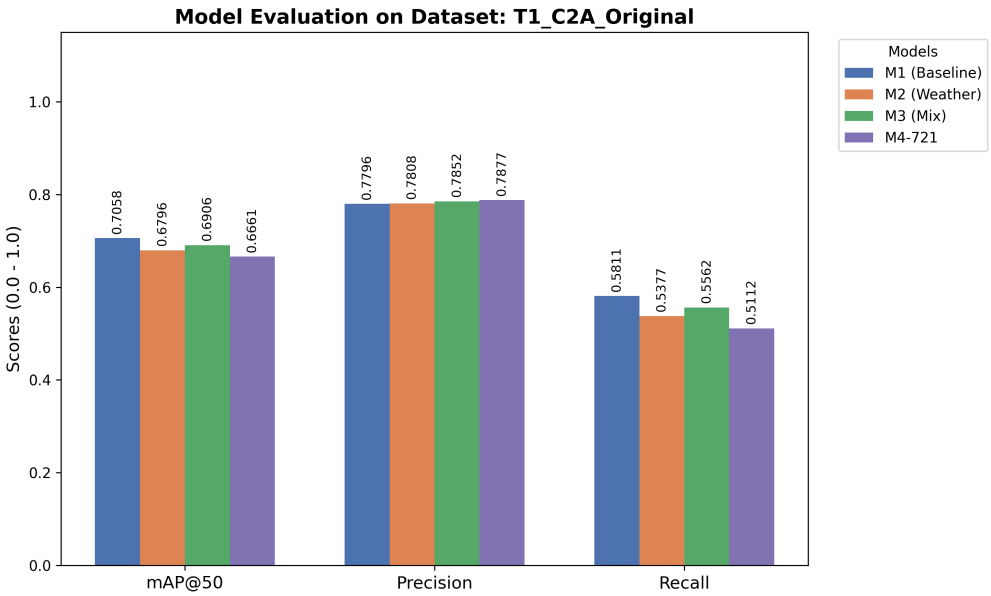
Dataset	Model	mAP@50	mAP@50-95	AP75	Precision	Recall	F1-Score
T1_C2A_Original	M1	0.7058	0.4414	0.4654	0.7796	0.5811	0.6659
	M2	0.6796	0.4140	0.4313	0.7808	0.5377	0.6368
	M3	0.6906	0.4256	0.4461	0.7852	0.5562	0.6511
	M4	0.6661	0.4041	0.4222	0.7877	0.5112	0.6200
T1_C2A_Weather	M1	0.5763	0.3286	0.3267	0.6841	0.4257	0.5248
	M2	0.6677	0.3987	0.4113	0.7597	0.5277	0.6228
	M3	0.6181	0.3674	0.3765	0.7529	0.4470	0.5609
	M4	0.6631	0.3959	0.4099	0.7683	0.5152	0.6168
T2_VisDrone	M1	0.1485	0.0648	0.0491	0.2608	0.0422	0.0726
	M2	0.1697	0.0709	0.0492	0.2910	0.0594	0.0986
	M3	0.4112	0.1869	0.1433	0.6816	0.1322	0.2215
	M4	0.4078	0.1808	0.1334	0.6568	0.1498	0.2440
T3_Heridal	M1	0.2579	0.1152	0.0877	0.3091	0.2774	0.2924
	M2	0.2481	0.1021	0.0684	0.3420	0.2336	0.2776
	M3	0.3305	0.1455	0.1028	0.4113	0.3441	0.3747
	M4	0.3723	0.1717	0.1398	0.5554	0.2276	0.3229
T3_SARD	M1	0.1573	0.0603	0.0362	0.3010	0.1351	0.1865
	M2	0.1419	0.0711	0.0624	0.3289	0.1134	0.1687
	M3	0.4358	0.2445	0.2488	0.6032	0.2961	0.3972
	M4	0.4852	0.2878	0.3087	0.6673	0.2790	0.3934

4.7.1 Mối liên hệ giữa các Model và các Tập Dataset

- **M1 (Baseline):** Mô hình cơ sở, chỉ học trên tập dữ liệu ảnh ghép C2A gốc.
- **M2 (Weather):** Phát triển từ M1, bổ sung nhiễu thời tiết nhân tạo vào tập dữ liệu C2A.
- **M3 (Mix):** Thay đổi chiến lược huấn luyện bằng cách trộn dữ liệu C2A gốc với dữ liệu người thật (VisDrone, CrowdHuman, ...), nhưng **không** thêm nhiễu thời tiết.
- **M4-721:** Kết hợp cả hai hướng tiếp cận: Trộn dữ liệu thật, Thêm nhiễu thời tiết. Sự khác biệt nằm ở tỷ lệ phân phối dữ liệu: 70% (normal) + 20% (weather medium) + 10% (weather hard)

4.7.2 Đánh giá trên miền dữ liệu gốc

Tập T1_C2A_Original (Dữ liệu gốc, không nhiễu thời tiết)



Hình 4.29: *Hiệu suất các mô hình trên tập dữ liệu T1_C2A_Origina*

- **M1 (Baseline):** Đạt kết quả rất tốt trên tập dữ liệu gốc với mAP@50 = 0.7058 và Recall = 0.5811. Điều này cho thấy mô hình đã học hiệu quả các đặc trưng đặc thù của ảnh cắt ghép (artefacts) trong tập C2A.
- **M2 (Weather):** Hiệu suất giảm nhẹ so với M1 (mAP@50 = 0.6796). Nguyên nhân là do mô hình phải phân bổ năng lực học để xử lý thêm các nhiễu thời tiết nhân tạo, dẫn đến giảm nhẹ độ nhạy trên ảnh gốc.
- **M3 (Mix – Không thời tiết):** Đạt mAP@50 = 0.6906. Việc bổ sung dữ liệu người thật vào quá trình huấn luyện làm giảm nhẹ hiệu suất trên ảnh cắt ghép so với M1, nhưng vẫn duy trì được độ ổn định cao.
- **M4-721 (Mix + 30% Thời tiết):** Đạt kết quả thấp nhất trong nhóm (mAP@50 = 0.6661). Việc học cả đặc trưng người thật (từ tập mix) và nhiễu thời tiết có vẻ làm mô hình bị phân tâm trên tập ảnh gốc hoàn toàn sạch và chỉ chứa ảnh cắt ghép.



(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*

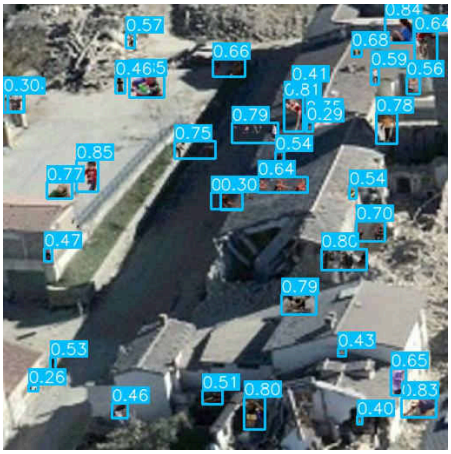


(d) *Model 3*

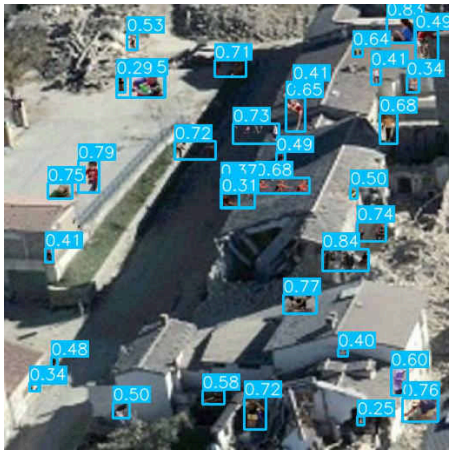


(e) *Model 4*

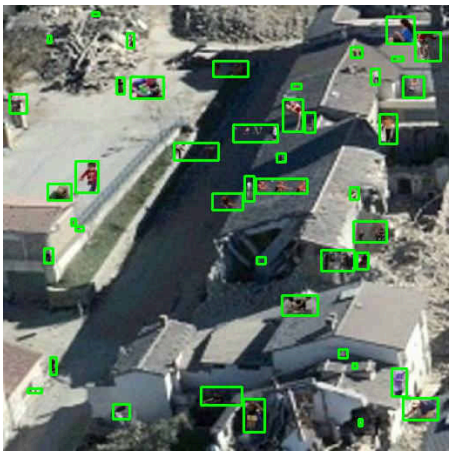
Hình 4.30: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



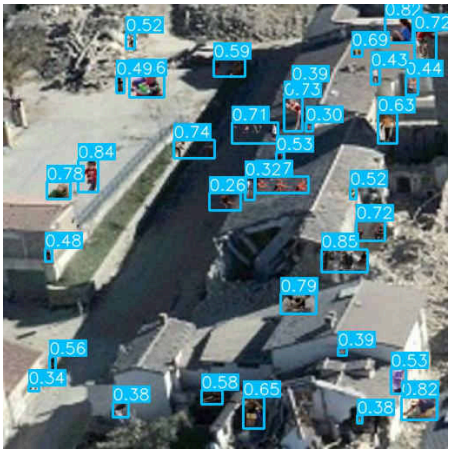
(a) *Model 1*



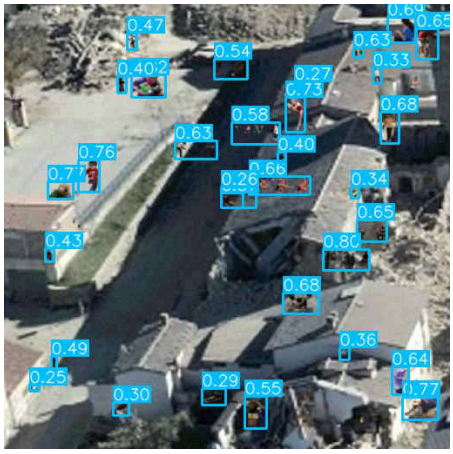
(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*



(e) *Model 4*

Hình 4.31: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*



(e) *Model 4*

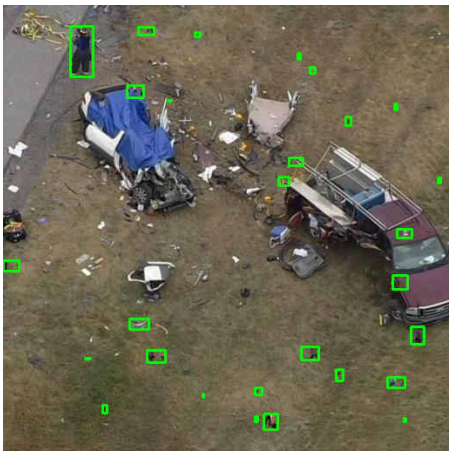
Hình 4.32: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



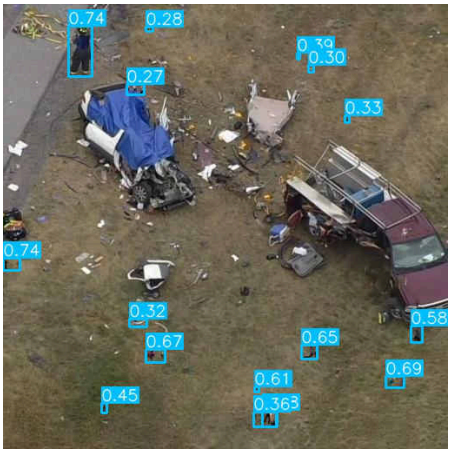
(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*



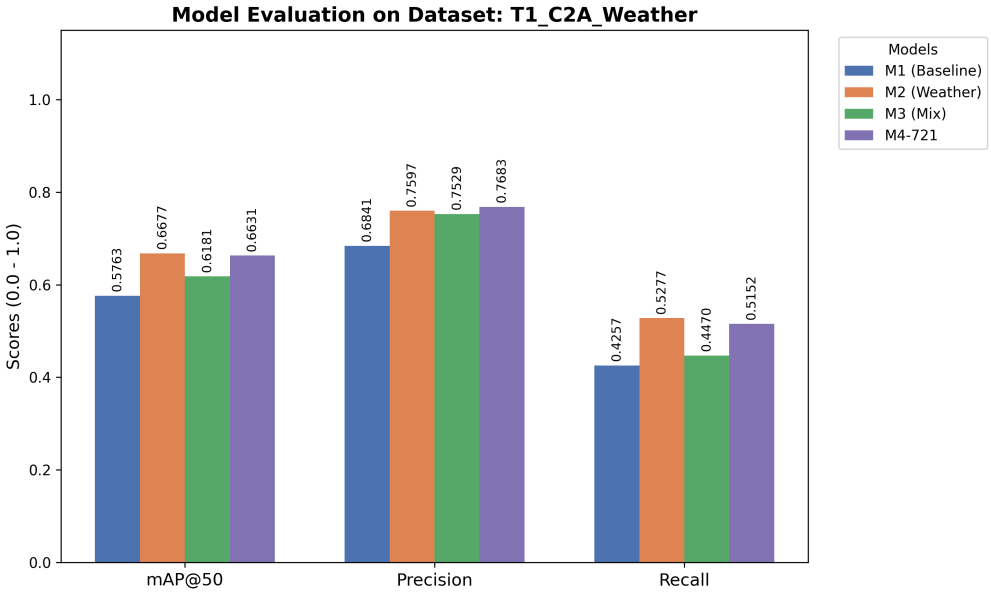
(d) *Model 3*



(e) *Model 4*

Hình 4.33: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*

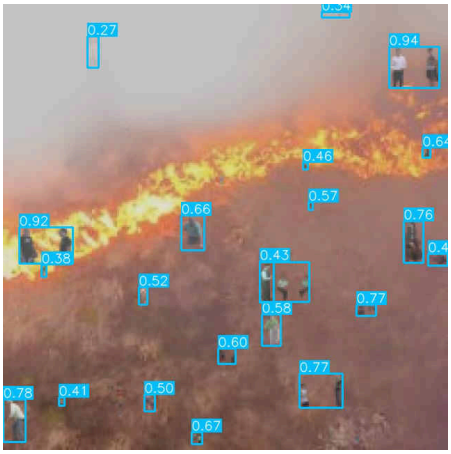
Tập T1_C2A_Weather (Dữ liệu gốc, có nhiễu thời tiết)



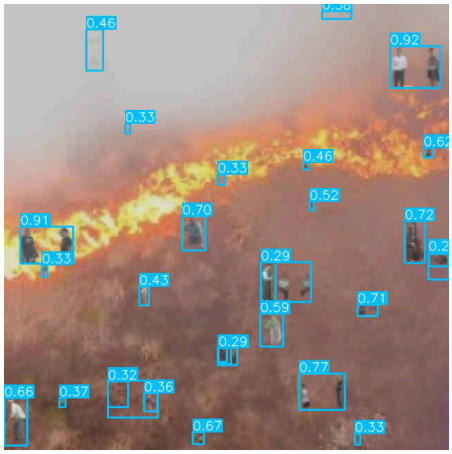
Hình 4.34: *Hiệu suất các mô hình trên tập dữ liệu T1_ C2A_ Weather*

Tập dữ liệu này được sử dụng để đánh giá trực tiếp khả năng detect của các mô hình trong các điều kiện quang học kém (ví dụ: mưa, sương mù).

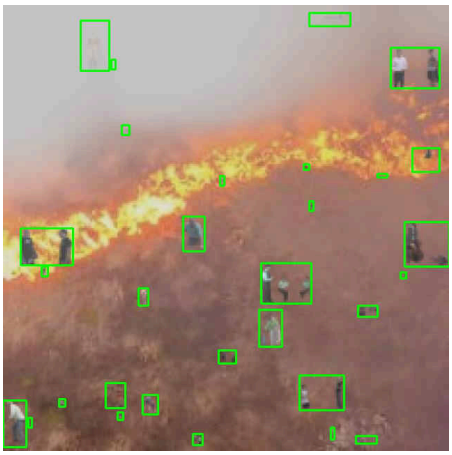
- **M1 (Baseline):** Hiệu suất sụt giảm nghiêm trọng với mAP@50 = 0.5763 và Recall = 0.4257. Do không được huấn luyện với các điều kiện thời tiết bất lợi, mô hình gặp khó khăn trong việc phát hiện đối tượng và dễ bỏ sót.
- **M2 (Weather):** Phục hồi hiệu suất đáng kể với mAP@50 = 0.6677, nhờ được huấn luyện chuyên biệt với các kỹ thuật augmentation mô phỏng thời tiết.
- **M3 (Mix – Không thời tiết):** Đạt mAP@50 = 0.6181. Tương tự M1, việc không được tiếp xúc với nhiễu thời tiết trong quá trình huấn luyện khiến mô hình bị hạn chế khả năng tổng quát trong môi trường này.
- **M4-721 (Mix + 30% Thời tiết):** Bám sát ngay sau M2 với mAP@50 = 0.6181 và Precision cao nhất nhóm đạt 0.7683. Lượng ảnh thời tiết (30%) đã giúp mô hình có khả năng nhìn xuyên nhiễu tốt.



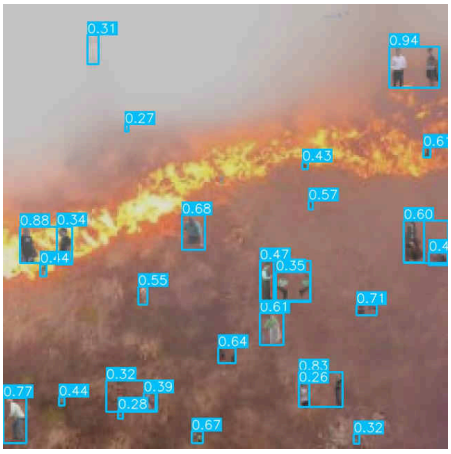
(a) *Model 1*



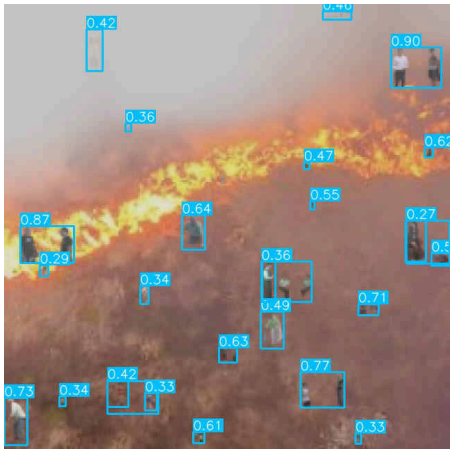
(b) *Model 2*



(c) *Ground Truth*

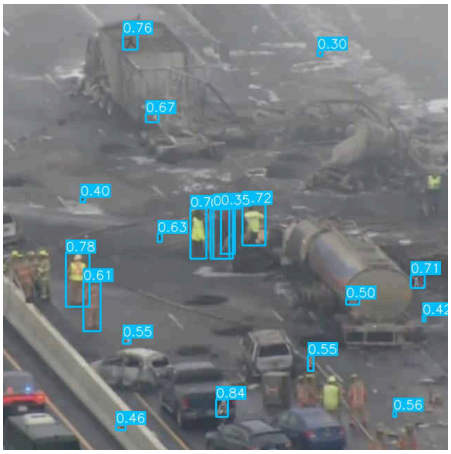


(d) *Model 3*



(e) *Model 4*

Hình 4.35: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



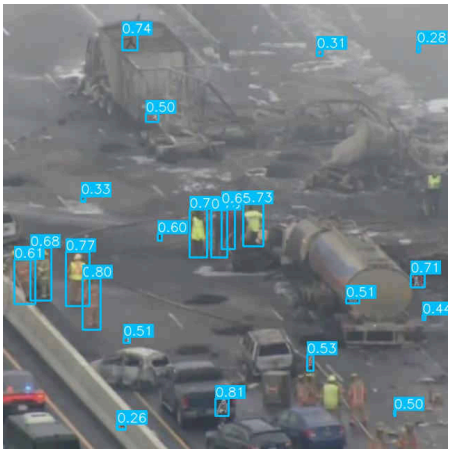
(a) *Model 1*



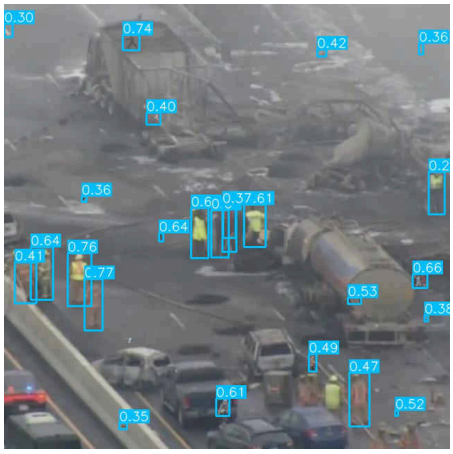
(b) *Model 2*



(c) *Ground Truth*

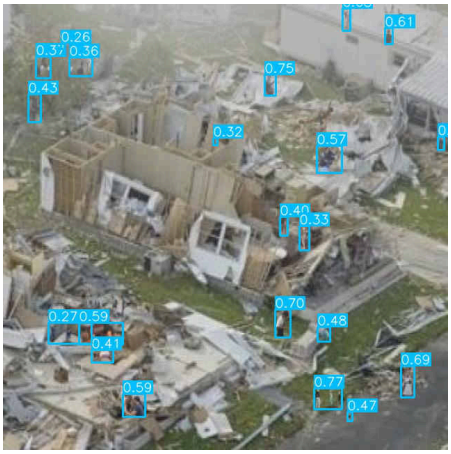


(d) *Model 3*



(e) *Model 4*

Hình 4.36: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



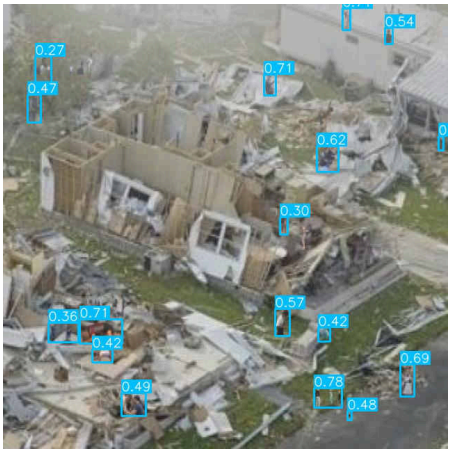
(a) *Model 1*



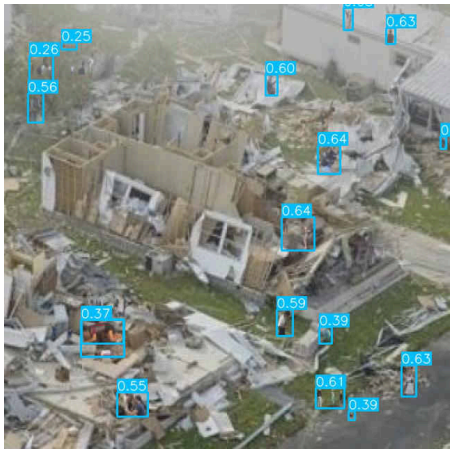
(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*



(e) *Model 4*

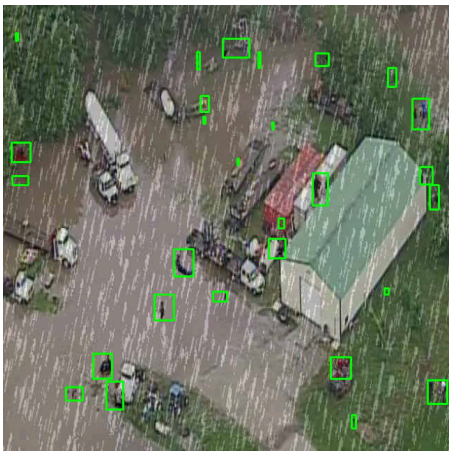
Hình 4.37: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*

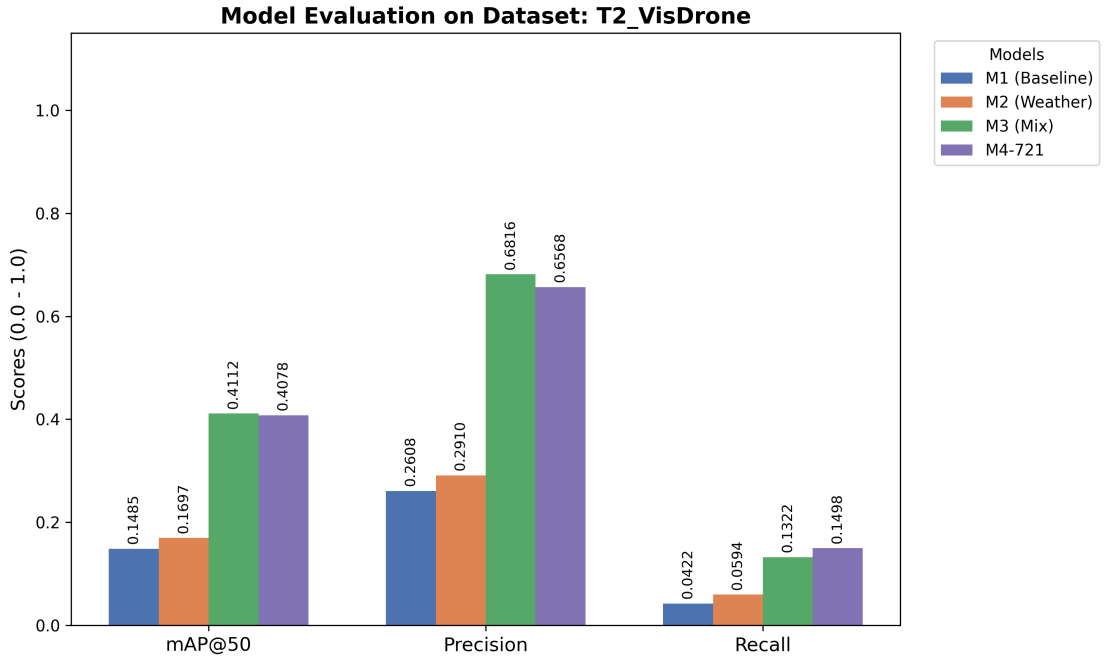


(e) *Model 4*

Hình 4.38: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*

4.7.3 Đánh giá chuyển giao sang dữ liệu thực (Seen Domain & Unseen Domain)

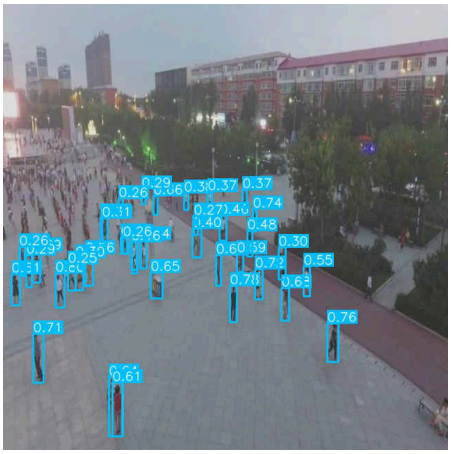
Tập T2_VisDrone (Dữ liệu thực - Seen)



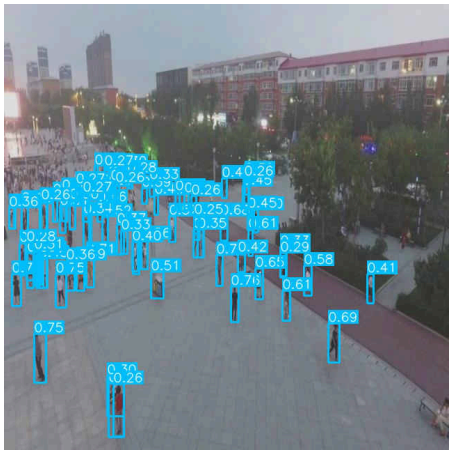
Hình 4.39: *Hiệu suất các mô hình trên tập dữ liệu T2_ VisDrone*

Đây là bước ngoặt quan trọng, phản ánh hạn chế của dữ liệu tổng hợp (ảnh cắt ghép) khi áp dụng vào các bối cảnh thực tế.

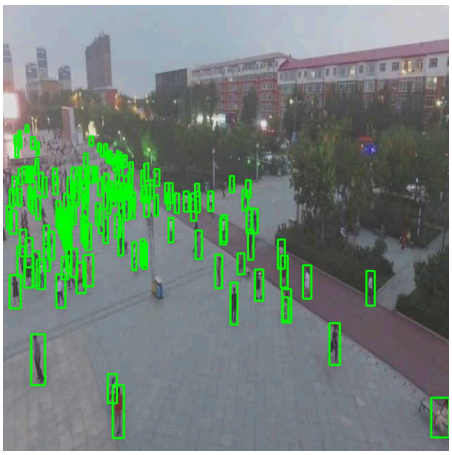
- **M1 (Baseline):** Thấp nhất với mAP@50 = 0.1485 và Recall = 0.0422. Mô hình gần như không phát hiện được người thật, do chỉ học các đặc trưng giả tạo (artefacts) từ dữ liệu cắt ghép.
- **M2 (Weather):** Có kết quả tương tự M1 với mAP@50 = 0.1697. Việc bổ sung nhiễu thời tiết không giúp cải thiện khả năng hiểu bối cảnh thực, đặc biệt là trong góc nhìn từ drone.
- **M3 (Mix – Không thời tiết):** Hiệu suất cải thiện đáng kể, đạt mAP@50 = 0.4112. Nhờ được huấn luyện với dữ liệu người thật (VisDrone, CrowdHuman, ...), mô hình đã học được các đặc trưng thực tế và nhận diện tốt hơn.
- **M4-721 (Mix + 30% Thời tiết):** Đạt mAP@50 = 0.4078, thấp hơn M3. Việc thêm nhiễu thời tiết vào quá trình train không mang lại lợi thế bổ sung rõ rệt nào trên tập ảnh Drone điều kiện bình thường so với M3.



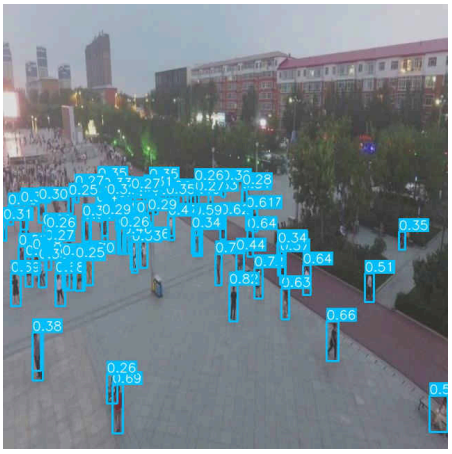
(a) *Model 1*



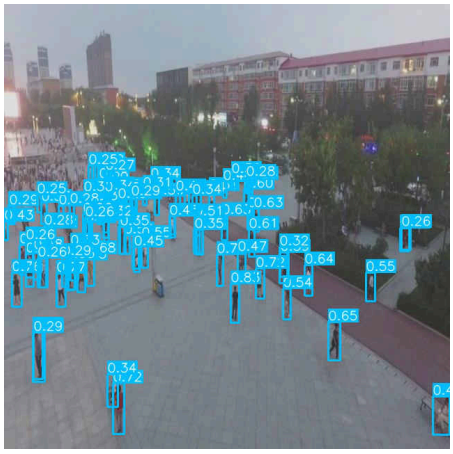
(b) *Model 2*



(c) *Ground Truth*

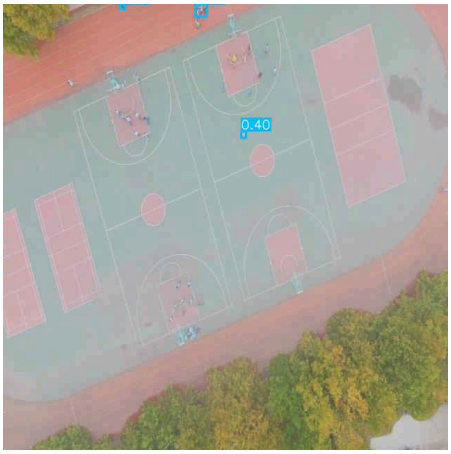


(d) *Model 3*



(e) *Model 4*

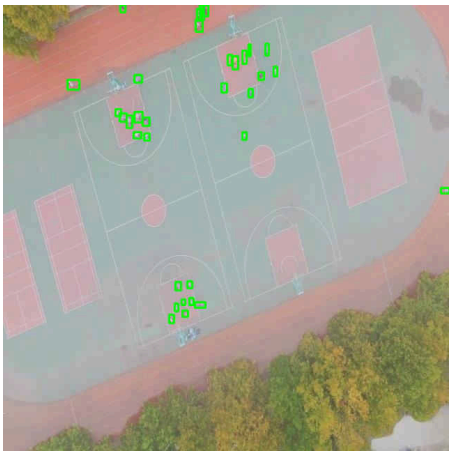
Hình 4.40: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



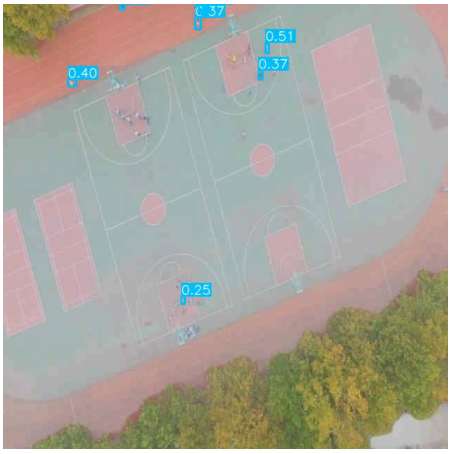
(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*

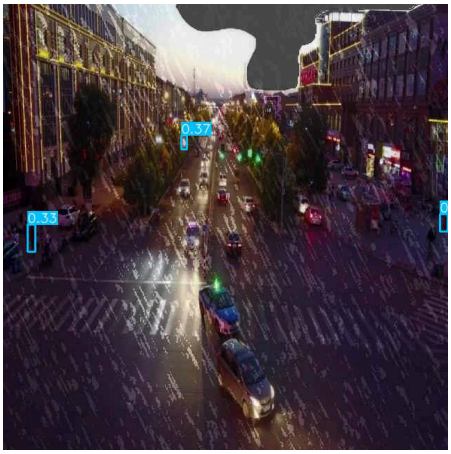


(d) *Model 3*

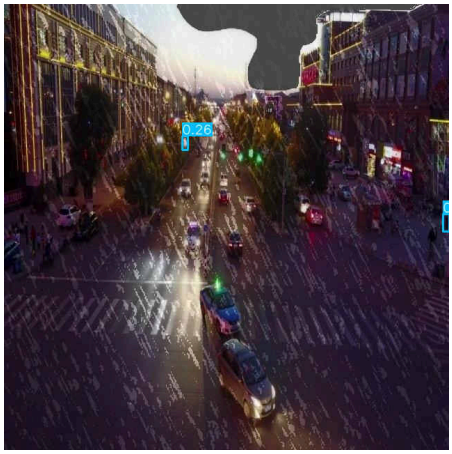


(e) *Model 4*

Hình 4.41: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



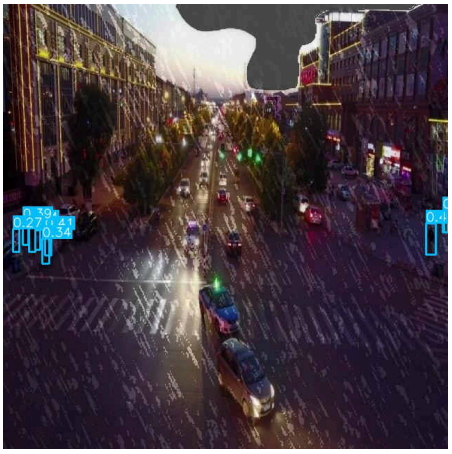
(a) *Model 1*



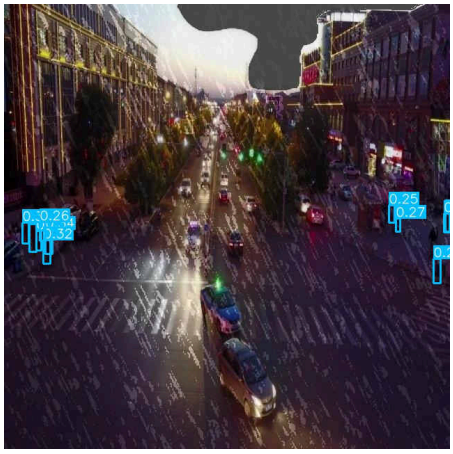
(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*



(e) *Model 4*

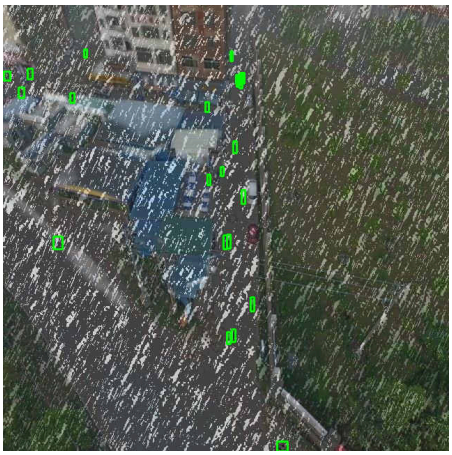
Hình 4.42: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



(a) *Model 1*



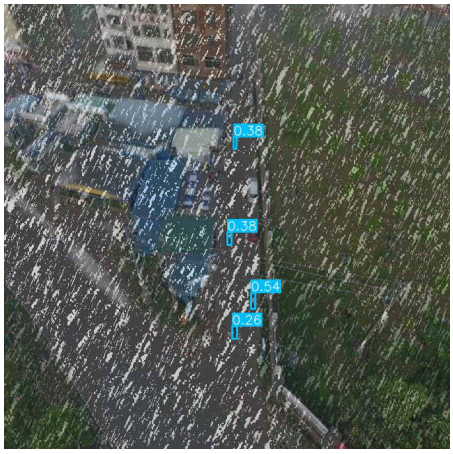
(b) *Model 2*



(c) *Ground Truth*



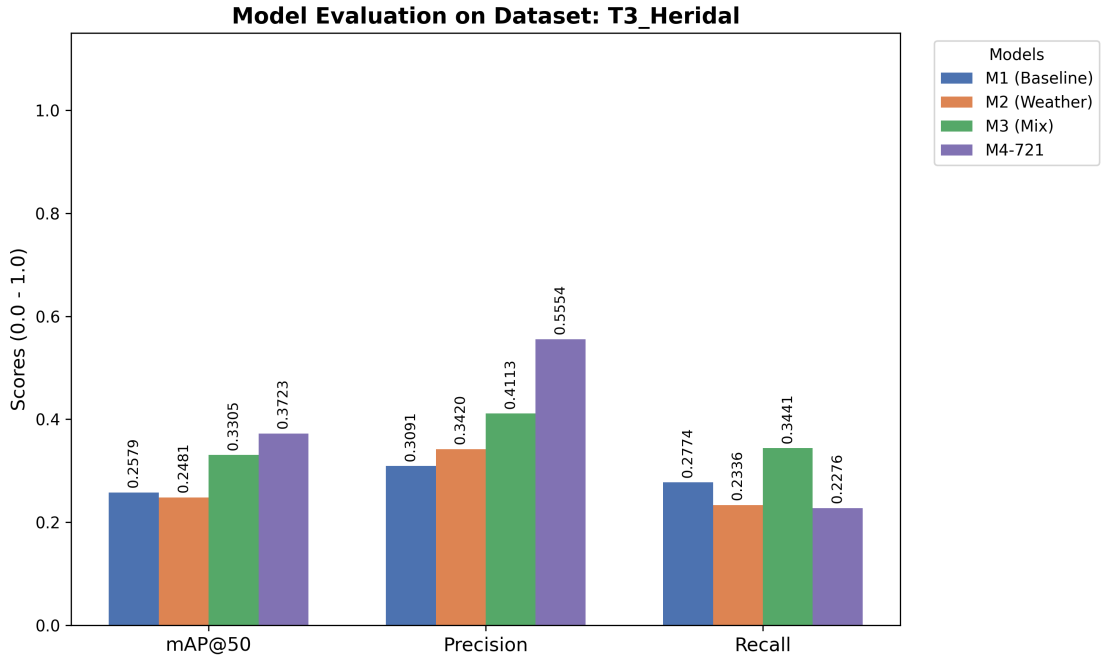
(d) *Model 3*



(e) *Model 4*

Hình 4.43: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*

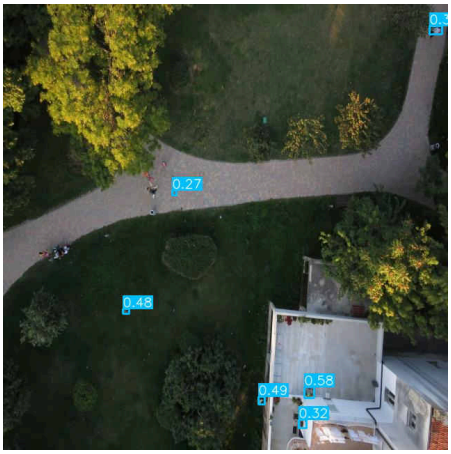
Tập T3_Heridal (Ảnh cứu hộ rừng núi - Unseen)



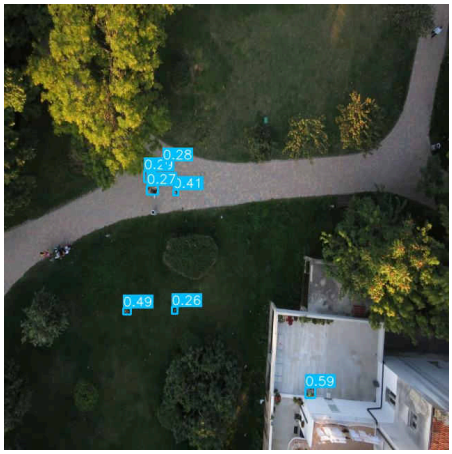
Hình 4.44: *Hiệu suất các mô hình trên tập dữ liệu T3_Heridal*

Tập dữ liệu này đặc biệt khó do nhiễu nền rất lớn (cây cối, đá, vật thể có hình dạng tương tự con người), là nguyên nhân chính gây ra các báo động giả (False Positives).

- **M1 (Baseline):** Đạt Recall = 0.2774 (tương đối cao), nhưng Precision = 0.3091 rất thấp. Điều này cho thấy mô hình có xu hướng dự đoán quá mức, nhầm lẫn nhiều đối tượng nền (ví dụ: gốc cây) thành người.
- **M2 (Weather):** Có hiệu suất thấp nhất với mAP@50 = 0.2481. Việc áp dụng augmentation thời tiết (làm mờ, giảm độ tương phản) trên dữ liệu cắt ghép khiến mô hình trở nên nhạy cảm với nhiễu nền, dẫn đến gia tăng báo động giả.
- **M3 (Mix – Không thời tiết):** Cải thiện đáng kể với mAP@50 = 0.3305 và Recall = 0.3441 (cao nhất trong nhóm). Nhờ học từ dữ liệu người thật, mô hình có khả năng phát hiện đối tượng tốt hơn và giảm hiện tượng bỏ sót.
- **M4-721 (Mix + 30% Thời tiết):** Đạt hiệu suất tốt nhất với mAP@50 = 0.3723. Kết quả này cho thấy mô hình có khả năng phân biệt hiệu quả giữa người và nhiễu nền, từ đó giảm thiểu đáng kể các báo động giả — yếu tố quan trọng trong các ứng dụng cứu hộ thực tế. nhưng Recall = 0.2276 khá thấp.



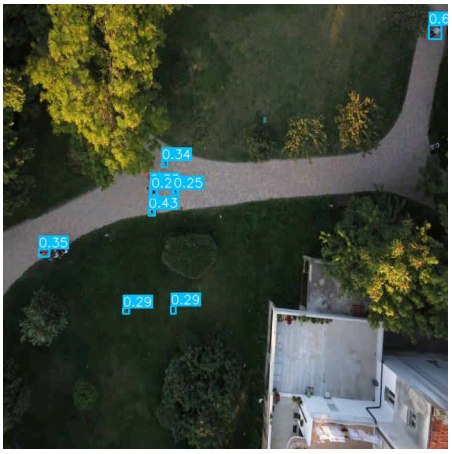
(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*

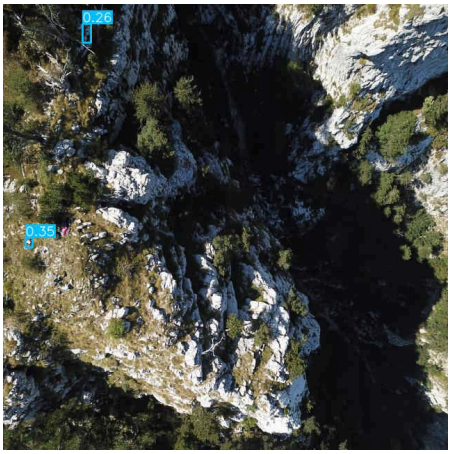


(d) *Model 3*

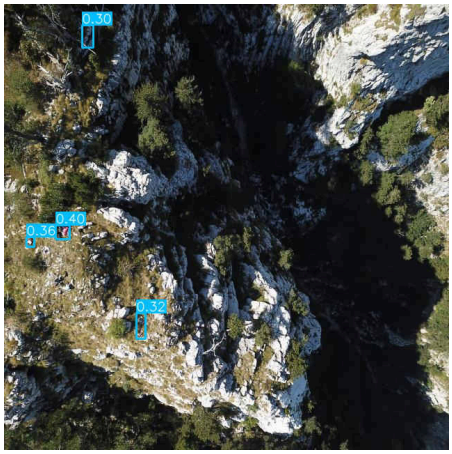


(e) *Model 4*

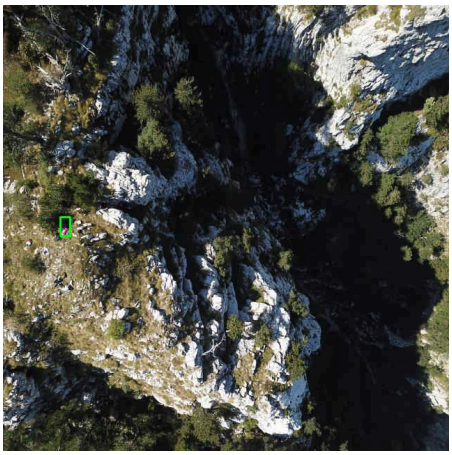
Hình 4.45: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



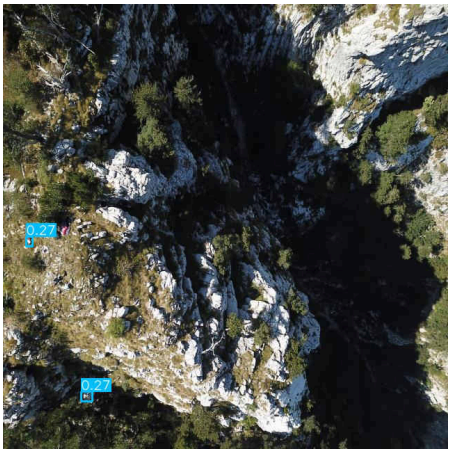
(a) *Model 1*



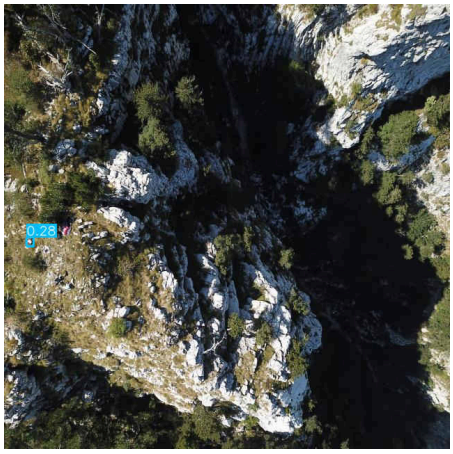
(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*

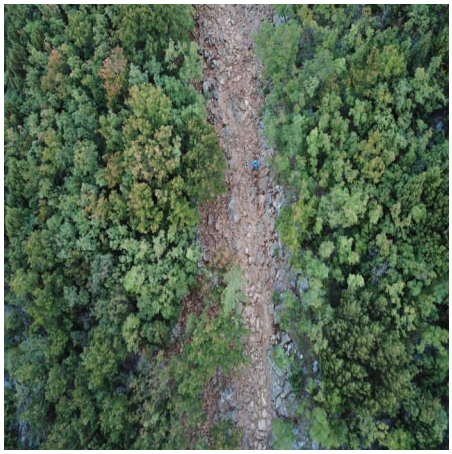


(e) *Model 4*

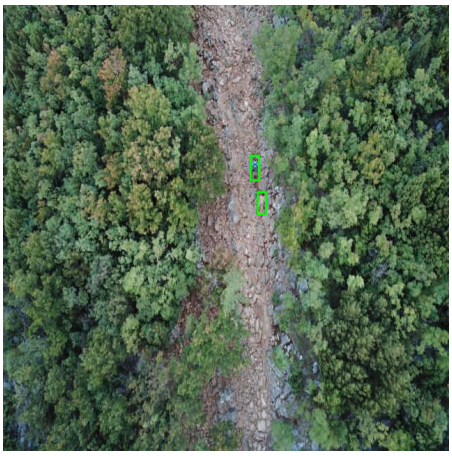
Hình 4.46: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



(a) *Model 1*



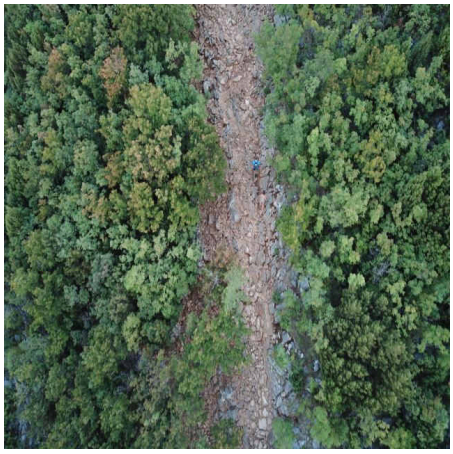
(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*



(e) *Model 4*

Hình 4.47: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*



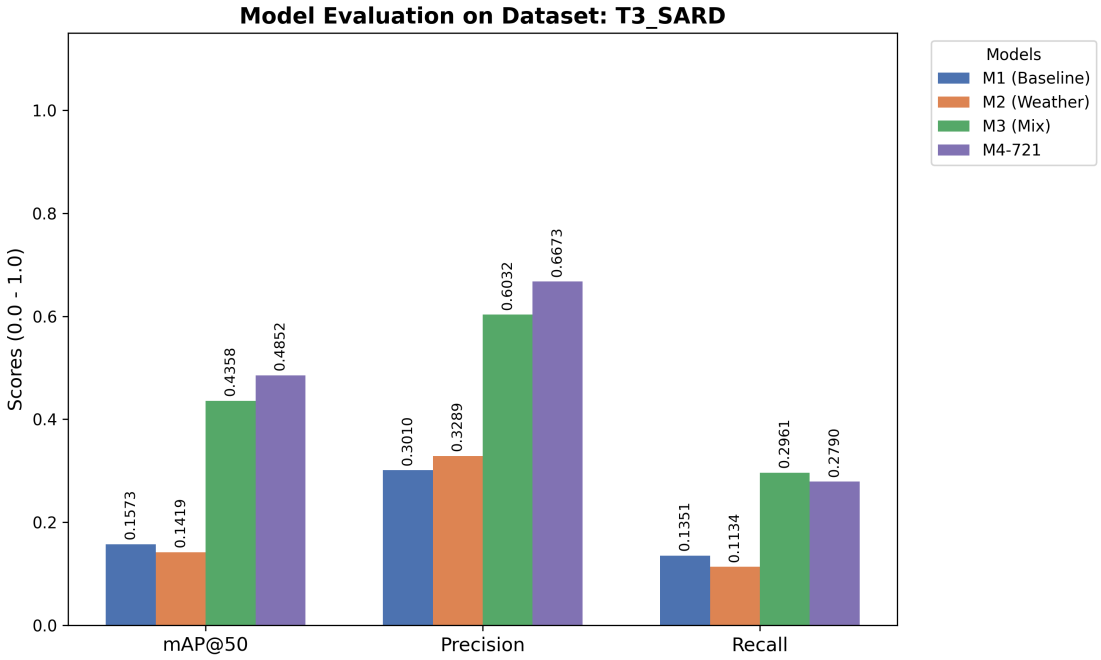
(d) *Model 3*



(e) *Model 4*

Hình 4.48: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*

Tập T3_SARD (Ảnh Drone cứu hộ - Unseen)



Hình 4.49: *Hiệu suất các mô hình trên tập dữ liệu T3_SARD*

Tập dữ liệu này đóng vai trò quan trọng trong việc đánh giá khả năng tổng quát hóa của mô hình khi triển khai trong môi trường thực tế (domain transfer).

- **M1 & M2 (Nhóm Baseline):** Tiếp tục cho thấy sự hạn chế với mAP@50 chỉ dao động trong khoảng 0.14–0.15 và Recall rất thấp ($\sim$ 0.11–0.13). Các mô hình này không có khả năng tổng quát hóa sang môi trường mới do chỉ học từ dữ liệu tổng hợp.
- **M3 (Mix – Không thời tiết):** Khẳng định hiệu quả của chiến lược trộn dữ liệu với mAP@50 = 0.4358 và Recall = 0.2961 (cao nhất trong nhóm). Việc bổ sung dữ liệu người thật giúp mô hình thích nghi tốt hơn với các điều kiện ngoài tập huấn luyện.
- **M4-721 (Mix + 30% Thời tiết):** Đạt hiệu suất cao nhất với mAP@50 = 0.4852 và Recall = 0.2790 Sự kết hợp giữa dữ liệu người thật và khả năng chống chịu nhiễu thời tiết giúp mô hình duy trì hiệu suất ổn định ngay cả khi chất lượng ảnh bị suy giảm.



(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*

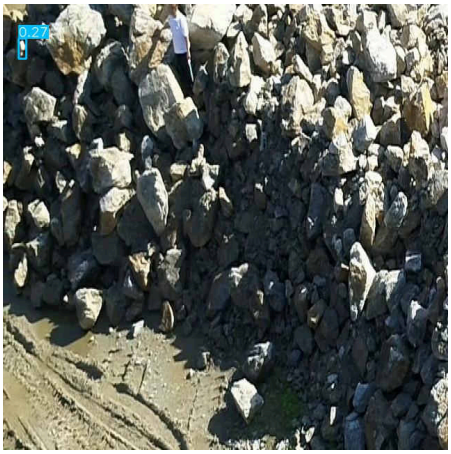


(d) *Model 3*



(e) *Model 4*

Hình 4.50: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



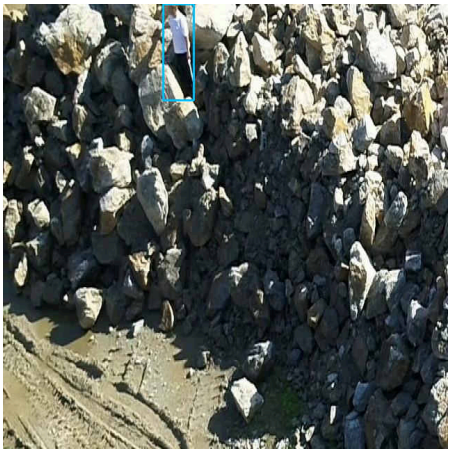
(a) *Model 1*



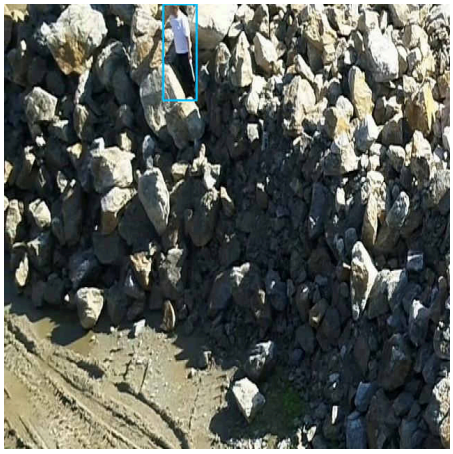
(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*



(e) *Model 4*

Hình 4.51: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*



(e) *Model 4*

Hình 4.52: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*



(a) *Model 1*



(b) *Model 2*



(c) *Ground Truth*



(d) *Model 3*

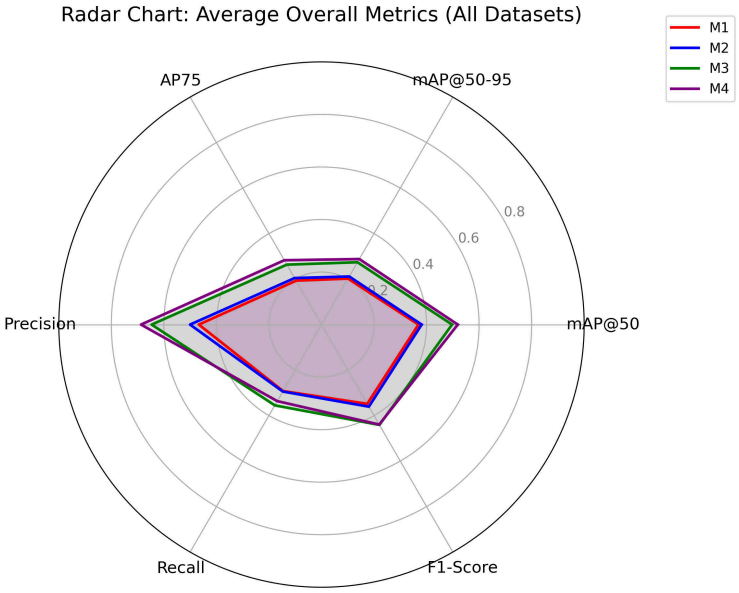


(e) *Model 4*

Hình 4.53: *Kết quả so sánh trực quan giữa đáp án và các mô hình đề xuất.*

4.7.4 Đánh giá chung

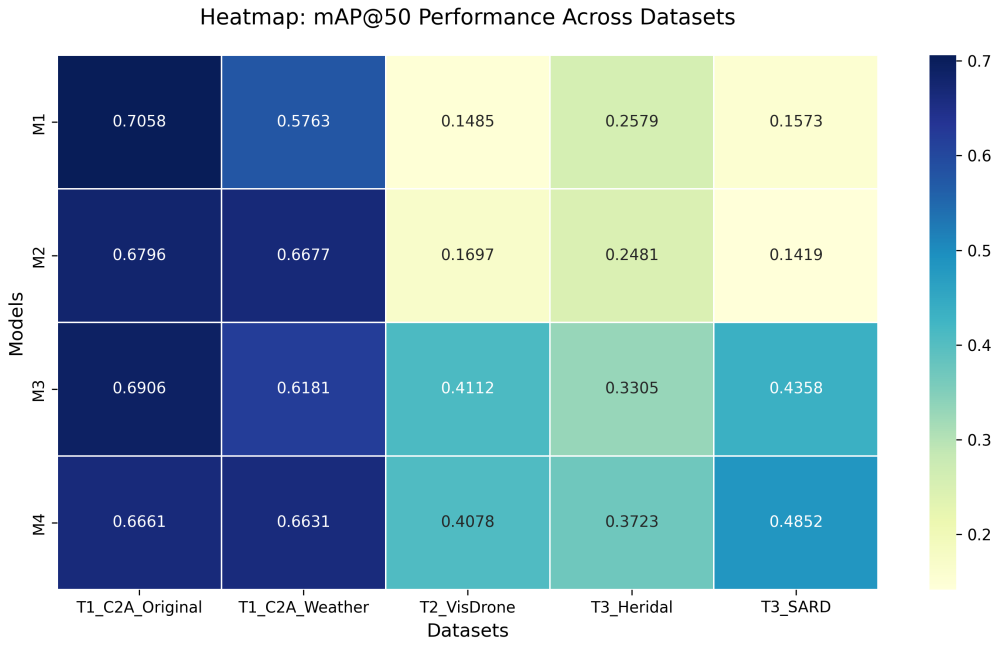
Để có cái nhìn tổng quan về sự chênh lệch hiệu suất và khả năng tổng quát hóa giữa các mô hình, nhóm tiến hành trực quan hóa các chỉ số đo lường thông qua hai biểu đồ dưới đây



Hình 4.54: *Biểu đồ Radar so sánh trung bình các chỉ số đánh giá toàn diện (mAP@50, mAP@50-95, AP75, Precision, Recall, F1-Score) của 4 mô hình trên toàn bộ các tập dữ liệu*

Dựa trên biểu đồ Radar ở Hình 4.54, có thể nhận thấy mô hình M4 có vùng bao phủ cân bằng và rộng nhất trong bốn mô hình trên hầu hết các chỉ số gồm mAP@50, mAP@50–95, AP75, Precision, Recall và F1-Score. Điều này cho thấy việc kết hợp dữ liệu hỗn hợp với tăng cường thời tiết giúp mô hình cải thiện đồng thời khả năng phát hiện mục tiêu và độ bền vững khi đánh giá trên nhiều miền dữ liệu khác nhau.

Mô hình M1 chỉ được huấn luyện trên tập C2A gốc nên đóng vai trò baseline. Kết quả của M1 còn hạn chế do mô hình dễ phụ thuộc vào đặc trưng của dữ liệu tổng hợp và khó tổng quát sang dữ liệu thực tế. M2 có bổ sung weather augmentation nên cải thiện phần nào khả năng chống chịu trong điều kiện nhiễu, tuy nhiên vẫn chưa giải quyết triệt để vấn đề chênh lệch miền dữ liệu. Trong khi đó, M3 cho thấy sự cải thiện rõ rệt hơn nhờ trộn thêm các bộ dữ liệu thực tế như VisDrone, TinyPerson và CrowdHuman, giúp mô hình học được đặc trưng người đa dạng hơn.



Hình 4.55: *Biểu đồ Heatmap thể hiện sự biến thiên của chỉ số mAP@50 qua các tập dữ liệu.*

1. **Nhóm mô hình huấn luyện trên dữ liệu tổng hợp (M1 & M2)**

   - **M1 (Baseline):** M1 tuy đạt điểm rất tốt trên C2A nhưng khi mang sang ảnh người thật thì thất bại nặng nề. Mô hình đã "học vẹt"các vết cắt ghép/ảo ảnh do thuật toán gỡ nền thay vì học được hình thái thực sự của con người.
   - **M2 (Thêm nhiễu thời tiết):** Việc bổ sung các phép biến đổi thời tiết (weather augmentation) nhằm cải thiện độ đa dạng của mô hình. Tuy nhiên, M2 vẫn không cải thiện đáng kể trên dữ liệu thực. Lý do là các nhiễu thời tiết chỉ làm biến đổi bề mặt dữ liệu tổng hợp, mà không cung cấp thông tin về đặc trưng của con người trong môi trường thực tế.

2. **Nhóm mô hình huấn luyện trên dữ liệu hỗn hợp (M3 & M4)**

   - **M3 (Trộn dữ liệu thực):** Để khắc phục khoảng cách miền, M3 được huấn luyện trên tập dữ liệu hỗn hợp giữa C2A và các bộ dữ liệu người thật (CrowdHuman, TinyPerson, VisDrone). Nhờ đó, mô hình học được các đặc điểm hình thái thực tế của con người và cải thiện đáng kể khả năng tổng quát hóa.
   - **M4 (Kết hợp tối ưu):** M4 kết hợp cả hai chiến lược: trộn dữ liệu thực và áp dụng nhiễu thời tiết. Đây được xem là cấu hình tối ưu, khi vừa giải quyết được khoảng cách miền thông qua dữ liệu thực, vừa nâng cao khả năng detect trong các điều kiện môi trường bất lợi. Kết quả thực nghiệm cho thấy M4 đạt hiệu suất vượt trội và ổn định trên nhiều tập kiểm thử khác nhau.

Chương 5

Hiện thực và đánh giá hệ thống

5.1 Đánh giá hiệu năng thực nghiệm trên Raspberry Pi 5

5.1.1 Mục tiêu

Đánh giá hiệu năng chạy suy luận của mô hình `M4-20k.onnx` trên thiết bị nhúng (dữ liệu kết quả lưu trong hai thư mục `m4`) theo hai chế độ:

- **Chế độ Video**: xử lý theo từng frame.
- **Chế độ Ảnh**: xử lý độc lập từng ảnh.

Ngoài tốc độ suy luận, báo cáo cũng thống kê số lượng đối tượng mà mô hình trả về (`detections`) để nhận xét khả năng phát hiện theo điều kiện thử nghiệm.

5.1.2 Dữ liệu và thiết lập thử nghiệm

Chế độ Video

- Nguồn video: `video_người_nhỏ_trong_lũ.mp4`
- Số mẫu (frame): 182

Chế độ Ảnh

- Nguồn ảnh: `test/images`
- Số mẫu (ảnh): 1123

5.1.3 Chỉ số đánh giá

Các chỉ số thống kê chính trích từ CSV:

- **Thời gian suy luận mỗi mẫu**: `time_ms` (ms)
- **Tốc độ suy luận**: `fps` (tính từ thời gian suy luận)
- **Số lượng detection**: `detections`

5.1.4 Kết quả benchmark chế độ Video

Thống kê tổng quan

Chỉ số	Giá trị
Samples	182
Total time (s)	10.2618
`fps_overall`	17.7356
`fps mean`	19.1248
`fps min / max`	6.4129 / 28.1121

Nhận xét: Trong bối cảnh "người nhỏ trong lũ", mô hình thường trả về số detection thấp; đa số frame không phát hiện đối tượng theo cách mà hệ thống ghi nhận trong CSV.

5.1.5 Kết quả benchmark chế độ Ảnh

Thống kê tổng quan

Chỉ số	Giá trị
Samples	1123
Total time (s)	51.8935
`fps_overall`	21.6405
`fps mean`	22.9613
`fps min / max`	4.9180 / 33.2466

5.1.6 So sánh Video vs Ảnh

- **Hiệu năng suy luận:** chế độ Ảnh nhanh hơn chế độ Video.
  - Video: `time_ms mean = 56.38 ms`, `fps mean = 19.12`
  - Ảnh: `time_ms mean = 46.21 ms`, `fps mean = 22.96`
- **Mức độ detection:** chế độ Ảnh có `detections mean = 2.0703`, cao hơn Video (`0.5549`).



Hình 5.1: *Hình ảnh các thảm họa về ngập lụt - 1*



Hình 5.2: *Hình ảnh các thảm họa về ngập lụt - 2*

5.1.7 Kết luận

- Về **tốc độ suy luận**, chế độ **ảnh** chạy nhanh hơn chế độ **video** (thời gian trung bình thấp hơn và FPS trung bình cao hơn).
- Về **mức độ detection**, chế độ **ảnh** có `detections mean` cao hơn chế độ **video**.
- Về **tác động thời tiết**, mô hình suy giảm rõ rệt trong điều kiện **mưa nặng** (`rain_hard`), thể hiện qua `detections mean` giảm và `zero rate` tăng.

5.2 Đánh giá hiệu năng thực nghiệm và cải thiện hiệu suất khi chạy thuần CPU trên Raspberry Pi-4B

5.2.1 Mục tiêu

Phần này tóm tắt quá trình chạy suy luận mô hình YOLOv8 (M4-721) định dạng NCNN trên Raspberry Pi 4B và so sánh hai chế độ:

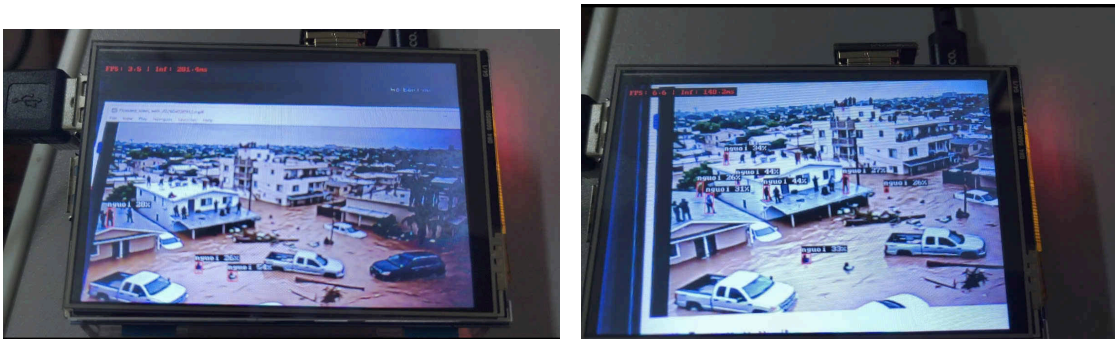
- **CPU cơ bản**: cấu hình không tối ưu (`OMP_NUM_THREADS=1`).
- **CPU tối ưu**: cấu hình đa luồng (`OMP_NUM_THREADS=4`) theo pipeline EdgeVisionRT.

Mục tiêu là đánh giá chênh lệch tốc độ trên cùng tập dữ liệu.

5.2.2 Kết quả

Chỉ số	CPU tối ưu	CPU baseline
FPS overall	6.07	3.45
Time mean (ms/frame)	164.75	290.05
Detections mean (box/ảnh)	3.97	3.97

Bảng 5.1: *So sánh hiệu năng giữa hai chế độ chạy.*



Hình 5.3: *So sánh trực quan kết quả phát hiện đối tượng giữa hai cấu hình: CPU đơn luồng (baseline) và CPU đa luồng tối ưu trên Raspberry Pi 4B*

5.2.3 Phân tích Ý nghĩa của logic tăng tốc CPU tối ưu tầng thấp (Kernel & SIMD)

Nghiên cứu gần đây của dự án EdgeVisionRT [14] cho thấy còn một lớp tối ưu sâu hơn ở tầng hệ điều hành và vi kiến trúc CPU. Theo góc nhìn kỹ thuật, nút thắt cổ chai hiệu năng trên thiết bị nhúng thường không chỉ đến từ việc "thiếu GHz" hay do bản thân mô hình AI, mà xuất phát từ **đường đi dữ liệu (data path)**. Các chi phí ngầm này bao gồm: chi phí truy cập bộ nhớ (cache miss, giới hạn băng thông RAM), các lớp trung gian không cần thiết (sao chép dữ liệu giữa kernel và user space), và sự thiếu hiệu quả của ngôn ngữ cấp cao (ví dụ Python/GIL làm giảm khả năng tận dụng prefetch và lệnh vector).

Vì vậy, ý nghĩa cốt lõi của việc tăng tốc CPU ở đây là **làm cho mỗi khung hình đi qua ít chu kỳ máy hơn** cho cùng một mô hình và cùng một đầu ra số học. Để giải quyết các cổ chai trên, logic tối ưu hóa được cụ thể hóa qua các tầng sau:

- **Tối ưu Pipeline và Quản lý bộ nhớ:**
  - *Zero-copy với V4L2 (MMAP):* Cho phép camera ghi trực tiếp vào buffer dùng cho suy luận, loại bỏ hoàn toàn chi phí `memcpy` rườm rà.
  - *Giảm phân bổ động trên đường nóng (hot path):* Các buffer tiền xử lý được khởi tạo sớm (pre-allocation), tránh việc gọi `malloc` lặp lại ở mỗi khung hình, giúp ổn định độ trễ và tăng tỷ lệ trúng cache (cache hit rate).

- **Vector hóa (Vectorization) bằng ARM NEON:** Viết lại bước tiền xử lý (resize, letterbox, chuẩn hóa, chuyển đổi YUYV/BGR → CHW tensor) bằng các tập lệnh SIMD. Cách tiếp cận này giúp xử lý nhiều pixel trong một chu kỳ máy, giảm thiểu các vòng lặp tuần tự tổng quát và truy cập bộ nhớ rời rạc so với pipeline thuần OpenCV/Python.
- **Tối ưu luồng và Lịch trình (Threading & Scheduling):**
  - *Suy luận NCNN đa luồng:* Khai thác song song phần tính toán nặng của mạng trên nhiều nhân Cortex-A72 của Pi 4B thông qua biến môi trường `OMP_NUM_THREADS`.
  - *CPU affinity:* Ghim các luồng xử lý vào các nhân CPU cố định nhằm tránh hiện tượng cache invalidation do scheduler của hệ điều hành gây ra.

Hệ quả đo được: Khi áp dụng các kỹ thuật trên (ví dụ tăng `OMP_NUM_THREADS` từ 1 lên 4), thời gian xử lý mỗi khung hình giảm rõ rệt nhưng **chỉ số mAP gần như không đổi** vì các bước trên không làm thay đổi công thức suy luận hay trọng số mạng. Chúng giúp giảm thời gian tiền xử lý từ vài chục mili-giây xuống mức tối thiểu, tăng thông lượng (throughput) và giảm độ trễ end-to-end, cực kỳ phù hợp cho triển khai thời gian thực trên thiết bị edge.

Liên hệ với thực nghiệm: Trong phạm vi dự án này, pipeline trên Raspberry Pi 4B đã được tối ưu ở mức C++/NCNN và đa luồng, do đó đạt được mức cải thiện $\sim 1.76\times$. Điều này củng cố nhận định: Tối ưu phần mềm theo chiều sâu kiến trúc có thể khai thác tối đa khả năng của thiết bị edge, ngay cả với nền tảng bị giới hạn tài nguyên như Raspberry Pi 4B.

5.2.4 Kết luận

Thực nghiệm trên Raspberry Pi 4B cho thấy pipeline tối ưu CPU của EdgeVisionRT mang lại cải thiện hiệu năng (gần 1.76 lần) so với baseline đơn luồng. Tuy nhiên, chất lượng phát hiện trên bộ Visdrone còn hạn chế ở nhóm đối tượng nhỏ. Hướng cải tiến tiếp theo gồm: tinh chỉnh ngưỡng confidence/NMS, đánh giá thêm mô hình đã huấn luyện.

5.3 Phân tích lựa chọn kiến trúc và độ phân giải cho mô hình nhận dạng trên Edge TPU

5.3.1 Mục tiêu

Mục tiêu của phần nghiên cứu là tìm hiểu và phân tích ảnh hưởng của hai yếu tố quan trọng là kiến trúc mô hình (Model Architecture) và độ phân giải đầu vào đến hiệu năng của hệ thống nhận dạng đối tượng khi chạy trên thiết bị Edge TPU.

Cụ thể, đề tài hướng đến các mục tiêu sau:

- Tìm hiểu các giới hạn phần cứng của Edge TPU và cách chúng ảnh hưởng đến quá trình chạy mô hình.

- So sánh hiệu năng giữa các kiến trúc mô hình YOLO ở các quy mô khác nhau (Nano, Small) dựa trên các tiêu chí như FPS, độ trễ (Latency).
- Phân tích ảnh hưởng của độ phân giải đầu vào đến tốc độ xử lý và khả năng nhận diện các đối tượng nhỏ.
- Xác định sự đánh đổi (trade-off) giữa tốc độ xử lý và độ chính xác trong bối cảnh triển khai thực tế.

Thông qua đó, đưa ra cơ sở để lựa chọn mô hình phù hợp khi triển khai trên các thiết bị có tài nguyên hạn chế như Edge TPU.

5.3.2 Google Coral Edge TPU và Định Mức Phần Cứng (Hardware Constraints)

Google Coral Edge TPU là bộ đồng xử lý chuyên dụng ASIC, được thiết kế thực thi riêng biệt cho tính toán ma trận. Dù có sức mạnh 4 TOPS với chi phí điện năng thấp, cấu trúc phần cứng này mang một số ràng buộc:

- **Tính toán lượng tử hóa INT8 (Full Integer Quantization):** Bộ vi xử lý này chỉ nhận các phép toán qua số nguyên 8-bit. Quá trình biến đổi độ chính xác từ Float32 sang INT8 khiến trọng số nhẹ hơn (khoảng $\frac{1}{4}$ dung lượng), giúp load bộ nhớ nhanh hơn, nhưng đổi lại sẽ có sự mất mát nhất định về độ chính xác.
- **Bottleneck về băng thông:** TPU sở hữu một lượng SRAM nội bộ rất nhỏ (khoảng 8MB). Việc truyền dữ liệu từ RAM của host (ví dụ Raspberry Pi) qua cổng USB đến TPU thường gây ra độ trễ nếu kích thước Tensor quá lớn.

Ảnh hưởng của kích thước mô hình (Nano vs Small)

Các mô hình YOLO thường được thiết kế với nhiều kích thước khác nhau, chủ yếu khác nhau ở số lượng tham số (parameters) và độ phức tạp tính toán (FLOPs). Trong đó, hai phiên bản phổ biến là Nano (n) và Small (s).

Tiêu chuẩn	Nano (n)	Small (s)
Độ sâu mạng	Cấu trúc nhẹ, ít block layer ẩn	Sâu hơn, lượng parameter và channel dày đặc
Số lượng parameters	2 - 3 triệu tham số (∼3MB)	11 - 12 triệu tham số (∼12MB)
Quản lý SRAM	Có thể load hoàn toàn vào SRAM của TPU	Dễ bị tràn SRAM
Hiệu suất Thực tiễn	Độ trễ (Latency) thấp, FPS cao nhưng dễ nhận diện sai hoặc bỏ sót các đối tượng nhỏ trong nền phức tạp	Nhận diện tốt hơn (mAP cao), đặc biệt với đối tượng bị che khuất, nhưng FPS giảm do chi phí tính toán lớn.

Nhận xét:

- Model Nano tận dụng tốt SRAM nên đạt tốc độ cao, nhưng dễ nhận diện sai.
- Model Small có thể gây swap bộ nhớ, làm tăng latency đáng kể, nhưng nhận diện tốt hơn.

5.3.3 Ảnh Hưởng Của Chiều Phân Giải Đầu Vào (Input Resolution: 320, 416, 512, 640)

Quy luật bình phương của khối lượng tính toán (Quadratic Complexity)

Độ phức tạp của các lớp tích chập trong một mạng Neural Network biến thiên tỷ lệ thuận với tỷ lệ với diện tích ảnh đầu vào:

$$\text{FLOPs} \propto \text{Width} \times \text{Height}$$

Khi tăng độ phân giải từ 320×320 lên 640×640:

$$\frac{640 \times 640}{320 \times 320} = 4$$

Tăng độ phân giải từ 320 lên 640 (gấp đôi cả chiều dài và chiều rộng) sẽ khiến số lượng phép tính tăng theo cấp số nhân (gấp 4 lần). Thời gian đẩy khối tensor này vào TPU cũng phình to dẫn tới thời gian suy diễn có thể bị nhân gấp 4.

Đánh đổi giữa tốc độ (FPS) và khả năng quan sát chi tiết

Trong quá trình xử lý ảnh, độ phân giải đầu vào (resolution) ảnh hưởng trực tiếp đến tốc độ xử lý và khả năng nhận diện các đối tượng nhỏ. Đây là một bài toán đánh đổi (trade-off) phổ biến trong thị giác máy tính.

- **Với độ phân giải thấp** (320×320)**:**
  - **Ưu điểm:** Kích thước ảnh nhỏ nên lượng dữ liệu đầu vào ít, giúp quá trình truyền dữ liệu từ CPU sang TPU diễn ra nhanh, từ đó đạt được FPS cao.
  - **Nhược điểm:** Dễ mất thông tin không gian (spatial loss). Các đối tượng ở xa hoặc kích thước nhỏ có thể bị biến mất sau khi resize, dẫn đến việc mô hình không thể nhận diện được.
- **Với độ phân giải cao** (512×512 **hoặc** 640×640)**:**
  - **Ưu điểm:** Giữ lại được nhiều chi tiết ảnh hơn, đặc biệt hữu ích khi cần phát hiện các đối tượng nhỏ hoặc ở xa.
  - **Nhược điểm:** Lượng dữ liệu lớn làm tăng thời gian xử lý, đồng thời có thể gây nghẽn trong quá trình truyền dữ liệu (ví dụ qua USB), dẫn đến giảm FPS.

5.3.4 Tổng kết cơ sở so sánh cho quá trình thực nghiệm

Dựa trên các phân tích lý thuyết ở trên, trước khi tiến hành thực nghiệm và đánh giá số liệu thực tế, có thể đưa ra một số nhận định ban đầu như sau:

1. Mô hình YOLO_Nano + độ phân giải 320 được kỳ vọng sẽ đạt FPS cao nhất. Nguyên nhân là do số lượng tham số nhỏ và dữ liệu đầu vào gọn nhẹ, giúp tận dụng tối đa băng thông bộ nhớ của TPU.

2. Mô hình YOLO_Small + độ phân giải 512/640 có khả năng sẽ cho tốc độ thấp nhất. Điều này đến từ việc mô hình có cấu trúc phức tạp hơn, dễ gây quá tải bộ nhớ (SRAM), đồng thời khối lượng dữ liệu lớn làm tăng chi phí truyền tải qua giao tiếp I/O.

3. **Ý nghĩa đối với bài toán cứu hộ:** Trong các hệ thống tìm kiếm cứu nạn, yếu tố quan trọng nhất là hạn chế bỏ sót các đối tượng nhỏ trong không gian rộng. Do đó, có thể chấp nhận việc giảm FPS để đổi lấy độ chính xác cao hơn.

   Vì vậy, cần tìm ra một "điểm tối ưu", ví dụ như sử dụng YOLO_Nano kết hợp với độ phân giải trung bình đến cao (416 hoặc 512) hoặc YOLO_Small kết hợp với độ phân giải thấp . Cách này vừa giúp giữ được thông tin của các đối tượng nhỏ, vừa đảm bảo tốc độ xử lý đủ nhanh cho các tác vụ.

5.4 Kiểm định thông số các mô hình trên EDGE TPU

5.4.1 Bảng tổng hợp kiểm định thông số các mô hình trên Edge TPU

Mức phân giải 320x320 (Hiệu năng tốt nhất)

Ở độ phân giải này, các mạng nhỏ (nano) đều lọt trọn vào băng thông, mang lại FPS cao. YOLO26 Nano dẫn đầu về độ mượt hệ thống.

Tên Model	Chỉ số	Mean	Min	Max	Std
YOLOv8 Nano	FPS	30.29	11.41	46.49	12.05
	Inference Time	20.87 ms	18.77 ms	31.19 ms	2.58 ms
YOLO26 Nano	FPS	35.50	23.19	40.79	3.63
	Inference Time	22.95 ms	21.73 ms	30.94 ms	1.10 ms
YOLO26 Small	FPS	14.11	8.28	19.82	3.45
	Inference Time	52.85 ms	47.72 ms	67.96 ms	4.36 ms

Bảng 5.2: *Hiệu năng các mô hình tại độ phân giải 320x320*

Mức phân giải 416x416

Bắt đầu thấy sự phân hóa. Mạng Small tụt dốc, khiến thời gian inference tăng mạnh.

Tên Model	Chỉ số	Mean	Min	Max	Std
YOLOv8 Nano	FPS	16.46	10.29	28.08	5.28
	Inference Time	34.14 ms	30.88 ms	45.38 ms	2.35 ms
YOLO26 Small	FPS	2.90	2.65	3.01	0.08
	Inference Time	333.21 ms	325.42 ms	365.59 ms	7.35 ms

Bảng 5.3: *Hiệu năng các mô hình tại độ phân giải 416x416*

Lưu ý: Không có dữ liệu YOLO26 Nano ở mức phân giải này trong log.

Mức phân giải 512x512

Độ phân giải lớn phù hợp cho vật thể nhỏ, nhưng chi phí tính toán tăng mạnh.

Tên Model	Chỉ số	Mean	Min	Max	Std
YOLO26 Nano	FPS	14.26	10.81	16.96	1.38
	Inference Time	56.84 ms	53.16 ms	72.30 ms	2.90 ms
YOLOv8 Nano	FPS	12.74	9.28	19.48	1.26
	Inference Time	48.52 ms	45.24 ms	62.24 ms	2.24 ms
YOLO26 Small	FPS	1.75	1.66	1.78	0.03
	Inference Time	560.49 ms	549.75 ms	585.69 ms	7.75 ms

Bảng 5.4: *Hiệu năng các mô hình tại độ phân giải 512x512*

Mức phân giải 640x640 (Nặng nhất)

Đây là ngưỡng gây nghẽn nghiêm trọng (bottleneck) trên Edge TPU.

Tên Model	Chỉ số	Mean	Min	Max	Std
YOLOv8 Nano	FPS	8.59	6.48	10.74	0.64
	Inference Time	90.20 ms	83.97 ms	116.23 ms	4.59 ms
YOLO26 Nano	FPS	3.17	2.76	3.34	0.10
	Inference Time	299.87 ms	290.75 ms	336.58 ms	8.16 ms

Bảng 5.5: *Hiệu năng các mô hình tại độ phân giải 640x640*

5.4.2 Kết luận

Từ các kết quả thực nghiệm, có thể rút ra một số nhận định quan trọng:

- **Ngưỡng phần cứng (Bottleneck):** YOLO26 Small không phù hợp với Edge TPU ở độ phân giải lớn hơn 416x416, FPS giảm xuống mức không thể sử dụng thực tế.
- **Độ chính xác tại 320x320: YOLOv8 Nano** cho kết quả ổn định và chính xác nhất, tiếp theo là **YOLO26 Small**, cuối cùng là YOLO26 Nano.

- **Khả năng giữ hiệu năng ở độ phân giải cao:** YOLOv8 Nano duy trì được sự cân bằng giữa tốc độ và độ chính xác ở các mức 416 và 512, trong khi YOLO26 Nano và Small suy giảm đáng kể.

Kết luận tổng thể: YOLOv8 Nano là lựa chọn tối ưu cho hệ thống Raspberry Pi kết hợp Edge TPU nhờ sự cân bằng giữa tốc độ (FPS) và độ chính xác (Accuracy).

5.5 Hệ thống Dashboard và giám sát realtime

Trong bài toán tìm kiếm – cứu nạn, việc theo dõi thông tin theo thời gian thực là rất quan trọng để hỗ trợ đưa ra quyết định nhanh chóng. Trong đề tài này, nhóm xây dựng một hệ thống Dashboard nhằm mục đích hiển thị dữ liệu từ các thiết bị ống nhòm thông minh ngoài thực địa về trung tâm.

Thông qua Dashboard, người vận hành có thể theo dõi trực tiếp hình ảnh từ thiết bị, số lượng đối tượng được phát hiện, cũng như các thông số hoạt động của hệ thống như FPS, nhiệt độ CPU,... Qua đó giúp quá trình giám sát và điều phối trở nên hiệu quả hơn.

5.5.1 Kiến trúc hệ thống dashboard

Hệ thống được thiết kế theo mô hình Client – Server, gồm 3 thành phần chính:

- **Edge Node (Thiết bị biên):** Là thiết bị đặt ngoài hiện trường, bao gồm camera và vi xử lý (Raspberry Pi). Thiết bị này thực hiện thu nhận ảnh và chạy mô hình AI để phát hiện đối tượng.
- **Backend Server:** Được xây dựng bằng FastAPI, có nhiệm vụ nhận dữ liệu từ các thiết bị biên, xử lý và lưu trữ vào cơ sở dữ liệu.
- **Frontend Dashboard:** Là giao diện web giúp người dùng theo dõi dữ liệu, hiển thị video và biểu đồ theo thời gian thực.

5.5.2 Backend

Backend sử dụng FastAPI do hỗ trợ bất đồng bộ tốt và hiệu năng cao. Các thiết bị biên giao tiếp với server thông qua các API chính:

- **POST /metrics:** Gửi thông tin hệ thống như FPS, nhiệt độ CPU, RAM và số lượng đối tượng phát hiện.
- **POST /boxes:** Gửi tọa độ bounding box của đối tượng trong ảnh.
- **POST /ai-logs:** Gửi ảnh kết quả (ảnh gốc và ảnh đã detect).
- **POST /stream-config:** Đăng ký luồng video từ thiết bị.
- **GET APIs:** Dùng để frontend lấy dữ liệu hiển thị (latest, timeseries, history).

5.5.3 Cơ sở dữ liệu

Hệ thống sử dụng **SQLite** (`metrics.db`) kết hợp với thư viện ORM **SQLAlchemy** (`app/db.py`) để tối ưu hóa sự tinh gọn, dễ dàng triển khai. Cơ sở dữ liệu được chia thành 2 bảng chính:

- **Metric:** Lưu thông tin theo thời gian như FPS, nhiệt độ CPU, RAM và số lượng đối tượng phát hiện.
- **AILog:** Lưu các ảnh có phát hiện đối tượng để phục vụ kiểm tra và đánh giá lại.

5.5.4 Frontend dashboard

Frontend được xây dựng bằng HTML, CSS và JavaScript thuần. Giao diện tập trung vào việc hiển thị thông tin rõ ràng và dễ quan sát.

- Sử dụng Chart.js để vẽ biểu đồ theo thời gian.
- Hiển thị video trực tiếp từ thiết bị thông qua MJPEG stream.
- Hiển thị các thông số hệ thống và danh sách ảnh đã detect.

5.5.5 Luồng dữ liệu realtime

Quá trình xử lý dữ liệu diễn ra theo các bước:

1. Thiết bị biên thu nhận hình ảnh từ camera.
2. Mô hình AI thực hiện nhận dạng và tạo bounding box.
3. Dữ liệu (metrics, ảnh, bounding box) được gửi về server qua HTTP.
4. Backend lưu trữ dữ liệu vào database.
5. Frontend liên tục gọi API để cập nhật dữ liệu và hiển thị lên Dashboard.

5.5.6 Kết quả hiển thị

Khi hệ thống hoạt động, Dashboard cung cấp các thông tin chính:

- Video trực tiếp từ thiết bị.
- Biểu đồ số lượng đối tượng phát hiện theo thời gian.
- Các thông số hoạt động như FPS, CPU, latency.
- Danh sách ảnh đã phát hiện đối tượng.

Hình 5.4 minh họa giao diện dashboard mà nhóm đã xây dựng cho hệ thống giám sát realtime. Giao diện được thiết kế theo hướng trực quan, ưu tiên khả năng quan sát nhanh các thông tin quan trọng khi hệ thống đang vận hành ngoài thực địa.

Ở giao diện này, người dùng có thể theo dõi đồng thời nhiều thành phần như luồng video trực tiếp từ thiết bị, các chỉ số hiệu năng của hệ thống, biểu đồ thống kê theo thời gian và khu vực lưu các ảnh đã được phát hiện đối tượng. Cách tổ chức này giúp người vận hành vừa nắm được tình trạng hoạt động của thiết bị biên, vừa quan sát được kết quả nhận dạng một cách liên tục.



Hình 5.4: *Giao diện dashboard giám sát realtime của hệ thống*

Trong đó, ảnh bên trái thể hiện bố cục tổng quan của dashboard với các khối thông tin chính phục vụ giám sát hệ thống. Ảnh bên phải minh họa một trạng thái vận hành cụ thể, cho thấy cách dashboard hiển thị dữ liệu trực tiếp từ thiết bị, hỗ trợ người dùng phát hiện nhanh các thay đổi bất thường hoặc đánh giá chất lượng hoạt động của pipeline nhận dạng.Giao diện lịch sử log hỗ trợ phân trang và lọc dữ liệu theo lớp đối tượng, ngưỡng độ tin cậy tối thiểu, thời gian bắt đầu và thời gian kết thúc. Nhờ đó, người dùng có thể nhanh chóng truy xuất các sự kiện quan trọng, ví dụ như các khung hình có số lượng đối tượng cao hoặc các phát hiện đạt độ tin cậy lớn hơn một ngưỡng nhất định. Nút *View* ở từng dòng cho phép mở chi tiết frame tương ứng, hỗ trợ kiểm tra trực quan kết quả phát hiện và phục vụ quá trình phân tích lỗi của mô hình.

Chương 6

Tổng kết và hướng phát triển

6.1 Tổng kết

Trong đồ án này, nhóm đã nghiên cứu và xây dựng một hệ thống hỗ trợ nhận dạng mục tiêu trong ảnh thu được từ ống nhòm, tập trung vào bài toán phát hiện con người trong các kịch bản tìm kiếm – cứu nạn (Search and Rescue – SAR). Xuất phát từ nhu cầu thực tế là việc quan sát thủ công trong thời gian dài dễ dẫn đến mệt mỏi, bỏ sót mục tiêu hoặc phản ứng chậm, đề tài hướng đến việc ứng dụng trí tuệ nhân tạo để đóng vai trò như một "trợ lý quan sát thông minh", hỗ trợ người dùng phát hiện nhanh các đối tượng quan trọng trong khung hình.

Về mặt định hướng hệ thống, đồ án đã lựa chọn hướng triển khai trên nền tảng tính toán biên, cụ thể là Raspberry Pi và các phần cứng tăng tốc phù hợp. Đây là lựa chọn phù hợp với đặc thù của thiết bị cầm tay hoặc thiết bị quan sát hiện trường, nơi hệ thống cần có kích thước nhỏ gọn, tiêu thụ điện năng thấp, chi phí hợp lý và có khả năng xử lý tại chỗ mà không phụ thuộc hoàn toàn vào kết nối mạng. Bên cạnh đó, hệ thống cũng được thiết kế theo hướng có thể mở rộng, cho phép tích hợp dashboard giám sát từ xa nhằm phục vụ việc theo dõi, lưu trữ và đánh giá kết quả trong quá trình vận hành.

Trước hết, nhóm đã xây dựng phần cơ sở lý thuyết cho bài toán, bao gồm các nội dung về ảnh số, xử lý ảnh, ảnh RGB, ảnh hồng ngoại, học sâu, thuật toán YOLO và các chỉ số đánh giá mô hình như Precision, Recall, mAP và F1-score. Những nội dung này tạo nền tảng cho việc lựa chọn mô hình phát hiện đối tượng, thiết kế pipeline xử lý ảnh và đánh giá định lượng hiệu quả của hệ thống. Trong phạm vi hiện thực, đề tài tập trung chủ yếu vào ảnh RGB do các ràng buộc về phần cứng, chi phí và khả năng triển khai trên thiết bị nhúng.

Về mặt dữ liệu, nhóm đã khảo sát và sử dụng bộ dữ liệu C2A làm nền tảng ban đầu cho bài toán phát hiện con người trong bối cảnh thảm họa. Bộ dữ liệu này phù hợp với hướng nghiên cứu SAR vì chứa các bối cảnh như hỏa hoạn, lũ lụt, sập công trình và tai nạn giao thông. Tuy nhiên, trong quá trình phân tích, nhóm nhận thấy dữ liệu C2A có bản chất tổng hợp, được tạo bằng cách ghép đối tượng người vào nền ảnh thảm họa. Điều này dẫn đến nguy cơ mô hình học các đặc trưng không mong muốn như vết cắt ghép, vùng biên bất thường hoặc shortcut của dữ liệu tổng hợp, thay vì học đầy đủ đặc trưng hình dáng con người trong môi trường thực tế.

Để khắc phục hạn chế trên, nhóm đã thực hiện nhiều hướng mở rộng và tăng cường dữ liệu. Thứ nhất, nhóm áp dụng các kỹ thuật data augmentation dựa trên mô hình vật lý để mô phỏng các điều kiện thời tiết bất lợi như sương mù và mưa. Phương pháp này

không chỉ tạo thêm dữ liệu đa dạng mà còn giúp mô hình tiếp xúc với các tình huống gần với thực tế hơn, đặc biệt là trong các môi trường SAR thường bị ảnh hưởng bởi thời tiết xấu, khói bụi, độ tương phản thấp hoặc tầm nhìn bị suy giảm. Thứ hai, nhóm đã nghiên cứu chiến lược trộn dữ liệu, kết hợp C2A với các bộ dữ liệu người thật như VisDrone, TinyPerson và CrowdHuman. Hướng tiếp cận này giúp mô hình giảm phụ thuộc vào đặc trưng tổng hợp của C2A, đồng thời học thêm đặc trưng con người thật từ nhiều góc nhìn, kích thước và bối cảnh khác nhau.

Về mặt thực nghiệm mô hình, nhóm đã xây dựng và so sánh nhiều phiên bản mô hình, bao gồm mô hình baseline huấn luyện trên C2A gốc, mô hình có bổ sung weather augmentation, mô hình sử dụng mixed dataset và mô hình kết hợp cả mixed dataset với weather augmentation. Kết quả cho thấy mô hình chỉ huấn luyện trên dữ liệu C2A có thể đạt kết quả tốt trong miền dữ liệu gốc, nhưng khả năng tổng quát hóa sang dữ liệu thực còn hạn chế. Ngược lại, các mô hình M3 và M4, nhờ được bổ sung dữ liệu người thật và bối cảnh đa dạng hơn, đã cải thiện rõ rệt khả năng phát hiện trên các tập dữ liệu thực và các miền chưa từng thấy như VisDrone, Heridal và SARD.

Một kết quả quan trọng của đồ án là việc chứng minh vai trò của chiến lược dữ liệu đối với bài toán SAR. Trong bài toán phát hiện người ở điều kiện thực tế, chỉ tối ưu mô hình hoặc tăng số lượng ảnh tổng hợp là chưa đủ. Mô hình cần được huấn luyện trên tập dữ liệu có tính đại diện cao, bao gồm nhiều kích thước đối tượng, góc nhìn, điều kiện ánh sáng, bối cảnh nền và mức độ che khuất khác nhau. Kết quả thực nghiệm cho thấy mixed-dataset strategy là một hướng tiếp cận có giá trị, giúp mô hình học được đặc trưng tổng quát của con người thay vì chỉ ghi nhớ đặc điểm của một miền dữ liệu cụ thể.

Bên cạnh phần mô hình, nhóm cũng đã triển khai và đánh giá hệ thống trên phần cứng biên. Các thử nghiệm trên Raspberry Pi 5 cho thấy mô hình có thể thực hiện suy luận trên dữ liệu ảnh và video, từ đó đánh giá được tốc độ xử lý, số lượng detection và khả năng vận hành trong điều kiện tài nguyên hạn chế. Ngoài ra, nhóm đã thực hiện tối ưu pipeline chạy trên Raspberry Pi 4B bằng các kỹ thuật như sử dụng C++/NCNN, tối ưu đa luồng, giảm chi phí tiền xử lý và cải thiện hiệu suất CPU. Kết quả cho thấy việc tối ưu phần mềm có thể cải thiện đáng kể tốc độ xử lý mà không làm thay đổi trọng số mô hình, qua đó khẳng định tầm quan trọng của tối ưu hệ thống khi triển khai AI trên edge device.

Đối với hướng tăng tốc phần cứng, nhóm đã phân tích khả năng triển khai mô hình trên Google Coral Edge TPU. Việc đánh giá các yếu tố như lượng tử hóa INT8, độ phân giải đầu vào, kích thước mô hình và giới hạn băng thông giúp nhóm có cơ sở lựa chọn cấu hình phù hợp cho các thiết bị có tài nguyên hạn chế. Đây là bước quan trọng để chuyển từ mô hình chạy thử nghiệm trên máy tính sang một prototype có khả năng vận hành thực tế, đáp ứng yêu cầu về tốc độ, độ trễ và tiêu thụ năng lượng.

Ngoài ra, nhóm đã xây dựng hệ thống dashboard nhằm giám sát kết quả nhận dạng theo thời gian thực. Hệ thống dashboard bao gồm các thành phần backend, cơ sở dữ liệu, frontend và luồng dữ liệu realtime. Việc tích hợp dashboard giúp hệ thống không chỉ dừng lại ở mức phát hiện đối tượng trên thiết bị cục bộ, mà còn có khả năng lưu trữ, theo dõi, phân tích và trực quan hóa kết quả từ xa. Đây là một thành phần có ý

nghĩa thực tiễn trong các kịch bản SAR, nơi nhiều thiết bị quan sát có thể được triển khai đồng thời và cần một trung tâm giám sát để tổng hợp thông tin.

Tổng hợp lại, đồ án đã đạt được các kết quả chính sau:

- Xây dựng được định hướng hệ thống nhận dạng mục tiêu trong ảnh ống nhòm phục vụ bài toán tìm kiếm – cứu nạn, trong đó hệ thống đóng vai trò hỗ trợ quan sát và không thay thế hoàn toàn quyết định của con người.
- Nghiên cứu và trình bày cơ sở lý thuyết liên quan đến xử lý ảnh, học sâu, YOLO, ảnh RGB/IR và các chỉ số đánh giá mô hình phát hiện đối tượng.
- Khảo sát, đánh giá và sử dụng bộ dữ liệu C2A cho bài toán SAR, đồng thời nhận diện được hạn chế của dữ liệu tổng hợp và nguy cơ shortcut learning.
- Xây dựng các kỹ thuật tăng cường dữ liệu mô phỏng điều kiện thời tiết xấu như sương mù, mưa và điều kiện quan sát ban đêm, góp phần cải thiện độ đa dạng của dữ liệu huấn luyện.
- Đề xuất và thực nghiệm chiến lược trộn dữ liệu giữa C2A và các bộ dữ liệu người thật như VisDrone, TinyPerson và CrowdHuman nhằm cải thiện khả năng tổng quát hóa của mô hình.
- Xây dựng ma trận thực nghiệm gồm nhiều mô hình M1, M2, M3 và M4 để đánh giá ảnh hưởng riêng biệt của dữ liệu gốc, weather augmentation, mixed dataset và sự kết hợp giữa hai hướng cải tiến.
- Đánh giá mô hình trên nhiều miền dữ liệu khác nhau, bao gồm dữ liệu gốc, dữ liệu có nhiễu thời tiết, dữ liệu thực và dữ liệu chưa từng thấy, từ đó rút ra nhận xét về độ bền vững và khả năng tổng quát hóa.
- Hiện thực và benchmark hệ thống trên Raspberry Pi 4B, Raspberry Pi 5 và định hướng triển khai trên Edge TPU, qua đó đánh giá được tính khả thi khi đưa mô hình AI lên nền tảng tính toán biên.
- Xây dựng dashboard giám sát realtime, hỗ trợ hiển thị kết quả, lưu trữ dữ liệu và theo dõi hoạt động của hệ thống từ xa.

Nhìn chung, kết quả đạt được cho thấy đề tài đã hoàn thành được mục tiêu nghiên cứu ban đầu ở mức prototype. Hệ thống đã chứng minh được tính khả thi của việc kết hợp mô hình YOLO, chiến lược dữ liệu phù hợp, tối ưu pipeline xử lý và phần cứng nhúng để xây dựng một giải pháp hỗ trợ phát hiện con người trong bối cảnh tìm kiếm – cứu nạn. Mặc dù chưa thể xem là một sản phẩm hoàn chỉnh để triển khai đại trà ngoài thực địa, đồ án đã tạo được nền tảng kỹ thuật quan trọng cho các bước phát triển tiếp theo.

6.2 Hạn chế của đề tài

Bên cạnh các kết quả đạt được, hệ thống hiện tại vẫn còn một số hạn chế cần được nhìn nhận rõ ràng để làm cơ sở cho hướng phát triển trong tương lai.

Thứ nhất, dữ liệu huấn luyện và đánh giá vẫn chưa hoàn toàn đại diện cho mọi điều kiện thực tế. Mặc dù nhóm đã sử dụng C2A, VisDrone, TinyPerson, CrowdHuman, Heridal và SARD, nhưng các tập dữ liệu này vẫn có sự khác biệt nhất định so với dữ liệu thu trực tiếp từ một thiết bị ống nhòm thật trong môi trường SAR. Dữ liệu thực tế có thể chịu ảnh hưởng bởi rung lắc, nhiễu cảm biến, méo quang học, thay đổi tiêu cự, chất lượng ống kính, điều kiện ánh sáng phức tạp và chuyển động của người vận hành. Những yếu tố này chưa được mô phỏng đầy đủ trong dữ liệu hiện tại.

Thứ hai, hệ thống hiện chủ yếu tập trung vào ảnh RGB. Ảnh RGB có ưu điểm là dễ thu thập, chi phí thấp và giàu thông tin màu sắc, nhưng lại phụ thuộc mạnh vào điều kiện ánh sáng. Trong các tình huống ban đêm, khói dày, mưa lớn hoặc sương mù nặng, ảnh RGB có thể suy giảm chất lượng đáng kể. Mặc dù nhóm đã mô phỏng một số điều kiện bất lợi thông qua data augmentation, việc chưa tích hợp cảm biến hồng ngoại hoặc ảnh nhiệt vẫn là một giới hạn đối với khả năng vận hành trong môi trường khắc nghiệt.

Thứ ba, độ chính xác của mô hình trên các tập dữ liệu unseen domain vẫn còn cần cải thiện. Kết quả thực nghiệm cho thấy các mô hình mixed dataset cải thiện đáng kể so với baseline, nhưng bài toán phát hiện người nhỏ, xa, bị che khuất hoặc xuất hiện trong nền phức tạp vẫn là một thách thức lớn. Trong SAR, việc bỏ sót người có thể gây hậu quả nghiêm trọng, do đó hệ thống cần tiếp tục được cải thiện về Recall, đồng thời vẫn kiểm soát được số lượng cảnh báo nhầm.

Thứ tư, hiệu năng thời gian thực trên thiết bị nhúng vẫn phụ thuộc mạnh vào cấu hình mô hình, độ phân giải đầu vào và mức độ tối ưu pipeline. Raspberry Pi có ưu điểm về chi phí và khả năng tiếp cận, nhưng tài nguyên CPU, RAM và băng thông vẫn còn hạn chế so với GPU máy tính. Khi tăng độ phân giải đầu vào để cải thiện khả năng phát hiện các mục tiêu nhỏ, thời gian suy luận cũng tăng tương ứng. Do đó, việc cân bằng giữa độ chính xác và tốc độ xử lý vẫn là một bài toán quan trọng.

Thứ năm, hệ thống dashboard hiện mới dừng ở mức hỗ trợ giám sát và trực quan hóa kết quả. Các chức năng nâng cao như cảnh báo tự động theo mức độ nguy hiểm, quản lý nhiều thiết bị đồng thời, định vị mục tiêu trên bản đồ, đồng bộ thời gian giữa các thiết bị và phân quyền người dùng vẫn cần được phát triển thêm để đáp ứng các kịch bản triển khai thực tế.

Cuối cùng, hệ thống chưa được kiểm thử toàn diện trong môi trường thực địa. Các đánh giá hiện tại chủ yếu dựa trên tập dữ liệu có sẵn và các thử nghiệm benchmark cục bộ. Để đánh giá đầy đủ độ ổn định, độ bền và khả năng sử dụng của hệ thống, cần thực hiện các thử nghiệm với camera thật, thiết bị ống nhòm thật, nhiều điều kiện môi trường khác nhau và nhiều tình huống vận hành kéo dài.

6.3 Hướng phát triển

Từ những hạn chế trên, đề tài có thể được tiếp tục phát triển theo nhiều hướng nhằm nâng cao độ chính xác, tính ổn định và khả năng ứng dụng thực tế.

6.3.1 Mở rộng và chuẩn hóa dữ liệu thực tế

Hướng phát triển quan trọng nhất là xây dựng thêm bộ dữ liệu thực tế được thu trực tiếp từ thiết bị quan sát hoặc camera gắn với ống nhòm. Bộ dữ liệu này nên bao gồm nhiều kịch bản SAR khác nhau như vùng lũ, rừng núi, khu vực sạt lở, công trình đổ nát, đô thị đông người và khu vực có nhiều vật cản. Ngoài ra, dữ liệu cần được thu ở nhiều khoảng cách, độ cao, góc nhìn, điều kiện ánh sáng và điều kiện thời tiết khác nhau.

Quá trình gán nhãn cũng cần được chuẩn hóa để đảm bảo tính nhất quán. Đối với bài toán phát hiện người nhỏ trong ảnh, sai lệch nhỏ trong bounding box cũng có thể ảnh hưởng lớn đến chỉ số đánh giá. Vì vậy, cần xây dựng quy trình kiểm tra nhãn, đánh giá chéo giữa nhiều người gán nhãn và loại bỏ các mẫu không đạt chất lượng.

Bên cạnh đó, có thể tiếp tục mở rộng mixed dataset theo hướng cân bằng giữa dữ liệu tổng hợp và dữ liệu thực. Dữ liệu tổng hợp vẫn có giá trị vì giúp tạo ra nhiều tình huống hiếm gặp, nhưng cần được kiểm soát để tránh shortcut learning. Một hướng tiềm năng là sử dụng domain adaptation hoặc style transfer có kiểm soát để làm cho dữ liệu tổng hợp gần hơn với dữ liệu thực tế.

6.3.2 Cải tiến mô hình phát hiện đối tượng

Về mô hình, có thể tiếp tục thử nghiệm các kiến trúc YOLO mới hơn hoặc các biến thể chuyên cho đối tượng nhỏ. Bài toán SAR thường có đặc điểm là người xuất hiện rất nhỏ trong ảnh, dễ bị che khuất và lẫn vào nền. Do đó, mô hình cần có khả năng khai thác tốt đặc trưng ở nhiều tỉ lệ khác nhau.

Một số hướng cải tiến có thể xem xét bao gồm:

- Tăng cường khả năng phát hiện small object bằng cách điều chỉnh kích thước input, anchor, feature pyramid hoặc head phát hiện.
- Sử dụng attention mechanism để giúp mô hình tập trung tốt hơn vào vùng có khả năng chứa người.
- Tối ưu hàm mất mát để ưu tiên Recall trong các tình huống mà bỏ sót mục tiêu nguy hiểm hơn cảnh báo nhầm.
- Áp dụng knowledge distillation để huấn luyện mô hình nhỏ học từ mô hình lớn, từ đó giữ được độ chính xác tương đối cao nhưng vẫn phù hợp với edge device.
- Kết hợp tracking theo thời gian trong video để giảm bỏ sót giữa các frame và làm mượt kết quả phát hiện.

Ngoài ra, cần tiếp tục đánh giá mô hình bằng cross-dataset evaluation. Thay vì chỉ đánh giá trên tập test cùng phân phối với tập train, mô hình nên được kiểm thử trên nhiều tập dữ liệu khác miền để đo khả năng tổng quát hóa thực sự. Đây là tiêu chí đặc biệt quan trọng đối với hệ thống SAR, vì môi trường triển khai thực tế thường không thể dự đoán đầy đủ trong giai đoạn huấn luyện.

6.3.3 Tích hợp đa cảm biến và dữ liệu hồng ngoại

Một hướng phát triển có giá trị thực tiễn cao là tích hợp thêm cảm biến hồng ngoại hoặc ảnh nhiệt. Ảnh RGB phù hợp trong điều kiện đủ sáng, nhưng ảnh nhiệt hoặc IR có thể hỗ trợ tốt hơn trong ban đêm, khói nhẹ hoặc điều kiện ánh sáng yếu. Khi kết hợp RGB và IR, hệ thống có thể tận dụng ưu điểm của cả hai miền dữ liệu: RGB cung cấp chi tiết màu sắc và kết cấu, trong khi IR hỗ trợ phát hiện mục tiêu sống dựa trên chênh lệch nhiệt hoặc phản xạ hồng ngoại.

Có thể nghiên cứu ba chiến lược fusion:

- Early fusion: ghép kênh ảnh RGB và IR ngay từ đầu vào.
- Mid-level fusion: trích xuất đặc trưng riêng cho từng modal rồi kết hợp ở tầng đặc trưng.
- Late fusion: chạy hai mô hình riêng biệt và hợp nhất kết quả dự đoán ở tầng quyết định.

Trong bối cảnh edge device, late fusion hoặc mid-level fusion nhẹ có thể là lựa chọn thực tế hơn vì dễ kiểm soát độ trễ và tài nguyên tính toán. Tuy nhiên, việc tích hợp đa cảm biến cũng đặt ra các vấn đề mới như đồng bộ thời gian, hiệu chỉnh hình học giữa camera RGB và IR, xử lý sai lệch trường nhìn và tăng chi phí phần cứng.

6.3.4 Tối ưu triển khai trên thiết bị biên

Để hệ thống có thể vận hành ổn định ngoài thực tế, cần tiếp tục tối ưu hóa mô hình và pipeline trên thiết bị biên. Các hướng tối ưu bao gồm lượng tử hóa mô hình, pruning, sử dụng runtime nhẹ như NCNN, TensorRT, TFLite hoặc Edge TPU runtime, đồng thời tối ưu tiền xử lý ảnh bằng C++ và SIMD thay vì phụ thuộc hoàn toàn vào Python.

Đối với Raspberry Pi, cần tiếp tục nghiên cứu cân bằng giữa độ phân giải đầu vào và tốc độ suy luận. Độ phân giải cao giúp phát hiện người nhỏ tốt hơn, nhưng làm tăng chi phí tính toán. Vì vậy, có thể áp dụng chiến lược adaptive resolution, chỉ xử lý độ phân giải cao ở những vùng nghi ngờ hoặc khi hệ thống phát hiện có chuyển động bất thường.

Đối với Edge TPU, hướng phát triển tiếp theo là hoàn thiện quy trình chuyển đổi mô hình sang TFLite INT8, kiểm tra các lớp tương thích với compiler của Coral và lựa chọn kiến trúc mô hình phù hợp với giới hạn phần cứng. Mục tiêu là đạt được tốc độ suy luận cao, độ trễ thấp và mức tiêu thụ điện năng phù hợp với thiết bị cầm tay.

6.3.5 Hoàn thiện dashboard và khả năng vận hành hệ thống

Dashboard có thể được phát triển thêm theo hướng trở thành trung tâm giám sát cho nhiều thiết bị edge. Thay vì chỉ hiển thị kết quả detection, hệ thống có thể bổ sung các chức năng như:

- Hiển thị stream realtime từ nhiều thiết bị.
- Lưu lịch sử phát hiện theo thời gian.
- Cảnh báo khi phát hiện người với độ tin cậy vượt ngưỡng.
- Tích hợp bản đồ để đánh dấu vị trí phát hiện nếu thiết bị có GPS.
- Thống kê số lượng detection, confidence trung bình và trạng thái hoạt động của từng thiết bị.
- Quản lý người dùng, phân quyền truy cập và bảo mật dữ liệu.

Ngoài ra, dashboard cũng có thể hỗ trợ chế độ human-in-the-loop, cho phép người vận hành xác nhận hoặc bác bỏ kết quả phát hiện. Các phản hồi này có thể được lưu lại để phục vụ quá trình cải thiện dữ liệu và huấn luyện lại mô hình trong tương lai.

6.3.6 Thử nghiệm thực địa và đánh giá độ tin cậy

Bước phát triển cuối cùng nhưng rất quan trọng là thử nghiệm hệ thống trong môi trường thực địa. Các thử nghiệm nên được thiết kế theo nhiều cấp độ: thử nghiệm trong phòng, thử nghiệm ngoài trời có kiểm soát và thử nghiệm trong các kịch bản gần với SAR thực tế. Trong mỗi thử nghiệm, cần đo các chỉ số như FPS, độ trễ end-to-end, nhiệt độ thiết bị, mức tiêu thụ điện, độ ổn định khi chạy lâu, số lần phát hiện đúng, số lần bỏ sót và số lần cảnh báo nhầm.

Đặc biệt, đối với bài toán tìm kiếm – cứu nạn, chỉ số Recall cần được ưu tiên vì bỏ sót người là rủi ro nghiêm trọng. Tuy nhiên, Precision cũng cần được kiểm soát để tránh gây quá tải cho người vận hành bởi quá nhiều cảnh báo giả. Do đó, hệ thống cần được hiệu chỉnh ngưỡng confidence và NMS theo từng bối cảnh cụ thể, thay vì sử dụng một ngưỡng cố định cho mọi tình huống.

Tài liệu tham khảo

[1] Gonzalez, R. C. và Woods, R. E. Digital Image Processing. 4th Edition, Pearson, 2018.

[2] Tutorialspoint. Digital Image Processing Tutorial. Truy cập tại: https://www.tutorialspoint.com/dip/index.htm.

[3] Nihal và cộng sự. UAV-Enhanced C2A Dataset for Human Detection in Disaster Scenarios. 2024.

[4] Nihal và cộng sự. C2A: Human Detection in Disaster Scenarios Dataset. 2024.

[5] Gupta và cộng sự. Robust Object Detection in Adverse Weather via Analytical Data Augmentation. 2024.

[6] R. Geirhos và cộng sự. Shortcut Learning in Deep Neural Networks. Nature Machine Intelligence, 2020.

[7] Dong và cộng sự. KITTI-C: Robustness Benchmarking for Computer Vision under Corruptions. 2023.

[8] Pengfei Zhu, Longyin Wen, Xiao Bian, Haibin Ling, và Qinghua Hu. Vision Meets Drones: A Challenge. arXiv preprint arXiv:2101.11904, 2021.

[9] Jifeng Shao, Kai Chen, Wuyang Zhang, Li Wang, Zhiqiang Li, Xiaogang Wang, và Hongsheng Li. CrowdHuman: A Benchmark for Detecting Human in a Crowd. arXiv preprint arXiv:1805.00123, 2018.

[10] R. Ranftl, A. Bochkovskiy, V. Koltun. Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer. IEEE TPAMI, 2022.

[11] R. Ranftl, A. Bochkovskiy, V. Koltun. MiDaS: Monocular Depth Estimation via Mixed Dataset Training. 2022.

[12] H. Zhang và V. M. Patel. Towards Robust Image Dehazing with Deep Learning. 2017.

[13] H. Zhang và V. M. Patel. Fog Simulation and Image Degradation Modeling for Robust Vision. 2017.

[14] Nguyễn Tùng Dương, *Triển khai YOLOv8n trên Raspberry Pi 5: Tối ưu từ Kernel đến Assembly để đạt hiệu năng tối đa*, Viblo, 2026. https://viblo.asia/p/trien-khai-yolov8n-tren-raspberry-pi-5-toi-uu-tu-kernel-den-assembly-de-dat-hieu-nang-toi-da-bNVQGN3AJvR

[15] Real-Time Object Detection System with YOLO and CNN Models: A Review. Truy cập từ https://arxiv.org/pdf/2208.00773

[16] Google Coral. Coral USB Accelerator Documentation. Truy cập từ https://coral.ai/products/accelerator/

[17] Machine Learning Cơ Bản. Evaluation Metrics cho Classification. Truy cập từ https://machinelearningcoban.com/2017/08/31/evaluation/

[18] rgbnihal. C2A Dataset. Kaggle Dataset. Truy cập từ https://www.kaggle.com/datasets/rgbnihal/c2a-dataset

[19] NikolasGegenava. SARD - Search And Rescue Dataset. Kaggle Dataset, 2025. Truy cập từ https://www.kaggle.com/datasets/nikolasgegenava/sard-search-and-rescue

[20] NikolasGegenava. SARD - Search And Rescue Dataset. Kaggle Dataset, 2025. Truy cập từ https://www.kaggle.com/datasets/nikolasgegenava/sard-search-and-rescue

[21] Imad Eddine Lassakeur. HERIDAL - Human Rescue Imaging Database on Land. Kaggle Dataset. Truy cập từ https://www.kaggle.com/datasets/imadeddinelassakeur/heridal

[22] permanalwep. CrowdHuman Crowd Detection. Kaggle Dataset. Truy cập từ https://www.kaggle.com/datasets/permanalwep/crowdhuman-crowd-detection

[23] H. Gupta, O. Kotlyar, H. Andreasson, and A. J. Lilienthal. Robust Object Detection in Challenging Weather Conditions. In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2024, pp. 7508–7517. Truy cập từ https://openaccess.thecvf.com/content/WACV2024/papers/Gupta_Robust_Object_Detection_in_Challenging_Weather_Conditions_WACV_2024_paper.pdf

[24] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. S. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann. Shortcut Learning in Deep Neural Networks. arXiv preprint arXiv:2004.07780, 2020. Truy cập từ https://arxiv.org/pdf/2004.07780

[25] Ragib Amin Nihal, Benjamin Yen, Katsutoshi Itoyama, and Kazuhiro Nakadai. UAV-Enhanced Combination to Application: Comprehensive Analysis and Benchmarking of a Human Detection Dataset for Disaster Scenarios. arXiv preprint arXiv:2408.04922, 2024. Truy cập từ https://arxiv.org/pdf/2408.04922

[26] Chris D. TinyPerson Dataset. Roboflow Universe, 2025. Truy cập từ https://universe.roboflow.com/chris-d-dbyby/tinyperson

[27] S. Ramírez, "FastAPI Documentation," FastAPI, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[28] S. Ramírez, "WebSockets - FastAPI," FastAPI Documentation, 2026. [Online]. Available: https://fastapi.tiangolo.com/advanced/websockets/

[29] Encode OSS Ltd., "Uvicorn: An ASGI web server, for Python," Uvicorn Documentation, 2026. [Online]. Available: https://www.uvicorn.org/

[30] M. Bayer, "SQLAlchemy 2.0 Documentation," SQLAlchemy, 2026. [Online]. Available: https://docs.sqlalchemy.org/en/20/

[31] SQLite Consortium, "SQLite Documentation," SQLite, 2026. [Online]. Available: https://www.sqlite.org/docs.html

[32] Chart.js Contributors, "Chart.js Documentation," Chart.js, 2025. [Online]. Available: https://www.chartjs.org/docs/latest/

[33] Mozilla Developer Network, "Using server-sent events," MDN Web Docs, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

[34] OpenCV Team, "Getting Started with Videos," OpenCV Documentation, 2026. [Online]. Available: https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html